

73 LL ADC Generic Driver

73.1 ADC Firmware driver registers structures

73.1.1 LL_ADC_CommonInitTypeDef

LL_ADC_CommonInitTypeDef is defined in the stm32f4xx_ll_adc.h

Data Fields

- *uint32_t CommonClock*
- *uint32_t Multimode*
- *uint32_t MultiDMATransfer*
- *uint32_t MultiTwoSamplingDelay*

Field Documentation

- **uint32_t LL_ADC_CommonInitTypeDef::CommonClock**
Set parameter common to several ADC: Clock source and prescaler. This parameter can be a value of [ADC_LL_EC_COMMON_CLOCK_SOURCE](#). This feature can be modified afterwards using unitary function [LL_ADC_SetCommonClock\(\)](#).
- **uint32_t LL_ADC_CommonInitTypeDef::Multimode**
Set ADC multimode configuration to operate in independent mode or multimode (for devices with several ADC instances). This parameter can be a value of [ADC_LL_EC_MULTI_MODE](#). This feature can be modified afterwards using unitary function [LL_ADC_SetMultimode\(\)](#).
- **uint32_t LL_ADC_CommonInitTypeDef::MultiDMATransfer**
Set ADC multimode conversion data transfer: no transfer or transfer by DMA. This parameter can be a value of [ADC_LL_EC_MULTI_DMA_TRANSFER](#). This feature can be modified afterwards using unitary function [LL_ADC_SetMultiDMATransfer\(\)](#).
- **uint32_t LL_ADC_CommonInitTypeDef::MultiTwoSamplingDelay**
Set ADC multimode delay between 2 sampling phases. This parameter can be a value of [ADC_LL_EC_MULTI_TWOSMP_DELAY](#). This feature can be modified afterwards using unitary function [LL_ADC_SetMultiTwoSamplingDelay\(\)](#).

73.1.2 LL_ADC_InitTypeDef

LL_ADC_InitTypeDef is defined in the stm32f4xx_ll_adc.h

Data Fields

- *uint32_t Resolution*
- *uint32_t DataAlignment*
- *uint32_t SequencersScanMode*

Field Documentation

- **uint32_t LL_ADC_InitTypeDef::Resolution**
Set ADC resolution. This parameter can be a value of [ADC_LL_EC_RESOLUTION](#). This feature can be modified afterwards using unitary function [LL_ADC_SetResolution\(\)](#).
- **uint32_t LL_ADC_InitTypeDef::DataAlignment**
Set ADC conversion data alignment. This parameter can be a value of [ADC_LL_EC_DATA_ALIGN](#). This feature can be modified afterwards using unitary function [LL_ADC_SetDataAlignment\(\)](#).
- **uint32_t LL_ADC_InitTypeDef::SequencersScanMode**
Set ADC scan selection. This parameter can be a value of [ADC_LL_EC_SCAN_SELECTION](#). This feature can be modified afterwards using unitary function [LL_ADC_SetSequencersScanMode\(\)](#).

73.1.3 LL_ADC_REG_InitTypeDef

LL_ADC_REG_InitTypeDef is defined in the stm32f4xx_ll_adc.h

Data Fields

- *uint32_t TriggerSource*
- *uint32_t SequencerLength*
- *uint32_t SequencerDiscont*

- *uint32_t ContinuousMode*
- *uint32_t DMATransfer*

Field Documentation

- *uint32_t LL_ADC_REG_InitTypeDef::TriggerSource*
Set ADC group regular conversion trigger source: internal (SW start) or from external IP (timer event, external interrupt line). This parameter can be a value of [ADC_LL_EC_REG_TRIGGER_SOURCE](#)
Note:
 - On this STM32 series, setting of external trigger edge is performed using function [LL_ADC_REG_StartConversionExtTrig\(\)](#).
This feature can be modified afterwards using unitary function [LL_ADC_REG_SetTriggerSource\(\)](#).
- *uint32_t LL_ADC_REG_InitTypeDef::SequencerLength*
Set ADC group regular sequencer length. This parameter can be a value of [ADC_LL_EC_REG_SEQ_SCAN_LENGTH](#)
Note:
 - This parameter is discarded if scan mode is disabled (refer to parameter 'ADC_SequencersScanMode').
This feature can be modified afterwards using unitary function [LL_ADC_REG_SetSequencerLength\(\)](#).
- *uint32_t LL_ADC_REG_InitTypeDef::SequencerDiscont*
Set ADC group regular sequencer discontinuous mode: sequence subdivided and scan conversions interrupted every selected number of ranks. This parameter can be a value of [ADC_LL_EC_REG_SEQ_DISCONT_MODE](#)
Note:
 - This parameter has an effect only if group regular sequencer is enabled (scan length of 2 ranks or more).
This feature can be modified afterwards using unitary function [LL_ADC_REG_SetSequencerDiscont\(\)](#).
- *uint32_t LL_ADC_REG_InitTypeDef::ContinuousMode*
Set ADC continuous conversion mode on ADC group regular, whether ADC conversions are performed in single mode (one conversion per trigger) or in continuous mode (after the first trigger, following conversions launched successively automatically). This parameter can be a value of [ADC_LL_EC_REG_CONTINUOUS_MODE](#) **Note:** It is not possible to enable both ADC group regular continuous mode and discontinuous mode. This feature can be modified afterwards using unitary function [LL_ADC_REG_SetContinuousMode\(\)](#).
- *uint32_t LL_ADC_REG_InitTypeDef::DMATransfer*
Set ADC group regular conversion data transfer: no transfer or transfer by DMA, and DMA requests mode. This parameter can be a value of [ADC_LL_EC_REG_DMA_TRANSFER](#) This feature can be modified afterwards using unitary function [LL_ADC_REG_SetDMATransfer\(\)](#).

73.1.4

LL_ADC_INJ_InitTypeDef

LL_ADC_INJ_InitTypeDef is defined in the `stm32f4xx_ll_adc.h`

Data Fields

- *uint32_t TriggerSource*
- *uint32_t SequencerLength*
- *uint32_t SequencerDiscont*
- *uint32_t TrigAuto*

Field Documentation

- *uint32_t LL_ADC_INJ_InitTypeDef::TriggerSource*
Set ADC group injected conversion trigger source: internal (SW start) or from external IP (timer event, external interrupt line). This parameter can be a value of [ADC_LL_EC_INJ_TRIGGER_SOURCE](#)
Note:
 - On this STM32 series, setting of external trigger edge is performed using function [LL_ADC_INJ_StartConversionExtTrig\(\)](#).
This feature can be modified afterwards using unitary function [LL_ADC_INJ_SetTriggerSource\(\)](#).

- **`uint32_t LL_ADC_INJ_InitTypeDef::SequencerLength`**
Set ADC group injected sequencer length. This parameter can be a value of **`ADC_LL_EC_INJ_SEQ_SCAN_LENGTH`**
Note:
 - This parameter is discarded if scan mode is disabled (refer to parameter 'ADC_SequencersScanMode').

This feature can be modified afterwards using unitary function **`LL_ADC_INJ_SetSequencerLength()`**.
- **`uint32_t LL_ADC_INJ_InitTypeDef::SequencerDiscont`**
Set ADC group injected sequencer discontinuous mode: sequence subdivided and scan conversions interrupted every selected number of ranks. This parameter can be a value of **`ADC_LL_EC_INJ_SEQ_DISCONT_MODE`**
Note:
 - This parameter has an effect only if group injected sequencer is enabled (scan length of 2 ranks or more).

This feature can be modified afterwards using unitary function **`LL_ADC_INJ_SetSequencerDiscont()`**.
- **`uint32_t LL_ADC_INJ_InitTypeDef::TrigAuto`**
Set ADC group injected conversion trigger: independent or from ADC group regular. This parameter can be a value of **`ADC_LL_EC_INJ_TRIG_AUTO`** **Note:** This parameter must be set to independent trigger if injected trigger source is set to an external trigger. This feature can be modified afterwards using unitary function **`LL_ADC_INJ_SetTrigAuto()`**.

73.2 ADC Firmware driver API description

The following section lists the various functions of the ADC library.

73.2.1 Detailed description of functions

LL_ADC_DMA_GetRegAddr

Function name

```
__STATIC_INLINE uint32_t LL_ADC_DMA_GetRegAddr (ADC_TypeDef * ADCx, uint32_t Register)
```

Function description

Function to help to configure DMA transfer from ADC: retrieve the ADC register address from ADC instance and a list of ADC registers intended to be used (most commonly) with DMA transfer.

Parameters

- **ADCx:** ADC instance
- **Register:** This parameter can be one of the following values:
 - **`LL_ADC_DMA_REG_REGULAR_DATA`**
 - **`LL_ADC_DMA_REG_REGULAR_DATA_MULTI (1)`**

(1) Available on devices with several ADC instances.

Return values

- **ADC:** register address

Notes

- These ADC registers are data registers: when ADC conversion data is available in ADC data registers, ADC generates a DMA transfer request.
- This macro is intended to be used with LL DMA driver, refer to function "LL_DMA_ConfigAddresses()". Example: **`LL_DMA_ConfigAddresses(DMA1, LL_DMA_CHANNEL_1, LL_ADC_DMA_GetRegAddr(ADC1, LL_ADC_DMA_REG_REGULAR_DATA), (uint32_t)< array or variable >, LL_DMA_DIRECTION_PERIPH_TO_MEMORY);`**
- For devices with several ADC: in multimode, some devices use a different data register outside of ADC instance scope (common data register). This macro manages this register difference, only ADC instance has to be set as parameter.

Reference Manual to LL API cross reference:

- DR RDATA LL_ADC_DMA_GetRegAddr
- CDR RDATA_MST LL_ADC_DMA_GetRegAddr
- CDR RDATA_SLV LL_ADC_DMA_GetRegAddr

LL_ADC_SetCommonClock

Function name

```
__STATIC_INLINE void LL_ADC_SetCommonClock (ADC_Common_TypeDef * ADCxy_COMMON,
uint32_t CommonClock)
```

Function description

Set parameter common to several ADC: Clock source and prescaler.

Parameters

- **ADCxy_COMMON:** ADC common instance (can be set directly from CMSIS definition or by using helper macro `__LL_ADC_COMMON_INSTANCE()`)
- **CommonClock:** This parameter can be one of the following values:
 - LL_ADC_CLOCK_SYNC_PCLK_DIV2
 - LL_ADC_CLOCK_SYNC_PCLK_DIV4
 - LL_ADC_CLOCK_SYNC_PCLK_DIV6
 - LL_ADC_CLOCK_SYNC_PCLK_DIV8

Return values

- **None:**

Reference Manual to LL API cross reference:

- CCR ADCPRE LL_ADC_SetCommonClock

LL_ADC_GetCommonClock

Function name

```
__STATIC_INLINE uint32_t LL_ADC_GetCommonClock (ADC_Common_TypeDef * ADCxy_COMMON)
```

Function description

Get parameter common to several ADC: Clock source and prescaler.

Parameters

- **ADCxy_COMMON:** ADC common instance (can be set directly from CMSIS definition or by using helper macro `__LL_ADC_COMMON_INSTANCE()`)

Return values

- **Returned:** value can be one of the following values:
 - LL_ADC_CLOCK_SYNC_PCLK_DIV2
 - LL_ADC_CLOCK_SYNC_PCLK_DIV4
 - LL_ADC_CLOCK_SYNC_PCLK_DIV6
 - LL_ADC_CLOCK_SYNC_PCLK_DIV8

Reference Manual to LL API cross reference:

- CCR ADCPRE LL_ADC_GetCommonClock

LL_ADC_SetCommonPathInternalCh

Function name

```
__STATIC_INLINE void LL_ADC_SetCommonPathInternalCh (ADC_Common_TypeDef *
ADCxy_COMMON, uint32_t PathInternal)
```

Function description

Set parameter common to several ADC: measurement path to internal channels (VrefInt, temperature sensor, ...).

Parameters

- **ADCxy_COMMON:** ADC common instance (can be set directly from CMSIS definition or by using helper macro `__LL_ADC_COMMON_INSTANCE()`)
- **PathInternal:** This parameter can be a combination of the following values:
 - `LL_ADC_PATH_INTERNAL_NONE`
 - `LL_ADC_PATH_INTERNAL_VREFINT`
 - `LL_ADC_PATH_INTERNAL_TEMPSENSOR`
 - `LL_ADC_PATH_INTERNAL_VBAT`

Return values

- **None:**

Notes

- One or several values can be selected. Example: (`LL_ADC_PATH_INTERNAL_VREFINT` | `LL_ADC_PATH_INTERNAL_TEMPSENSOR`)
- Stabilization time of measurement path to internal channel: After enabling internal paths, before starting ADC conversion, a delay is required for internal voltage reference and temperature sensor stabilization time. Refer to device datasheet. Refer to literal `LL_ADC_DELAY_VREFINT_STAB_US`. Refer to literal `LL_ADC_DELAY_TEMPSENSOR_STAB_US`.
- ADC internal channel sampling time constraint: For ADC conversion of internal channels, a sampling time minimum value is required. Refer to device datasheet.

Reference Manual to LL API cross reference:

- CCR TSVREFE `LL_ADC_SetCommonPathInternalCh`
- CCR VBATE `LL_ADC_SetCommonPathInternalCh`

LL_ADC_GetCommonPathInternalCh

Function name

`__STATIC_INLINE uint32_t LL_ADC_GetCommonPathInternalCh (ADC_Common_TypeDef * ADCxy_COMMON)`

Function description

Get parameter common to several ADC: measurement path to internal channels (VrefInt, temperature sensor, ...).

Parameters

- **ADCxy_COMMON:** ADC common instance (can be set directly from CMSIS definition or by using helper macro `__LL_ADC_COMMON_INSTANCE()`)

Return values

- **Returned:** value can be a combination of the following values:
 - `LL_ADC_PATH_INTERNAL_NONE`
 - `LL_ADC_PATH_INTERNAL_VREFINT`
 - `LL_ADC_PATH_INTERNAL_TEMPSENSOR`
 - `LL_ADC_PATH_INTERNAL_VBAT`

Notes

- One or several values can be selected. Example: (`LL_ADC_PATH_INTERNAL_VREFINT` | `LL_ADC_PATH_INTERNAL_TEMPSENSOR`)

Reference Manual to LL API cross reference:

- CCR TSVREFE LL_ADC_GetCommonPathInternalCh
- CCR VBATE LL_ADC_GetCommonPathInternalCh

LL_ADC_SetResolution

Function name

```
__STATIC_INLINE void LL_ADC_SetResolution (ADC_TypeDef * ADCx, uint32_t Resolution)
```

Function description

Set ADC resolution.

Parameters

- **ADCx:** ADC instance
- **Resolution:** This parameter can be one of the following values:
 - LL_ADC_RESOLUTION_12B
 - LL_ADC_RESOLUTION_10B
 - LL_ADC_RESOLUTION_8B
 - LL_ADC_RESOLUTION_6B

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR1 RES LL_ADC_SetResolution

LL_ADC_GetResolution

Function name

```
__STATIC_INLINE uint32_t LL_ADC_GetResolution (ADC_TypeDef * ADCx)
```

Function description

Get ADC resolution.

Parameters

- **ADCx:** ADC instance

Return values

- **Returned:** value can be one of the following values:
 - LL_ADC_RESOLUTION_12B
 - LL_ADC_RESOLUTION_10B
 - LL_ADC_RESOLUTION_8B
 - LL_ADC_RESOLUTION_6B

Reference Manual to LL API cross reference:

- CR1 RES LL_ADC_GetResolution

LL_ADC_SetDataAlignment

Function name

```
__STATIC_INLINE void LL_ADC_SetDataAlignment (ADC_TypeDef * ADCx, uint32_t DataAlignment)
```

Function description

Set ADC conversion data alignment.

Parameters

- **ADCx:** ADC instance
- **DataAlignment:** This parameter can be one of the following values:
 - LL_ADC_DATA_ALIGN_RIGHT
 - LL_ADC_DATA_ALIGN_LEFT

Return values

- **None:**

Notes

- Refer to reference manual for alignments formats dependencies to ADC resolutions.

Reference Manual to LL API cross reference:

- CR2 ALIGN LL_ADC_SetDataAlignment

LL_ADC_GetDataAlignment

Function name

```
__STATIC_INLINE uint32_t LL_ADC_GetDataAlignment (ADC_TypeDef * ADCx)
```

Function description

Get ADC conversion data alignment.

Parameters

- **ADCx:** ADC instance

Return values

- **Returned:** value can be one of the following values:
 - LL_ADC_DATA_ALIGN_RIGHT
 - LL_ADC_DATA_ALIGN_LEFT

Notes

- Refer to reference manual for alignments formats dependencies to ADC resolutions.

Reference Manual to LL API cross reference:

- CR2 ALIGN LL_ADC_SetDataAlignment

LL_ADC_SetSequencersScanMode

Function name

```
__STATIC_INLINE void LL_ADC_SetSequencersScanMode (ADC_TypeDef * ADCx, uint32_t ScanMode)
```

Function description

Set ADC sequencers scan mode, for all ADC groups (group regular, group injected).

Parameters

- **ADCx:** ADC instance
- **ScanMode:** This parameter can be one of the following values:
 - LL_ADC_SEQ_SCAN_DISABLE
 - LL_ADC_SEQ_SCAN_ENABLE

Return values

- **None:**

Notes

- According to sequencers scan mode : If disabled: ADC conversion is performed in unitary conversion mode (one channel converted, that defined in rank 1). Configuration of sequencers of all ADC groups (sequencer scan length, ...) is discarded: equivalent to scan length of 1 rank. If enabled: ADC conversions are performed in sequence conversions mode, according to configuration of sequencers of each ADC group (sequencer scan length, ...). Refer to function LL_ADC_REG_SetSequencerLength() and to function LL_ADC_INJ_SetSequencerLength().

Reference Manual to LL API cross reference:

- CR1 SCAN LL_ADC_SetSequencersScanMode

LL_ADC_GetSequencersScanMode

Function name

__STATIC_INLINE uint32_t LL_ADC_GetSequencersScanMode (ADC_TypeDef * ADCx)

Function description

Get ADC sequencers scan mode, for all ADC groups (group regular, group injected).

Parameters

- ADCx:** ADC instance

Return values

- Returned:** value can be one of the following values:
 - LL_ADC_SEQ_SCAN_DISABLE
 - LL_ADC_SEQ_SCAN_ENABLE

Notes

- According to sequencers scan mode : If disabled: ADC conversion is performed in unitary conversion mode (one channel converted, that defined in rank 1). Configuration of sequencers of all ADC groups (sequencer scan length, ...) is discarded: equivalent to scan length of 1 rank. If enabled: ADC conversions are performed in sequence conversions mode, according to configuration of sequencers of each ADC group (sequencer scan length, ...). Refer to function LL_ADC_REG_SetSequencerLength() and to function LL_ADC_INJ_SetSequencerLength().

Reference Manual to LL API cross reference:

- CR1 SCAN LL_ADC_GetSequencersScanMode

LL_ADC_REG_SetTriggerSource

Function name

__STATIC_INLINE void LL_ADC_REG_SetTriggerSource (ADC_TypeDef * ADCx, uint32_t TriggerSource)

Function description

Set ADC group regular conversion trigger source: internal (SW start) or from external IP (timer event, external interrupt line).

Parameters

- **ADCx:** ADC instance
- **TriggerSource:** This parameter can be one of the following values:
 - LL_ADC_REG_TRIG_SOFTWARE
 - LL_ADC_REG_TRIG_EXT_TIM1_CH1
 - LL_ADC_REG_TRIG_EXT_TIM1_CH2
 - LL_ADC_REG_TRIG_EXT_TIM1_CH3
 - LL_ADC_REG_TRIG_EXT_TIM2_CH2
 - LL_ADC_REG_TRIG_EXT_TIM2_CH3
 - LL_ADC_REG_TRIG_EXT_TIM2_CH4
 - LL_ADC_REG_TRIG_EXT_TIM2_TRGO
 - LL_ADC_REG_TRIG_EXT_TIM3_CH1
 - LL_ADC_REG_TRIG_EXT_TIM3_TRGO
 - LL_ADC_REG_TRIG_EXT_TIM4_CH4
 - LL_ADC_REG_TRIG_EXT_TIM5_CH1
 - LL_ADC_REG_TRIG_EXT_TIM5_CH2
 - LL_ADC_REG_TRIG_EXT_TIM5_CH3
 - LL_ADC_REG_TRIG_EXT_TIM8_CH1
 - LL_ADC_REG_TRIG_EXT_TIM8_TRGO
 - LL_ADC_REG_TRIG_EXT_EXTI_LINE11

Return values

- **None:**

Notes

- On this STM32 series, setting of external trigger edge is performed using function LL_ADC_REG_StartConversionExtTrig().
- Availability of parameters of trigger sources from timer depends on timers availability on the selected device.

Reference Manual to LL API cross reference:

- CR2 EXTSEL LL_ADC_REG_SetTriggerSource
- CR2 EXTEN LL_ADC_REG_SetTriggerSource

LL_ADC_REG_GetTriggerSource

Function name

```
__STATIC_INLINE uint32_t LL_ADC_REG_GetTriggerSource(ADC_TypeDef * ADCx)
```

Function description

Get ADC group regular conversion trigger source: internal (SW start) or from external IP (timer event, external interrupt line).

Parameters

- **ADCx:** ADC instance

Return values

- **Returned:** value can be one of the following values:
 - LL_ADC_REG_TRIG_SOFTWARE
 - LL_ADC_REG_TRIG_EXT_TIM1_CH1
 - LL_ADC_REG_TRIG_EXT_TIM1_CH2
 - LL_ADC_REG_TRIG_EXT_TIM1_CH3
 - LL_ADC_REG_TRIG_EXT_TIM2_CH2
 - LL_ADC_REG_TRIG_EXT_TIM2_CH3
 - LL_ADC_REG_TRIG_EXT_TIM2_CH4
 - LL_ADC_REG_TRIG_EXT_TIM2_TRGO
 - LL_ADC_REG_TRIG_EXT_TIM3_CH1
 - LL_ADC_REG_TRIG_EXT_TIM3_TRGO
 - LL_ADC_REG_TRIG_EXT_TIM4_CH4
 - LL_ADC_REG_TRIG_EXT_TIM5_CH1
 - LL_ADC_REG_TRIG_EXT_TIM5_CH2
 - LL_ADC_REG_TRIG_EXT_TIM5_CH3
 - LL_ADC_REG_TRIG_EXT_TIM8_CH1
 - LL_ADC_REG_TRIG_EXT_TIM8_TRGO
 - LL_ADC_REG_TRIG_EXT_EXTI_LINE11

Notes

- To determine whether group regular trigger source is internal (SW start) or external, without detail of which peripheral is selected as external trigger, (equivalent to "if(LL_ADC_REG_GetTriggerSource(ADC1) == LL_ADC_REG_TRIG_SOFTWARE)") use function LL_ADC_REG_IsTriggerSourceSWStart.
- Availability of parameters of trigger sources from timer depends on timers availability on the selected device.

Reference Manual to LL API cross reference:

- CR2 EXTSEL LL_ADC_REG_GetTriggerSource
- CR2 EXTEN LL_ADC_REG_GetTriggerSource

LL_ADC_REG_IsTriggerSourceSWStart

Function name

```
__STATIC_INLINE uint32_t LL_ADC_REG_IsTriggerSourceSWStart (ADC_TypeDef * ADCx)
```

Function description

Get ADC group regular conversion trigger source internal (SW start) or external.

Parameters

- **ADCx:** ADC instance

Return values

- **Value:** "0" if trigger source external trigger Value "1" if trigger source SW start.

Notes

- In case of group regular trigger source set to external trigger, to determine which peripheral is selected as external trigger, use function LL_ADC_REG_GetTriggerSource().

Reference Manual to LL API cross reference:

- CR2 EXTEN LL_ADC_REG_IsTriggerSourceSWStart

LL_ADC_REG_GetTriggerEdge

Function name

```
__STATIC_INLINE uint32_t LL_ADC_REG_GetTriggerEdge (ADC_TypeDef * ADCx)
```

Function description

Get ADC group regular conversion trigger polarity.

Parameters

- **ADCx:** ADC instance

Return values

- **Returned:** value can be one of the following values:
 - LL_ADC_REG_TRIG_EXT_RISING
 - LL_ADC_REG_TRIG_EXT_FALLING
 - LL_ADC_REG_TRIG_EXT_RISINGFALLING

Notes

- Applicable only for trigger source set to external trigger.
- On this STM32 series, setting of external trigger edge is performed using function LL_ADC_REG_StartConversionExtTrig().

Reference Manual to LL API cross reference:

- CR2 EXTEN LL_ADC_REG_GetTriggerEdge

LL_ADC_REG_SetSequencerLength

Function name

```
__STATIC_INLINE void LL_ADC_REG_SetSequencerLength (ADC_TypeDef * ADCx, uint32_t SequencerNbRanks)
```

Function description

Set ADC group regular sequencer length and scan direction.

Parameters

- **ADCx:** ADC instance
- **SequencerNbRanks:** This parameter can be one of the following values:
 - LL_ADC_REG_SEQ_SCAN_DISABLE
 - LL_ADC_REG_SEQ_SCAN_ENABLE_2RANKS
 - LL_ADC_REG_SEQ_SCAN_ENABLE_3RANKS
 - LL_ADC_REG_SEQ_SCAN_ENABLE_4RANKS
 - LL_ADC_REG_SEQ_SCAN_ENABLE_5RANKS
 - LL_ADC_REG_SEQ_SCAN_ENABLE_6RANKS
 - LL_ADC_REG_SEQ_SCAN_ENABLE_7RANKS
 - LL_ADC_REG_SEQ_SCAN_ENABLE_8RANKS
 - LL_ADC_REG_SEQ_SCAN_ENABLE_9RANKS
 - LL_ADC_REG_SEQ_SCAN_ENABLE_10RANKS
 - LL_ADC_REG_SEQ_SCAN_ENABLE_11RANKS
 - LL_ADC_REG_SEQ_SCAN_ENABLE_12RANKS
 - LL_ADC_REG_SEQ_SCAN_ENABLE_13RANKS
 - LL_ADC_REG_SEQ_SCAN_ENABLE_14RANKS
 - LL_ADC_REG_SEQ_SCAN_ENABLE_15RANKS
 - LL_ADC_REG_SEQ_SCAN_ENABLE_16RANKS

Return values

- **None:**

Notes

- Description of ADC group regular sequencer features: For devices with sequencer fully configurable (function "LL_ADC_REG_SetSequencerRanks()" available): sequencer length and each rank affectation to a channel are configurable. This function performs configuration of: Sequence length: Number of ranks in the scan sequence. Sequence direction: Unless specified in parameters, sequencer scan direction is forward (from rank 1 to rank n). Sequencer ranks are selected using function "LL_ADC_REG_SetSequencerRanks()". For devices with sequencer not fully configurable (function "LL_ADC_REG_SetSequencerChannels()" available): sequencer length and each rank affectation to a channel are defined by channel number. This function performs configuration of: Sequence length: Number of ranks in the scan sequence is defined by number of channels set in the sequence, rank of each channel is fixed by channel HW number. (channel 0 fixed on rank 0, channel 1 fixed on rank1, ...). Sequence direction: Unless specified in parameters, sequencer scan direction is forward (from lowest channel number to highest channel number). Sequencer ranks are selected using function "LL_ADC_REG_SetSequencerChannels()".
- On this STM32 series, group regular sequencer configuration is conditioned to ADC instance sequencer mode. If ADC instance sequencer mode is disabled, sequencers of all groups (group regular, group injected) can be configured but their execution is disabled (limited to rank 1). Refer to function LL_ADC_SetSequencersScanMode().
- Sequencer disabled is equivalent to sequencer of 1 rank: ADC conversion on only 1 channel.

Reference Manual to LL API cross reference:

- SQR1 L LL_ADC_REG_SetSequencerLength

LL_ADC_REG_GetSequencerLength

Function name

__STATIC_INLINE uint32_t LL_ADC_REG_GetSequencerLength (ADC_TypeDef * ADCx)

Function description

Get ADC group regular sequencer length and scan direction.

Parameters

- **ADCx:** ADC instance

Return values

- **Returned:** value can be one of the following values:
 - LL_ADC_REG_SEQ_SCAN_DISABLE
 - LL_ADC_REG_SEQ_SCAN_ENABLE_2RANKS
 - LL_ADC_REG_SEQ_SCAN_ENABLE_3RANKS
 - LL_ADC_REG_SEQ_SCAN_ENABLE_4RANKS
 - LL_ADC_REG_SEQ_SCAN_ENABLE_5RANKS
 - LL_ADC_REG_SEQ_SCAN_ENABLE_6RANKS
 - LL_ADC_REG_SEQ_SCAN_ENABLE_7RANKS
 - LL_ADC_REG_SEQ_SCAN_ENABLE_8RANKS
 - LL_ADC_REG_SEQ_SCAN_ENABLE_9RANKS
 - LL_ADC_REG_SEQ_SCAN_ENABLE_10RANKS
 - LL_ADC_REG_SEQ_SCAN_ENABLE_11RANKS
 - LL_ADC_REG_SEQ_SCAN_ENABLE_12RANKS
 - LL_ADC_REG_SEQ_SCAN_ENABLE_13RANKS
 - LL_ADC_REG_SEQ_SCAN_ENABLE_14RANKS
 - LL_ADC_REG_SEQ_SCAN_ENABLE_15RANKS
 - LL_ADC_REG_SEQ_SCAN_ENABLE_16RANKS

Notes

- Description of ADC group regular sequencer features: For devices with sequencer fully configurable (function "LL_ADC_REG_SetSequencerRanks()" available): sequencer length and each rank affectation to a channel are configurable. This function retrieves: Sequence length: Number of ranks in the scan sequence. Sequence direction: Unless specified in parameters, sequencer scan direction is forward (from rank 1 to rank n). Sequencer ranks are selected using function "LL_ADC_REG_SetSequencerRanks()". For devices with sequencer not fully configurable (function "LL_ADC_REG_SetSequencerChannels()" available): sequencer length and each rank affectation to a channel are defined by channel number. This function retrieves: Sequence length: Number of ranks in the scan sequence is defined by number of channels set in the sequence, rank of each channel is fixed by channel HW number. (channel 0 fixed on rank 0, channel 1 fixed on rank1, ...). Sequence direction: Unless specified in parameters, sequencer scan direction is forward (from lowest channel number to highest channel number). Sequencer ranks are selected using function "LL_ADC_REG_SetSequencerChannels()".
- On this STM32 series, group regular sequencer configuration is conditioned to ADC instance sequencer mode. If ADC instance sequencer mode is disabled, sequencers of all groups (group regular, group injected) can be configured but their execution is disabled (limited to rank 1). Refer to function LL_ADC_SetSequencersScanMode().
- Sequencer disabled is equivalent to sequencer of 1 rank: ADC conversion on only 1 channel.

Reference Manual to LL API cross reference:

- SQR1 L LL_ADC_REG_SetSequencerLength

LL_ADC_REG_SetSequencerDiscont

Function name

__STATIC_INLINE void LL_ADC_REG_SetSequencerDiscont (ADC_TypeDef * ADCx, uint32_t SeqDiscont)

Function description

Set ADC group regular sequencer discontinuous mode: sequence subdivided and scan conversions interrupted every selected number of ranks.

Parameters

- **ADCx:** ADC instance
- **SeqDiscont:** This parameter can be one of the following values:
 - LL_ADC_REG_SEQ_DISCONT_DISABLE
 - LL_ADC_REG_SEQ_DISCONT_1RANK
 - LL_ADC_REG_SEQ_DISCONT_2RANKS
 - LL_ADC_REG_SEQ_DISCONT_3RANKS
 - LL_ADC_REG_SEQ_DISCONT_4RANKS
 - LL_ADC_REG_SEQ_DISCONT_5RANKS
 - LL_ADC_REG_SEQ_DISCONT_6RANKS
 - LL_ADC_REG_SEQ_DISCONT_7RANKS
 - LL_ADC_REG_SEQ_DISCONT_8RANKS

Return values

- **None:**

Notes

- It is not possible to enable both ADC group regular continuous mode and sequencer discontinuous mode.
- It is not possible to enable both ADC auto-injected mode and ADC group regular sequencer discontinuous mode.

Reference Manual to LL API cross reference:

- CR1 DISCEN LL_ADC_REG_SetSequencerDiscont
- CR1 DISCNUM LL_ADC_REG_SetSequencerDiscont

LL_ADC_REG_GetSequencerDiscont

Function name

__STATIC_INLINE uint32_t LL_ADC_REG_GetSequencerDiscont (ADC_TypeDef * ADCx)

Function description

Get ADC group regular sequencer discontinuous mode: sequence subdivided and scan conversions interrupted every selected number of ranks.

Parameters

- **ADCx:** ADC instance

Return values

- **Returned:** value can be one of the following values:
 - LL_ADC_REG_SEQ_DISCONT_DISABLE
 - LL_ADC_REG_SEQ_DISCONT_1RANK
 - LL_ADC_REG_SEQ_DISCONT_2RANKS
 - LL_ADC_REG_SEQ_DISCONT_3RANKS
 - LL_ADC_REG_SEQ_DISCONT_4RANKS
 - LL_ADC_REG_SEQ_DISCONT_5RANKS
 - LL_ADC_REG_SEQ_DISCONT_6RANKS
 - LL_ADC_REG_SEQ_DISCONT_7RANKS
 - LL_ADC_REG_SEQ_DISCONT_8RANKS

Reference Manual to LL API cross reference:

- CR1 DISCEN LL_ADC_REG_GetSequencerDiscont
- CR1 DISCNUM LL_ADC_REG_GetSequencerDiscont

LL_ADC_REG_SetSequencerRanks

Function name

__STATIC_INLINE void LL_ADC_REG_SetSequencerRanks (ADC_TypeDef * ADCx, uint32_t Rank, uint32_t Channel)

Function description

Set ADC group regular sequence: channel on the selected scan sequence rank.

Parameters

- **ADCx:** ADC instance
- **Rank:** This parameter can be one of the following values:
 - LL_ADC_REG_RANK_1
 - LL_ADC_REG_RANK_2
 - LL_ADC_REG_RANK_3
 - LL_ADC_REG_RANK_4
 - LL_ADC_REG_RANK_5
 - LL_ADC_REG_RANK_6
 - LL_ADC_REG_RANK_7
 - LL_ADC_REG_RANK_8
 - LL_ADC_REG_RANK_9
 - LL_ADC_REG_RANK_10
 - LL_ADC_REG_RANK_11
 - LL_ADC_REG_RANK_12
 - LL_ADC_REG_RANK_13
 - LL_ADC_REG_RANK_14
 - LL_ADC_REG_RANK_15
 - LL_ADC_REG_RANK_16
- **Channel:** This parameter can be one of the following values:
 - LL_ADC_CHANNEL_0
 - LL_ADC_CHANNEL_1
 - LL_ADC_CHANNEL_2
 - LL_ADC_CHANNEL_3
 - LL_ADC_CHANNEL_4
 - LL_ADC_CHANNEL_5
 - LL_ADC_CHANNEL_6
 - LL_ADC_CHANNEL_7
 - LL_ADC_CHANNEL_8
 - LL_ADC_CHANNEL_9
 - LL_ADC_CHANNEL_10
 - LL_ADC_CHANNEL_11
 - LL_ADC_CHANNEL_12
 - LL_ADC_CHANNEL_13
 - LL_ADC_CHANNEL_14
 - LL_ADC_CHANNEL_15
 - LL_ADC_CHANNEL_16
 - LL_ADC_CHANNEL_17
 - LL_ADC_CHANNEL_18
 - LL_ADC_CHANNEL_VREFINT (1)
 - LL_ADC_CHANNEL_TEMPSENSOR (1)(2)
 - LL_ADC_CHANNEL_VBAT (1)

(1) On STM32F4, parameter available only on ADC instance: ADC1.

(2) On devices STM32F42x and STM32F43x, limitation: this internal channel is shared between temperature sensor and Vbat, only 1 measurement path must be enabled.

Return values

- **None:**

Notes

- This function performs configuration of: Channels ordering into each rank of scan sequence: whatever channel can be placed into whatever rank.
- On this STM32 series, ADC group regular sequencer is fully configurable: sequencer length and each rank affectation to a channel are configurable. Refer to description of function `LL_ADC_REG_SetSequencerLength()`.
- Depending on devices and packages, some channels may not be available. Refer to device datasheet for channels availability.
- On this STM32 series, to measure internal channels (VrefInt, TempSensor, ...), measurement paths to internal channels must be enabled separately. This can be done using function `LL_ADC_SetCommonPathInternalCh()`.

Reference Manual to LL API cross reference:

- SQR3 SQ1 `LL_ADC_REG_SetSequencerRanks`
- SQR3 SQ2 `LL_ADC_REG_SetSequencerRanks`
- SQR3 SQ3 `LL_ADC_REG_SetSequencerRanks`
- SQR3 SQ4 `LL_ADC_REG_SetSequencerRanks`
- SQR3 SQ5 `LL_ADC_REG_SetSequencerRanks`
- SQR3 SQ6 `LL_ADC_REG_SetSequencerRanks`
- SQR2 SQ7 `LL_ADC_REG_SetSequencerRanks`
- SQR2 SQ8 `LL_ADC_REG_SetSequencerRanks`
- SQR2 SQ9 `LL_ADC_REG_SetSequencerRanks`
- SQR2 SQ10 `LL_ADC_REG_SetSequencerRanks`
- SQR2 SQ11 `LL_ADC_REG_SetSequencerRanks`
- SQR2 SQ12 `LL_ADC_REG_SetSequencerRanks`
- SQR1 SQ13 `LL_ADC_REG_SetSequencerRanks`
- SQR1 SQ14 `LL_ADC_REG_SetSequencerRanks`
- SQR1 SQ15 `LL_ADC_REG_SetSequencerRanks`
- SQR1 SQ16 `LL_ADC_REG_SetSequencerRanks`

LL_ADC_REG_GetSequencerRanks

Function name

`__STATIC_INLINE uint32_t LL_ADC_REG_GetSequencerRanks (ADC_TypeDef * ADCx, uint32_t Rank)`

Function description

Get ADC group regular sequence: channel on the selected scan sequence rank.

Parameters

- **ADCx:** ADC instance
- **Rank:** This parameter can be one of the following values:
 - LL_ADC_REG_RANK_1
 - LL_ADC_REG_RANK_2
 - LL_ADC_REG_RANK_3
 - LL_ADC_REG_RANK_4
 - LL_ADC_REG_RANK_5
 - LL_ADC_REG_RANK_6
 - LL_ADC_REG_RANK_7
 - LL_ADC_REG_RANK_8
 - LL_ADC_REG_RANK_9
 - LL_ADC_REG_RANK_10
 - LL_ADC_REG_RANK_11
 - LL_ADC_REG_RANK_12
 - LL_ADC_REG_RANK_13
 - LL_ADC_REG_RANK_14
 - LL_ADC_REG_RANK_15
 - LL_ADC_REG_RANK_16

Return values

- **Returned:** value can be one of the following values:
 - LL_ADC_CHANNEL_0
 - LL_ADC_CHANNEL_1
 - LL_ADC_CHANNEL_2
 - LL_ADC_CHANNEL_3
 - LL_ADC_CHANNEL_4
 - LL_ADC_CHANNEL_5
 - LL_ADC_CHANNEL_6
 - LL_ADC_CHANNEL_7
 - LL_ADC_CHANNEL_8
 - LL_ADC_CHANNEL_9
 - LL_ADC_CHANNEL_10
 - LL_ADC_CHANNEL_11
 - LL_ADC_CHANNEL_12
 - LL_ADC_CHANNEL_13
 - LL_ADC_CHANNEL_14
 - LL_ADC_CHANNEL_15
 - LL_ADC_CHANNEL_16
 - LL_ADC_CHANNEL_17
 - LL_ADC_CHANNEL_18
 - LL_ADC_CHANNEL_VREFINT (1)
 - LL_ADC_CHANNEL_TEMPSENSOR (1)(2)
 - LL_ADC_CHANNEL_VBAT (1)
- (1) On STM32F4, parameter available only on ADC instance: ADC1.
- (2) On devices STM32F42x and STM32F43x, limitation: this internal channel is shared between temperature sensor and Vbat, only 1 measurement path must be enabled.
- (1) For ADC channel read back from ADC register, comparison with internal channel parameter to be done using helper macro `__LL_ADC_CHANNEL_INTERNAL_TO_EXTERNAL()`.

Notes

- On this STM32 series, ADC group regular sequencer is fully configurable: sequencer length and each rank affectation to a channel are configurable. Refer to description of function `LL_ADC_REG_SetSequencerLength()`.
- Depending on devices and packages, some channels may not be available. Refer to device datasheet for channels availability.
- Usage of the returned channel number: To reinject this channel into another function `LL_ADC_xxx`: the returned channel number is only partly formatted on definition of literals `LL_ADC_CHANNEL_x`. Therefore, it has to be compared with parts of literals `LL_ADC_CHANNEL_x` or using helper macro `__LL_ADC_CHANNEL_TO_DECIMAL_NB()`. Then the selected literal `LL_ADC_CHANNEL_x` can be used as parameter for another function. To get the channel number in decimal format: process the returned value with the helper macro `__LL_ADC_CHANNEL_TO_DECIMAL_NB()`.

Reference Manual to LL API cross reference:

- SQR3 SQ1 `LL_ADC_REG_GetSequencerRanks`
- SQR3 SQ2 `LL_ADC_REG_GetSequencerRanks`
- SQR3 SQ3 `LL_ADC_REG_GetSequencerRanks`
- SQR3 SQ4 `LL_ADC_REG_GetSequencerRanks`
- SQR3 SQ5 `LL_ADC_REG_GetSequencerRanks`
- SQR3 SQ6 `LL_ADC_REG_GetSequencerRanks`
- SQR2 SQ7 `LL_ADC_REG_GetSequencerRanks`
- SQR2 SQ8 `LL_ADC_REG_GetSequencerRanks`
- SQR2 SQ9 `LL_ADC_REG_GetSequencerRanks`
- SQR2 SQ10 `LL_ADC_REG_GetSequencerRanks`
- SQR2 SQ11 `LL_ADC_REG_GetSequencerRanks`
- SQR2 SQ12 `LL_ADC_REG_GetSequencerRanks`
- SQR1 SQ13 `LL_ADC_REG_GetSequencerRanks`
- SQR1 SQ14 `LL_ADC_REG_GetSequencerRanks`
- SQR1 SQ15 `LL_ADC_REG_GetSequencerRanks`
- SQR1 SQ16 `LL_ADC_REG_GetSequencerRanks`

LL_ADC_REG_SetContinuousMode

Function name

`__STATIC_INLINE void LL_ADC_REG_SetContinuousMode (ADC_TypeDef * ADCx, uint32_t Continuous)`

Function description

Set ADC continuous conversion mode on ADC group regular.

Parameters

- ADCx**: ADC instance
- Continuous**: This parameter can be one of the following values:
 - `LL_ADC_REG_CONV_SINGLE`
 - `LL_ADC_REG_CONV_CONTINUOUS`

Return values

- None**:

Notes

- Description of ADC continuous conversion mode: single mode: one conversion per triggercontinuous mode: after the first trigger, following conversions launched successively automatically.
- It is not possible to enable both ADC group regular continuous mode and sequencer discontinuous mode.

Reference Manual to LL API cross reference:

- CR2 CONT `LL_ADC_REG_SetContinuousMode`

LL_ADC_REG_GetContinuousMode

Function name

```
__STATIC_INLINE uint32_t LL_ADC_REG_GetContinuousMode(ADC_TypeDef * ADCx)
```

Function description

Get ADC continuous conversion mode on ADC group regular.

Parameters

- **ADCx:** ADC instance

Return values

- **Returned:** value can be one of the following values:
 - LL_ADC_REG_CONV_SINGLE
 - LL_ADC_REG_CONV_CONTINUOUS

Notes

- Description of ADC continuous conversion mode: single mode: one conversion per triggercontinuous mode: after the first trigger, following conversions launched successively automatically.

Reference Manual to LL API cross reference:

- CR2 CONT LL_ADC_REG_GetContinuousMode

LL_ADC_REG_SetDMATransfer

Function name

```
__STATIC_INLINE void LL_ADC_REG_SetDMATransfer(ADC_TypeDef * ADCx, uint32_t DMATransfer)
```

Function description

Set ADC group regular conversion data transfer: no transfer or transfer by DMA, and DMA requests mode.

Parameters

- **ADCx:** ADC instance
- **DMATransfer:** This parameter can be one of the following values:
 - LL_ADC_REG_DMA_TRANSFER_NONE
 - LL_ADC_REG_DMA_TRANSFER_LIMITED
 - LL_ADC_REG_DMA_TRANSFER_UNLIMITED

Return values

- **None:**

Notes

- If transfer by DMA selected, specifies the DMA requests mode: Limited mode (One shot mode): DMA transfer requests are stopped when number of DMA data transfers (number of ADC conversions) is reached. This ADC mode is intended to be used with DMA mode non-circular.Unlimited mode: DMA transfer requests are unlimited, whatever number of DMA data transfers (number of ADC conversions). This ADC mode is intended to be used with DMA mode circular.
- If ADC DMA requests mode is set to unlimited and DMA is set to mode non-circular: when DMA transfers size will be reached, DMA will stop transfers of ADC conversions data ADC will raise an overrun error (overrun flag and interruption if enabled).
- For devices with several ADC instances: ADC multimode DMA settings are available using function LL_ADC_SetMultiDMATransfer().
- To configure DMA source address (peripheral address), use function LL_ADC_DMA_GetRegAddr().

Reference Manual to LL API cross reference:

- CR2 DMA LL_ADC_REG_SetDMATransfer
- CR2 DDS LL_ADC_REG_SetDMATransfer

LL_ADC_REG_GetDMATransfer

Function name

```
__STATIC_INLINE uint32_t LL_ADC_REG_GetDMATransfer (ADC_TypeDef * ADCx)
```

Function description

Get ADC group regular conversion data transfer: no transfer or transfer by DMA, and DMA requests mode.

Parameters

- **ADCx:** ADC instance

Return values

- **Returned:** value can be one of the following values:
 - LL_ADC_REG_DMA_TRANSFER_NONE
 - LL_ADC_REG_DMA_TRANSFER_LIMITED
 - LL_ADC_REG_DMA_TRANSFER_UNLIMITED

Notes

- If transfer by DMA selected, specifies the DMA requests mode: Limited mode (One shot mode): DMA transfer requests are stopped when number of DMA data transfers (number of ADC conversions) is reached. This ADC mode is intended to be used with DMA mode non-circular. Unlimited mode: DMA transfer requests are unlimited, whatever number of DMA data transfers (number of ADC conversions). This ADC mode is intended to be used with DMA mode circular.
- If ADC DMA requests mode is set to unlimited and DMA is set to mode non-circular: when DMA transfers size will be reached, DMA will stop transfers of ADC conversions data ADC will raise an overrun error (overrun flag and interruption if enabled).
- For devices with several ADC instances: ADC multimode DMA settings are available using function LL_ADC_GetMultiDMATransfer().
- To configure DMA source address (peripheral address), use function LL_ADC_DMA_GetRegAddr().

Reference Manual to LL API cross reference:

- CR2 DMA LL_ADC_REG_GetDMATransfer
- CR2 DDS LL_ADC_REG_GetDMATransfer

LL_ADC_REG_SetFlagEndOfConversion

Function name

```
__STATIC_INLINE void LL_ADC_REG_SetFlagEndOfConversion (ADC_TypeDef * ADCx, uint32_t EocSelection)
```

Function description

Specify which ADC flag between EOC (end of unitary conversion) or EOS (end of sequence conversions) is used to indicate the end of conversion.

Parameters

- **ADCx:** ADC instance
- **EocSelection:** This parameter can be one of the following values:
 - LL_ADC_REG_FLAG_EOC_SEQUENCE_CONV
 - LL_ADC_REG_FLAG_EOC_UNITARY_CONV

Return values

- **None:**

Notes

- This feature is aimed to be set when using ADC with programming model by polling or interruption (programming model by DMA usually uses DMA interruptions to indicate end of conversion and data transfer).
- For ADC group injected, end of conversion (flag&IT) is raised only at the end of the sequence.

Reference Manual to LL API cross reference:

- CR2 EOCS LL_ADC_REG_SetFlagEndOfConversion

LL_ADC_REG_GetFlagEndOfConversion

Function name

```
__STATIC_INLINE uint32_t LL_ADC_REG_GetFlagEndOfConversion (ADC_TypeDef * ADCx)
```

Function description

Get which ADC flag between EOC (end of unitary conversion) or EOS (end of sequence conversions) is used to indicate the end of conversion.

Parameters

- **ADCx:** ADC instance

Return values

- **Returned:** value can be one of the following values:
 - LL_ADC_REG_FLAG_EOC_SEQUENCE_CONV
 - LL_ADC_REG_FLAG_EOC_UNITARY_CONV

Reference Manual to LL API cross reference:

- CR2 EOCS LL_ADC_REG_GetFlagEndOfConversion

LL_ADC_INJ_SetTriggerSource

Function name

```
__STATIC_INLINE void LL_ADC_INJ_SetTriggerSource (ADC_TypeDef * ADCx, uint32_t TriggerSource)
```

Function description

Set ADC group injected conversion trigger source: internal (SW start) or from external IP (timer event, external interrupt line).

Parameters

- **ADCx:** ADC instance
- **TriggerSource:** This parameter can be one of the following values:
 - LL_ADC_INJ_TRIG_SOFTWARE
 - LL_ADC_INJ_TRIG_EXT_TIM1_CH4
 - LL_ADC_INJ_TRIG_EXT_TIM1_TRGO
 - LL_ADC_INJ_TRIG_EXT_TIM2_CH1
 - LL_ADC_INJ_TRIG_EXT_TIM2_TRGO
 - LL_ADC_INJ_TRIG_EXT_TIM3_CH2
 - LL_ADC_INJ_TRIG_EXT_TIM3_CH4
 - LL_ADC_INJ_TRIG_EXT_TIM4_CH1
 - LL_ADC_INJ_TRIG_EXT_TIM4_CH2
 - LL_ADC_INJ_TRIG_EXT_TIM4_CH3
 - LL_ADC_INJ_TRIG_EXT_TIM4_TRGO
 - LL_ADC_INJ_TRIG_EXT_TIM5_CH4
 - LL_ADC_INJ_TRIG_EXT_TIM5_TRGO
 - LL_ADC_INJ_TRIG_EXT_TIM8_CH2
 - LL_ADC_INJ_TRIG_EXT_TIM8_CH3
 - LL_ADC_INJ_TRIG_EXT_TIM8_CH4
 - LL_ADC_INJ_TRIG_EXT_EXTI_LINE15

Return values

- **None:**

Notes

- On this STM32 series, setting of external trigger edge is performed using function LL_ADC_INJ_StartConversionExtTrig().
- Availability of parameters of trigger sources from timer depends on timers availability on the selected device.

Reference Manual to LL API cross reference:

- CR2 JEXTSEL LL_ADC_INJ_SetTriggerSource
- CR2 JEXTEN LL_ADC_INJ_SetTriggerSource

LL_ADC_INJ_GetTriggerSource

Function name

__STATIC_INLINE uint32_t LL_ADC_INJ_GetTriggerSource (ADC_TypeDef * ADCx)

Function description

Get ADC group injected conversion trigger source: internal (SW start) or from external IP (timer event, external interrupt line).

Parameters

- **ADCx:** ADC instance

Return values

- **Returned:** value can be one of the following values:
 - LL_ADC_INJ_TRIG_SOFTWARE
 - LL_ADC_INJ_TRIG_EXT_TIM1_CH4
 - LL_ADC_INJ_TRIG_EXT_TIM1_TRGO
 - LL_ADC_INJ_TRIG_EXT_TIM2_CH1
 - LL_ADC_INJ_TRIG_EXT_TIM2_TRGO
 - LL_ADC_INJ_TRIG_EXT_TIM3_CH2
 - LL_ADC_INJ_TRIG_EXT_TIM3_CH4
 - LL_ADC_INJ_TRIG_EXT_TIM4_CH1
 - LL_ADC_INJ_TRIG_EXT_TIM4_CH2
 - LL_ADC_INJ_TRIG_EXT_TIM4_CH3
 - LL_ADC_INJ_TRIG_EXT_TIM4_TRGO
 - LL_ADC_INJ_TRIG_EXT_TIM5_CH4
 - LL_ADC_INJ_TRIG_EXT_TIM5_TRGO
 - LL_ADC_INJ_TRIG_EXT_TIM8_CH2
 - LL_ADC_INJ_TRIG_EXT_TIM8_CH3
 - LL_ADC_INJ_TRIG_EXT_TIM8_CH4
 - LL_ADC_INJ_TRIG_EXT_EXTI_LINE15

Notes

- To determine whether group injected trigger source is internal (SW start) or external, without detail of which peripheral is selected as external trigger, (equivalent to "if(LL_ADC_INJ_GetTriggerSource(ADC1) == LL_ADC_INJ_TRIG_SOFTWARE)") use function LL_ADC_INJ_IsTriggerSourceSWStart.
- Availability of parameters of trigger sources from timer depends on timers availability on the selected device.

Reference Manual to LL API cross reference:

- CR2 JEXTSEL LL_ADC_INJ_GetTriggerSource
- CR2 JEXTEN LL_ADC_INJ_GetTriggerSource

LL_ADC_INJ_IsTriggerSourceSWStart

Function name

__STATIC_INLINE uint32_t LL_ADC_INJ_IsTriggerSourceSWStart (ADC_TypeDef * ADCx)

Function description

Get ADC group injected conversion trigger source internal (SW start) or external.

Parameters

- **ADCx:** ADC instance

Return values

- **Value:** "0" if trigger source external trigger Value "1" if trigger source SW start.

Notes

- In case of group injected trigger source set to external trigger, to determine which peripheral is selected as external trigger, use function LL_ADC_INJ_GetTriggerSource.

Reference Manual to LL API cross reference:

- CR2 JEXTEN LL_ADC_INJ_IsTriggerSourceSWStart

LL_ADC_INJ_GetTriggerEdge

Function name

```
__STATIC_INLINE uint32_t LL_ADC_INJ_GetTriggerEdge (ADC_TypeDef * ADCx)
```

Function description

Get ADC group injected conversion trigger polarity.

Parameters

- **ADCx:** ADC instance

Return values

- **Returned:** value can be one of the following values:
 - LL_ADC_INJ_TRIG_EXT_RISING
 - LL_ADC_INJ_TRIG_EXT_FALLING
 - LL_ADC_INJ_TRIG_EXT_RISINGFALLING

Reference Manual to LL API cross reference:

- CR2 JEXTEN LL_ADC_INJ_GetTriggerEdge

LL_ADC_INJ_SetSequencerLength

Function name

```
__STATIC_INLINE void LL_ADC_INJ_SetSequencerLength (ADC_TypeDef * ADCx, uint32_t SequencerNbRanks)
```

Function description

Set ADC group injected sequencer length and scan direction.

Parameters

- **ADCx:** ADC instance
- **SequencerNbRanks:** This parameter can be one of the following values:
 - LL_ADC_INJ_SEQ_SCAN_DISABLE
 - LL_ADC_INJ_SEQ_SCAN_ENABLE_2RANKS
 - LL_ADC_INJ_SEQ_SCAN_ENABLE_3RANKS
 - LL_ADC_INJ_SEQ_SCAN_ENABLE_4RANKS

Return values

- **None:**

Notes

- This function performs configuration of: Sequence length: Number of ranks in the scan sequence. Sequence direction: Unless specified in parameters, sequencer scan direction is forward (from rank 1 to rank n).
- On this STM32 series, group injected sequencer configuration is conditioned to ADC instance sequencer mode. If ADC instance sequencer mode is disabled, sequencers of all groups (group regular, group injected) can be configured but their execution is disabled (limited to rank 1). Refer to function LL_ADC_SetSequencersScanMode().
- Sequencer disabled is equivalent to sequencer of 1 rank: ADC conversion on only 1 channel.

Reference Manual to LL API cross reference:

- JSQR JL LL_ADC_INJ_SetSequencerLength

LL_ADC_INJ_GetSequencerLength

Function name

```
__STATIC_INLINE uint32_t LL_ADC_INJ_GetSequencerLength (ADC_TypeDef * ADCx)
```

Function description

Get ADC group injected sequencer length and scan direction.

Parameters

- **ADCx:** ADC instance

Return values

- **Returned:** value can be one of the following values:
 - LL_ADC_INJ_SEQ_SCAN_DISABLE
 - LL_ADC_INJ_SEQ_SCAN_ENABLE_2RANKS
 - LL_ADC_INJ_SEQ_SCAN_ENABLE_3RANKS
 - LL_ADC_INJ_SEQ_SCAN_ENABLE_4RANKS

Notes

- This function retrieves: Sequence length: Number of ranks in the scan sequence. Sequence direction: Unless specified in parameters, sequencer scan direction is forward (from rank 1 to rank n).
- On this STM32 series, group injected sequencer configuration is conditioned to ADC instance sequencer mode. If ADC instance sequencer mode is disabled, sequencers of all groups (group regular, group injected) can be configured but their execution is disabled (limited to rank 1). Refer to function LL_ADC_SetSequencersScanMode().
- Sequencer disabled is equivalent to sequencer of 1 rank: ADC conversion on only 1 channel.

Reference Manual to LL API cross reference:

- JSQR JL LL_ADC_INJ_GetSequencerLength

LL_ADC_INJ_SetSequencerDiscont

Function name

```
__STATIC_INLINE void LL_ADC_INJ_SetSequencerDiscont (ADC_TypeDef * ADCx, uint32_t SeqDiscont)
```

Function description

Set ADC group injected sequencer discontinuous mode: sequence subdivided and scan conversions interrupted every selected number of ranks.

Parameters

- **ADCx:** ADC instance
- **SeqDiscont:** This parameter can be one of the following values:
 - LL_ADC_INJ_SEQ_DISCONT_DISABLE
 - LL_ADC_INJ_SEQ_DISCONT_1RANK

Return values

- **None:**

Notes

- It is not possible to enable both ADC group injected auto-injected mode and sequencer discontinuous mode.

Reference Manual to LL API cross reference:

- CR1 DISCEN LL_ADC_INJ_SetSequencerDiscont

LL_ADC_INJ_GetSequencerDiscont

Function name

`__STATIC_INLINE uint32_t LL_ADC_INJ_GetSequencerDiscont (ADC_TypeDef * ADCx)`

Function description

Get ADC group injected sequencer discontinuous mode: sequence subdivided and scan conversions interrupted every selected number of ranks.

Parameters

- **ADCx**: ADC instance

Return values

- **Returned**: value can be one of the following values:
 - LL_ADC_INJ_SEQ_DISCONT_DISABLE
 - LL_ADC_INJ_SEQ_DISCONT_1RANK

Reference Manual to LL API cross reference:

- CR1 DISCEN LL_ADC_REG_GetSequencerDiscont

LL_ADC_INJ_SetSequencerRanks

Function name

`__STATIC_INLINE void LL_ADC_INJ_SetSequencerRanks (ADC_TypeDef * ADCx, uint32_t Rank, uint32_t Channel)`

Function description

Set ADC group injected sequence: channel on the selected sequence rank.

Parameters

- **ADCx:** ADC instance
- **Rank:** This parameter can be one of the following values:
 - LL_ADC_INJ_RANK_1
 - LL_ADC_INJ_RANK_2
 - LL_ADC_INJ_RANK_3
 - LL_ADC_INJ_RANK_4
- **Channel:** This parameter can be one of the following values:
 - LL_ADC_CHANNEL_0
 - LL_ADC_CHANNEL_1
 - LL_ADC_CHANNEL_2
 - LL_ADC_CHANNEL_3
 - LL_ADC_CHANNEL_4
 - LL_ADC_CHANNEL_5
 - LL_ADC_CHANNEL_6
 - LL_ADC_CHANNEL_7
 - LL_ADC_CHANNEL_8
 - LL_ADC_CHANNEL_9
 - LL_ADC_CHANNEL_10
 - LL_ADC_CHANNEL_11
 - LL_ADC_CHANNEL_12
 - LL_ADC_CHANNEL_13
 - LL_ADC_CHANNEL_14
 - LL_ADC_CHANNEL_15
 - LL_ADC_CHANNEL_16
 - LL_ADC_CHANNEL_17
 - LL_ADC_CHANNEL_18
 - LL_ADC_CHANNEL_VREFINT (1)
 - LL_ADC_CHANNEL_TEMPSENSOR (1)(2)
 - LL_ADC_CHANNEL_VBAT (1)

(1) On STM32F4, parameter available only on ADC instance: ADC1.

(2) On devices STM32F42x and STM32F43x, limitation: this internal channel is shared between temperature sensor and Vbat, only 1 measurement path must be enabled.

Return values

- **None:**

Notes

- Depending on devices and packages, some channels may not be available. Refer to device datasheet for channels availability.
- On this STM32 series, to measure internal channels (VrefInt, TempSensor, ...), measurement paths to internal channels must be enabled separately. This can be done using function LL_ADC_SetCommonPathInternalCh().

Reference Manual to LL API cross reference:

- JSQR JSQ1 LL_ADC_INJ_SetSequencerRanks
- JSQR JSQ2 LL_ADC_INJ_SetSequencerRanks
- JSQR JSQ3 LL_ADC_INJ_SetSequencerRanks
- JSQR JSQ4 LL_ADC_INJ_SetSequencerRanks

LL_ADC_INJ_GetSequencerRanks

Function name

```
__STATIC_INLINE uint32_t LL_ADC_INJ_GetSequencerRanks (ADC_TypeDef * ADCx, uint32_t Rank)
```

Function description

Get ADC group injected sequence: channel on the selected sequence rank.

Parameters

- **ADCx:** ADC instance
- **Rank:** This parameter can be one of the following values:
 - LL_ADC_INJ_RANK_1
 - LL_ADC_INJ_RANK_2
 - LL_ADC_INJ_RANK_3
 - LL_ADC_INJ_RANK_4

Return values

- **Returned:** value can be one of the following values:
 - LL_ADC_CHANNEL_0
 - LL_ADC_CHANNEL_1
 - LL_ADC_CHANNEL_2
 - LL_ADC_CHANNEL_3
 - LL_ADC_CHANNEL_4
 - LL_ADC_CHANNEL_5
 - LL_ADC_CHANNEL_6
 - LL_ADC_CHANNEL_7
 - LL_ADC_CHANNEL_8
 - LL_ADC_CHANNEL_9
 - LL_ADC_CHANNEL_10
 - LL_ADC_CHANNEL_11
 - LL_ADC_CHANNEL_12
 - LL_ADC_CHANNEL_13
 - LL_ADC_CHANNEL_14
 - LL_ADC_CHANNEL_15
 - LL_ADC_CHANNEL_16
 - LL_ADC_CHANNEL_17
 - LL_ADC_CHANNEL_18
 - LL_ADC_CHANNEL_VREFINT (1)
 - LL_ADC_CHANNEL_TEMPSENSOR (1)(2)
 - LL_ADC_CHANNEL_VBAT (1)
- (1) On STM32F4, parameter available only on ADC instance: ADC1.
- (2) On devices STM32F42x and STM32F43x, limitation: this internal channel is shared between temperature sensor and Vbat, only 1 measurement path must be enabled.
- (1) For ADC channel read back from ADC register, comparison with internal channel parameter to be done using helper macro `__LL_ADC_CHANNEL_INTERNAL_TO_EXTERNAL()`.

Notes

- Depending on devices and packages, some channels may not be available. Refer to device datasheet for channels availability.
- Usage of the returned channel number: To reinject this channel into another function LL_ADC_XXX: the returned channel number is only partly formatted on definition of literals LL_ADC_CHANNEL_x. Therefore, it has to be compared with parts of literals LL_ADC_CHANNEL_x or using helper macro __LL_ADC_CHANNEL_TO_DECIMAL_NB(). Then the selected literal LL_ADC_CHANNEL_x can be used as parameter for another function. To get the channel number in decimal format: process the returned value with the helper macro __LL_ADC_CHANNEL_TO_DECIMAL_NB().

Reference Manual to LL API cross reference:

- JSQR JSQ1 LL_ADC_INJ_SetSequencerRanks
- JSQR JSQ2 LL_ADC_INJ_SetSequencerRanks
- JSQR JSQ3 LL_ADC_INJ_SetSequencerRanks
- JSQR JSQ4 LL_ADC_INJ_SetSequencerRanks

LL_ADC_INJ_SetTrigAuto

Function name

```
__STATIC_INLINE void LL_ADC_INJ_SetTrigAuto (ADC_TypeDef * ADCx, uint32_t TrigAuto)
```

Function description

Set ADC group injected conversion trigger: independent or from ADC group regular.

Parameters

- **ADCx**: ADC instance
- **TrigAuto**: This parameter can be one of the following values:
 - LL_ADC_INJ_TRIG_INDEPENDENT
 - LL_ADC_INJ_TRIG_FROM_GRP_REGULAR

Return values

- **None:**

Notes

- This mode can be used to extend number of data registers updated after one ADC conversion trigger and with data permanently kept (not erased by successive conversions of scan of ADC sequencer ranks), up to 5 data registers: 1 data register on ADC group regular, 4 data registers on ADC group injected.
- If ADC group injected injected trigger source is set to an external trigger, this feature must be must be set to independent trigger. ADC group injected automatic trigger is compliant only with group injected trigger source set to SW start, without any further action on ADC group injected conversion start or stop: in this case, ADC group injected is controlled only from ADC group regular.
- It is not possible to enable both ADC group injected auto-injected mode and sequencer discontinuous mode.

Reference Manual to LL API cross reference:

- CR1 JAUTO LL_ADC_INJ_SetTrigAuto

LL_ADC_INJ_GetTrigAuto

Function name

```
__STATIC_INLINE uint32_t LL_ADC_INJ_GetTrigAuto (ADC_TypeDef * ADCx)
```

Function description

Get ADC group injected conversion trigger: independent or from ADC group regular.

Parameters

- **ADCx**: ADC instance

Return values

- **Returned:** value can be one of the following values:
 - LL_ADC_INJ_TRIG_INDEPENDENT
 - LL_ADC_INJ_TRIG_FROM_GRP_REGULAR

Reference Manual to LL API cross reference:

- CR1 JAUTO LL_ADC_INJ_GetTrigAuto

LL_ADC_INJ_SetOffset

Function name

__STATIC_INLINE void LL_ADC_INJ_SetOffset (ADC_TypeDef * ADCx, uint32_t Rank, uint32_t OffsetLevel)

Function description

Set ADC group injected offset.

Parameters

- **ADCx:** ADC instance
- **Rank:** This parameter can be one of the following values:
 - LL_ADC_INJ_RANK_1
 - LL_ADC_INJ_RANK_2
 - LL_ADC_INJ_RANK_3
 - LL_ADC_INJ_RANK_4
- **OffsetLevel:** Value between Min_Data=0x000 and Max_Data=0xFFF

Return values

- **None:**

Notes

- It sets: ADC group injected rank to which the offset programmed will be appliedOffset level (offset to be subtracted from the raw converted data). Caution: Offset format is dependent to ADC resolution: offset has to be left-aligned on bit 11, the LSB (right bits) are set to 0.
- Offset cannot be enabled or disabled. To emulate offset disabled, set an offset value equal to 0.

Reference Manual to LL API cross reference:

- JOFR1 JOFFSET1 LL_ADC_INJ_SetOffset
- JOFR2 JOFFSET2 LL_ADC_INJ_SetOffset
- JOFR3 JOFFSET3 LL_ADC_INJ_SetOffset
- JOFR4 JOFFSET4 LL_ADC_INJ_SetOffset

LL_ADC_INJ_GetOffset

Function name

__STATIC_INLINE uint32_t LL_ADC_INJ_GetOffset (ADC_TypeDef * ADCx, uint32_t Rank)

Function description

Get ADC group injected offset.

Parameters

- **ADCx:** ADC instance
- **Rank:** This parameter can be one of the following values:
 - LL_ADC_INJ_RANK_1
 - LL_ADC_INJ_RANK_2
 - LL_ADC_INJ_RANK_3
 - LL_ADC_INJ_RANK_4

Return values

- **Value:** between Min_Data=0x000 and Max_Data=0xFF

Notes

- It gives offset level (offset to be subtracted from the raw converted data). Caution: Offset format is dependent to ADC resolution: offset has to be left-aligned on bit 11, the LSB (right bits) are set to 0.

Reference Manual to LL API cross reference:

- JOFR1 JOFFSET1 LL_ADC_INJ_GetOffset
- JOFR2 JOFFSET2 LL_ADC_INJ_GetOffset
- JOFR3 JOFFSET3 LL_ADC_INJ_GetOffset
- JOFR4 JOFFSET4 LL_ADC_INJ_GetOffset

LL_ADC_SetChannelSamplingTime

Function name

```
__STATIC_INLINE void LL_ADC_SetChannelSamplingTime (ADC_TypeDef * ADCx, uint32_t Channel,  
uint32_t SamplingTime)
```

Function description

Set sampling time of the selected ADC channel Unit: ADC clock cycles.

Parameters

- **ADCx:** ADC instance
- **Channel:** This parameter can be one of the following values:
 - LL_ADC_CHANNEL_0
 - LL_ADC_CHANNEL_1
 - LL_ADC_CHANNEL_2
 - LL_ADC_CHANNEL_3
 - LL_ADC_CHANNEL_4
 - LL_ADC_CHANNEL_5
 - LL_ADC_CHANNEL_6
 - LL_ADC_CHANNEL_7
 - LL_ADC_CHANNEL_8
 - LL_ADC_CHANNEL_9
 - LL_ADC_CHANNEL_10
 - LL_ADC_CHANNEL_11
 - LL_ADC_CHANNEL_12
 - LL_ADC_CHANNEL_13
 - LL_ADC_CHANNEL_14
 - LL_ADC_CHANNEL_15
 - LL_ADC_CHANNEL_16
 - LL_ADC_CHANNEL_17
 - LL_ADC_CHANNEL_18
 - LL_ADC_CHANNEL_VREFINT (1)
 - LL_ADC_CHANNEL_TEMPSENSOR (1)(2)
 - LL_ADC_CHANNEL_VBAT (1)
- (1) On STM32F4, parameter available only on ADC instance: ADC1.
- (2) On devices STM32F42x and STM32F43x, limitation: this internal channel is shared between temperature sensor and Vbat, only 1 measurement path must be enabled.
- **SamplingTime:** This parameter can be one of the following values:
 - LL_ADC_SAMPLINGTIME_3CYCLES
 - LL_ADC_SAMPLINGTIME_15CYCLES
 - LL_ADC_SAMPLINGTIME_28CYCLES
 - LL_ADC_SAMPLINGTIME_56CYCLES
 - LL_ADC_SAMPLINGTIME_84CYCLES
 - LL_ADC_SAMPLINGTIME_112CYCLES
 - LL_ADC_SAMPLINGTIME_144CYCLES
 - LL_ADC_SAMPLINGTIME_480CYCLES

Return values

- **None:**

Notes

- On this device, sampling time is on channel scope: independently of channel mapped on ADC group regular or injected.
- In case of internal channel (VrefInt, TempSensor, ...) to be converted: sampling time constraints must be respected (sampling time can be adjusted in function of ADC clock frequency and sampling time setting). Refer to device datasheet for timings values (parameters TS_vrefint, TS_temp, ...).
- Conversion time is the addition of sampling time and processing time. Refer to reference manual for ADC processing time of this STM32 series.
- In case of ADC conversion of internal channel (VrefInt, temperature sensor, ...), a sampling time minimum value is required. Refer to device datasheet.

Reference Manual to LL API cross reference:

- SMPR1 SMP18 LL_ADC_SetChannelSamplingTime
- SMPR1 SMP17 LL_ADC_SetChannelSamplingTime
- SMPR1 SMP16 LL_ADC_SetChannelSamplingTime
- SMPR1 SMP15 LL_ADC_SetChannelSamplingTime
- SMPR1 SMP14 LL_ADC_SetChannelSamplingTime
- SMPR1 SMP13 LL_ADC_SetChannelSamplingTime
- SMPR1 SMP12 LL_ADC_SetChannelSamplingTime
- SMPR1 SMP11 LL_ADC_SetChannelSamplingTime
- SMPR1 SMP10 LL_ADC_SetChannelSamplingTime
- SMPR2 SMP9 LL_ADC_SetChannelSamplingTime
- SMPR2 SMP8 LL_ADC_SetChannelSamplingTime
- SMPR2 SMP7 LL_ADC_SetChannelSamplingTime
- SMPR2 SMP6 LL_ADC_SetChannelSamplingTime
- SMPR2 SMP5 LL_ADC_SetChannelSamplingTime
- SMPR2 SMP4 LL_ADC_SetChannelSamplingTime
- SMPR2 SMP3 LL_ADC_SetChannelSamplingTime
- SMPR2 SMP2 LL_ADC_SetChannelSamplingTime
- SMPR2 SMP1 LL_ADC_SetChannelSamplingTime
- SMPR2 SMP0 LL_ADC_SetChannelSamplingTime

LL_ADC_GetChannelSamplingTime**Function name**

__STATIC_INLINE uint32_t LL_ADC_GetChannelSamplingTime (ADC_TypeDef * ADCx, uint32_t Channel)

Function description

Get sampling time of the selected ADC channel Unit: ADC clock cycles.

Parameters

- **ADCx:** ADC instance
 - **Channel:** This parameter can be one of the following values:
 - LL_ADC_CHANNEL_0
 - LL_ADC_CHANNEL_1
 - LL_ADC_CHANNEL_2
 - LL_ADC_CHANNEL_3
 - LL_ADC_CHANNEL_4
 - LL_ADC_CHANNEL_5
 - LL_ADC_CHANNEL_6
 - LL_ADC_CHANNEL_7
 - LL_ADC_CHANNEL_8
 - LL_ADC_CHANNEL_9
 - LL_ADC_CHANNEL_10
 - LL_ADC_CHANNEL_11
 - LL_ADC_CHANNEL_12
 - LL_ADC_CHANNEL_13
 - LL_ADC_CHANNEL_14
 - LL_ADC_CHANNEL_15
 - LL_ADC_CHANNEL_16
 - LL_ADC_CHANNEL_17
 - LL_ADC_CHANNEL_18
 - LL_ADC_CHANNEL_VREFINT (1)
 - LL_ADC_CHANNEL_TEMPSENSOR (1)(2)
 - LL_ADC_CHANNEL_VBAT (1)
- (1) On STM32F4, parameter available only on ADC instance: ADC1.
- (2) On devices STM32F42x and STM32F43x, limitation: this internal channel is shared between temperature sensor and Vbat, only 1 measurement path must be enabled.

Return values

- **Returned:** value can be one of the following values:
 - LL_ADC_SAMPLINGTIME_3CYCLES
 - LL_ADC_SAMPLINGTIME_15CYCLES
 - LL_ADC_SAMPLINGTIME_28CYCLES
 - LL_ADC_SAMPLINGTIME_56CYCLES
 - LL_ADC_SAMPLINGTIME_84CYCLES
 - LL_ADC_SAMPLINGTIME_112CYCLES
 - LL_ADC_SAMPLINGTIME_144CYCLES
 - LL_ADC_SAMPLINGTIME_480CYCLES

Notes

- On this device, sampling time is on channel scope: independently of channel mapped on ADC group regular or injected.
- Conversion time is the addition of sampling time and processing time. Refer to reference manual for ADC processing time of this STM32 series.

Reference Manual to LL API cross reference:

- SMPR1 SMP18 LL_ADC_GetChannelSamplingTime
- SMPR1 SMP17 LL_ADC_GetChannelSamplingTime
- SMPR1 SMP16 LL_ADC_GetChannelSamplingTime
- SMPR1 SMP15 LL_ADC_GetChannelSamplingTime
- SMPR1 SMP14 LL_ADC_GetChannelSamplingTime
- SMPR1 SMP13 LL_ADC_GetChannelSamplingTime
- SMPR1 SMP12 LL_ADC_GetChannelSamplingTime
- SMPR1 SMP11 LL_ADC_GetChannelSamplingTime
- SMPR1 SMP10 LL_ADC_GetChannelSamplingTime
- SMPR2 SMP9 LL_ADC_GetChannelSamplingTime
- SMPR2 SMP8 LL_ADC_GetChannelSamplingTime
- SMPR2 SMP7 LL_ADC_GetChannelSamplingTime
- SMPR2 SMP6 LL_ADC_GetChannelSamplingTime
- SMPR2 SMP5 LL_ADC_GetChannelSamplingTime
- SMPR2 SMP4 LL_ADC_GetChannelSamplingTime
- SMPR2 SMP3 LL_ADC_GetChannelSamplingTime
- SMPR2 SMP2 LL_ADC_GetChannelSamplingTime
- SMPR2 SMP1 LL_ADC_GetChannelSamplingTime
- SMPR2 SMP0 LL_ADC_GetChannelSamplingTime

LL_ADC_SetAnalogWDMonitChannels
Function name

__STATIC_INLINE void LL_ADC_SetAnalogWDMonitChannels (ADC_TypeDef * ADCx, uint32_t AWDChannelGroup)

Function description

Set ADC analog watchdog monitored channels: a single channel or all channels, on ADC groups regular and-or injected.

Parameters

- **ADCx:** ADC instance

- **AWDChannelGroup:** This parameter can be one of the following values:

- LL_ADC_AWD_DISABLE
- LL_ADC_AWD_ALL_CHANNELS_REG
- LL_ADC_AWD_ALL_CHANNELS_INJ
- LL_ADC_AWD_ALL_CHANNELS_REG_INJ
- LL_ADC_AWD_CHANNEL_0_REG
- LL_ADC_AWD_CHANNEL_0_INJ
- LL_ADC_AWD_CHANNEL_0_REG_INJ
- LL_ADC_AWD_CHANNEL_1_REG
- LL_ADC_AWD_CHANNEL_1_INJ
- LL_ADC_AWD_CHANNEL_1_REG_INJ
- LL_ADC_AWD_CHANNEL_2_REG
- LL_ADC_AWD_CHANNEL_2_INJ
- LL_ADC_AWD_CHANNEL_2_REG_INJ
- LL_ADC_AWD_CHANNEL_3_REG
- LL_ADC_AWD_CHANNEL_3_INJ
- LL_ADC_AWD_CHANNEL_3_REG_INJ
- LL_ADC_AWD_CHANNEL_4_REG
- LL_ADC_AWD_CHANNEL_4_INJ
- LL_ADC_AWD_CHANNEL_4_REG_INJ
- LL_ADC_AWD_CHANNEL_5_REG
- LL_ADC_AWD_CHANNEL_5_INJ
- LL_ADC_AWD_CHANNEL_5_REG_INJ
- LL_ADC_AWD_CHANNEL_6_REG
- LL_ADC_AWD_CHANNEL_6_INJ
- LL_ADC_AWD_CHANNEL_6_REG_INJ
- LL_ADC_AWD_CHANNEL_7_REG
- LL_ADC_AWD_CHANNEL_7_INJ
- LL_ADC_AWD_CHANNEL_7_REG_INJ
- LL_ADC_AWD_CHANNEL_8_REG
- LL_ADC_AWD_CHANNEL_8_INJ
- LL_ADC_AWD_CHANNEL_8_REG_INJ
- LL_ADC_AWD_CHANNEL_9_REG
- LL_ADC_AWD_CHANNEL_9_INJ
- LL_ADC_AWD_CHANNEL_9_REG_INJ
- LL_ADC_AWD_CHANNEL_10_REG
- LL_ADC_AWD_CHANNEL_10_INJ
- LL_ADC_AWD_CHANNEL_10_REG_INJ
- LL_ADC_AWD_CHANNEL_11_REG
- LL_ADC_AWD_CHANNEL_11_INJ
- LL_ADC_AWD_CHANNEL_11_REG_INJ
- LL_ADC_AWD_CHANNEL_12_REG
- LL_ADC_AWD_CHANNEL_12_INJ
- LL_ADC_AWD_CHANNEL_12_REG_INJ
- LL_ADC_AWD_CHANNEL_13_REG
- LL_ADC_AWD_CHANNEL_13_INJ
- LL_ADC_AWD_CHANNEL_13_REG_INJ
- LL_ADC_AWD_CHANNEL_14_REG
- LL_ADC_AWD_CHANNEL_14_INJ
- LL_ADC_AWD_CHANNEL_14_REG_INJ

- LL_ADC_AWD_CHANNEL_15_REG
- LL_ADC_AWD_CHANNEL_15_INJ
- LL_ADC_AWD_CHANNEL_15_REG_INJ
- LL_ADC_AWD_CHANNEL_16_REG
- LL_ADC_AWD_CHANNEL_16_INJ
- LL_ADC_AWD_CHANNEL_16_REG_INJ
- LL_ADC_AWD_CHANNEL_17_REG
- LL_ADC_AWD_CHANNEL_17_INJ
- LL_ADC_AWD_CHANNEL_17_REG_INJ
- LL_ADC_AWD_CHANNEL_18_REG
- LL_ADC_AWD_CHANNEL_18_INJ
- LL_ADC_AWD_CHANNEL_18_REG_INJ
- LL_ADC_AWD_CH_VREFINT_REG (1)
- LL_ADC_AWD_CH_VREFINT_INJ (1)
- LL_ADC_AWD_CH_VREFINT_REG_INJ (1)
- LL_ADC_AWD_CH_TEMPSENSOR_REG (1)(2)
- LL_ADC_AWD_CH_TEMPSENSOR_INJ (1)(2)
- LL_ADC_AWD_CH_TEMPSENSOR_REG_INJ (1)(2)
- LL_ADC_AWD_CH_VBAT_REG (1)
- LL_ADC_AWD_CH_VBAT_INJ (1)
- LL_ADC_AWD_CH_VBAT_REG_INJ (1)

(1) On STM32F4, parameter available only on ADC instance: ADC1.

- (2) On devices STM32F42x and STM32F43x, limitation: this internal channel is shared between temperature sensor and Vbat, only 1 measurement path must be enabled.

Return values

- **None:**

Notes

- Once monitored channels are selected, analog watchdog is enabled.
- In case of need to define a single channel to monitor with analog watchdog from sequencer channel definition, use helper macro `__LL_ADC_ANALOGWD_CHANNEL_GROUP()`.
- On this STM32 series, there is only 1 kind of analog watchdog instance: AWD standard (instance AWD1): channels monitored: can monitor 1 channel or all channels.groups monitored: ADC groups regular and-or injected.resolution: resolution is not limited (corresponds to ADC resolution configured).

Reference Manual to LL API cross reference:

- CR1 AWD1CH LL_ADC_SetAnalogWDMonitChannels
- CR1 AWD1SGL LL_ADC_SetAnalogWDMonitChannels
- CR1 AWD1EN LL_ADC_SetAnalogWDMonitChannels

LL_ADC_GetAnalogWDMonitChannels

Function name

`__STATIC_INLINE uint32_t LL_ADC_GetAnalogWDMonitChannels (ADC_TypeDef * ADCx)`

Function description

Get ADC analog watchdog monitored channel.

Parameters

- **ADCx:** ADC instance

Return values

- **Returned:** value can be one of the following values:
 - LL_ADC_AWD_DISABLE
 - LL_ADC_AWD_ALL_CHANNELS_REG
 - LL_ADC_AWD_ALL_CHANNELS_INJ
 - LL_ADC_AWD_ALL_CHANNELS_REG_INJ
 - LL_ADC_AWD_CHANNEL_0_REG
 - LL_ADC_AWD_CHANNEL_0_INJ
 - LL_ADC_AWD_CHANNEL_0_REG_INJ
 - LL_ADC_AWD_CHANNEL_1_REG
 - LL_ADC_AWD_CHANNEL_1_INJ
 - LL_ADC_AWD_CHANNEL_1_REG_INJ
 - LL_ADC_AWD_CHANNEL_2_REG
 - LL_ADC_AWD_CHANNEL_2_INJ
 - LL_ADC_AWD_CHANNEL_2_REG_INJ
 - LL_ADC_AWD_CHANNEL_3_REG
 - LL_ADC_AWD_CHANNEL_3_INJ
 - LL_ADC_AWD_CHANNEL_3_REG_INJ
 - LL_ADC_AWD_CHANNEL_4_REG
 - LL_ADC_AWD_CHANNEL_4_INJ
 - LL_ADC_AWD_CHANNEL_4_REG_INJ
 - LL_ADC_AWD_CHANNEL_5_REG
 - LL_ADC_AWD_CHANNEL_5_INJ
 - LL_ADC_AWD_CHANNEL_5_REG_INJ
 - LL_ADC_AWD_CHANNEL_6_REG
 - LL_ADC_AWD_CHANNEL_6_INJ
 - LL_ADC_AWD_CHANNEL_6_REG_INJ
 - LL_ADC_AWD_CHANNEL_7_REG
 - LL_ADC_AWD_CHANNEL_7_INJ
 - LL_ADC_AWD_CHANNEL_7_REG_INJ
 - LL_ADC_AWD_CHANNEL_8_REG
 - LL_ADC_AWD_CHANNEL_8_INJ
 - LL_ADC_AWD_CHANNEL_8_REG_INJ
 - LL_ADC_AWD_CHANNEL_9_REG
 - LL_ADC_AWD_CHANNEL_9_INJ
 - LL_ADC_AWD_CHANNEL_9_REG_INJ
 - LL_ADC_AWD_CHANNEL_10_REG
 - LL_ADC_AWD_CHANNEL_10_INJ
 - LL_ADC_AWD_CHANNEL_10_REG_INJ
 - LL_ADC_AWD_CHANNEL_11_REG
 - LL_ADC_AWD_CHANNEL_11_INJ
 - LL_ADC_AWD_CHANNEL_11_REG_INJ
 - LL_ADC_AWD_CHANNEL_12_REG
 - LL_ADC_AWD_CHANNEL_12_INJ
 - LL_ADC_AWD_CHANNEL_12_REG_INJ
 - LL_ADC_AWD_CHANNEL_13_REG
 - LL_ADC_AWD_CHANNEL_13_INJ
 - LL_ADC_AWD_CHANNEL_13_REG_INJ
 - LL_ADC_AWD_CHANNEL_14_REG
 - LL_ADC_AWD_CHANNEL_14_INJ

- LL_ADC_AWD_CHANNEL_14_REG_INJ
- LL_ADC_AWD_CHANNEL_15_REG
- LL_ADC_AWD_CHANNEL_15_INJ
- LL_ADC_AWD_CHANNEL_15_REG_INJ
- LL_ADC_AWD_CHANNEL_16_REG
- LL_ADC_AWD_CHANNEL_16_INJ
- LL_ADC_AWD_CHANNEL_16_REG_INJ
- LL_ADC_AWD_CHANNEL_17_REG
- LL_ADC_AWD_CHANNEL_17_INJ
- LL_ADC_AWD_CHANNEL_17_REG_INJ
- LL_ADC_AWD_CHANNEL_18_REG
- LL_ADC_AWD_CHANNEL_18_INJ
- LL_ADC_AWD_CHANNEL_18_REG_INJ

Notes

- Usage of the returned channel number: To reinject this channel into another function LL_ADC_xxx: the returned channel number is only partly formatted on definition of literals LL_ADC_CHANNEL_x. Therefore, it has to be compared with parts of literals LL_ADC_CHANNEL_x or using helper macro __LL_ADC_CHANNEL_TO_DECIMAL_NB(). Then the selected literal LL_ADC_CHANNEL_x can be used as parameter for another function. To get the channel number in decimal format: process the returned value with the helper macro __LL_ADC_CHANNEL_TO_DECIMAL_NB(). Applicable only when the analog watchdog is set to monitor one channel.
- On this STM32 series, there is only 1 kind of analog watchdog instance: AWD standard (instance AWD1): channels monitored: can monitor 1 channel or all channels. groups monitored: ADC groups regular and-or injected. resolution: resolution is not limited (corresponds to ADC resolution configured).

Reference Manual to LL API cross reference:

- CR1 AWD1CH LL_ADC_GetAnalogWDMonitChannels
- CR1 AWD1SGL LL_ADC_GetAnalogWDMonitChannels
- CR1 AWD1EN LL_ADC_GetAnalogWDMonitChannels

LL_ADC_SetAnalogWDThresholds

Function name

__STATIC_INLINE void LL_ADC_SetAnalogWDThresholds (ADC_TypeDef * ADCx, uint32_t AWDThresholdsHighLow, uint32_t AWDThresholdValue)

Function description

Set ADC analog watchdog threshold value of threshold high or low.

Parameters

- **ADCx**: ADC instance
- **AWDThresholdsHighLow**: This parameter can be one of the following values:
 - LL_ADC_AWD_THRESHOLD_HIGH
 - LL_ADC_AWD_THRESHOLD_LOW
- **AWDThresholdValue**: Value between Min_Data=0x000 and Max_Data=0xFF

Return values

- **None:**

Notes

- In case of ADC resolution different of 12 bits, analog watchdog thresholds data require a specific shift. Use helper macro __LL_ADC_ANALOGWD_SET_THRESHOLD_RESOLUTION().
- On this STM32 series, there is only 1 kind of analog watchdog instance: AWD standard (instance AWD1): channels monitored: can monitor 1 channel or all channels. groups monitored: ADC groups regular and-or injected. resolution: resolution is not limited (corresponds to ADC resolution configured).

Reference Manual to LL API cross reference:

- HTR HT LL_ADC_SetAnalogWDThresholds
- LTR LT LL_ADC_SetAnalogWDThresholds

LL_ADC_GetAnalogWDThresholds

Function name

```
__STATIC_INLINE uint32_t LL_ADC_GetAnalogWDThresholds (ADC_TypeDef * ADCx, uint32_t AWDThresholdsHighLow)
```

Function description

Get ADC analog watchdog threshold value of threshold high or threshold low.

Parameters

- **ADCx:** ADC instance
- **AWDThresholdsHighLow:** This parameter can be one of the following values:
 - LL_ADC_AWD_THRESHOLD_HIGH
 - LL_ADC_AWD_THRESHOLD_LOW

Return values

- **Value:** between Min_Data=0x000 and Max_Data=0xFF

Notes

- In case of ADC resolution different of 12 bits, analog watchdog thresholds data require a specific shift. Use helper macro `__LL_ADC_ANALOGWD_GET_THRESHOLD_RESOLUTION()`.

Reference Manual to LL API cross reference:

- HTR HT LL_ADC_GetAnalogWDThresholds
- LTR LT LL_ADC_GetAnalogWDThresholds

LL_ADC_SetMultimode

Function name

```
__STATIC_INLINE void LL_ADC_SetMultimode (ADC_Common_TypeDef * ADCxy_COMMON, uint32_t Multimode)
```

Function description

Set ADC multimode configuration to operate in independent mode or multimode (for devices with several ADC instances).

Parameters

- **ADCxy_COMMON:** ADC common instance (can be set directly from CMSIS definition or by using helper macro `__LL_ADC_COMMON_INSTANCE()`)
- **Multimode:** This parameter can be one of the following values:
 - `LL_ADC_MULTI_INDEPENDENT`
 - `LL_ADC_MULTI_DUAL_REG_SIMULT`
 - `LL_ADC_MULTI_DUAL_REG_INTERL`
 - `LL_ADC_MULTI_DUAL_INJ_SIMULT`
 - `LL_ADC_MULTI_DUAL_INJ_ALTERN`
 - `LL_ADC_MULTI_DUAL_REG_SIM_INJ_SIM`
 - `LL_ADC_MULTI_DUAL_REG_SIM_INJ_ALT`
 - `LL_ADC_MULTI_DUAL_REG_INT_INJ_SIM`
 - `LL_ADC_MULTI_TRIPLE_REG_SIM_INJ_SIM`
 - `LL_ADC_MULTI_TRIPLE_REG_SIM_INJ_ALT`
 - `LL_ADC_MULTI_TRIPLE_INJ_SIMULT`
 - `LL_ADC_MULTI_TRIPLE_REG_SIMULT`
 - `LL_ADC_MULTI_TRIPLE_REG_INTERL`
 - `LL_ADC_MULTI_TRIPLE_INJ_ALTERN`

Return values

- **None:**

Notes

- If multimode configuration: the selected ADC instance is either master or slave depending on hardware. Refer to reference manual.

Reference Manual to LL API cross reference:

- CCR MULTI `LL_ADC_SetMultimode`

LL_ADC_GetMultimode

Function name

`__STATIC_INLINE uint32_t LL_ADC_GetMultimode (ADC_Common_TypeDef * ADCxy_COMMON)`

Function description

Get ADC multimode configuration to operate in independent mode or multimode (for devices with several ADC instances).

Parameters

- **ADCxy_COMMON:** ADC common instance (can be set directly from CMSIS definition or by using helper macro `__LL_ADC_COMMON_INSTANCE()`)

Return values

- **Returned:** value can be one of the following values:
 - LL_ADC_MULTI_INDEPENDENT
 - LL_ADC_MULTI_DUAL_REG_SIMULT
 - LL_ADC_MULTI_DUAL_REG_INTERL
 - LL_ADC_MULTI_DUAL_INJ_SIMULT
 - LL_ADC_MULTI_DUAL_INJ_ALTERN
 - LL_ADC_MULTI_DUAL_REG_SIM_INJ_SIM
 - LL_ADC_MULTI_DUAL_REG_SIM_INJ_ALT
 - LL_ADC_MULTI_DUAL_REG_INT_INJ_SIM
 - LL_ADC_MULTI_TRIPLE_REG_SIM_INJ_SIM
 - LL_ADC_MULTI_TRIPLE_REG_SIM_INJ_ALT
 - LL_ADC_MULTI_TRIPLE_INJ_SIMULT
 - LL_ADC_MULTI_TRIPLE_REG_SIMULT
 - LL_ADC_MULTI_TRIPLE_REG_INTERL
 - LL_ADC_MULTI_TRIPLE_INJ_ALTERN

Notes

- If multimode configuration: the selected ADC instance is either master or slave depending on hardware. Refer to reference manual.

Reference Manual to LL API cross reference:

- CCR MULTI LL_ADC_GetMultimode

LL_ADC_SetMultiDMATransfer

Function name

```
__STATIC_INLINE void LL_ADC_SetMultiDMATransfer (ADC_Common_TypeDef * ADCxy_COMMON,
uint32_t MultiDMATransfer)
```

Function description

Set ADC multimode conversion data transfer: no transfer or transfer by DMA.

Parameters

- **ADCxy_COMMON:** ADC common instance (can be set directly from CMSIS definition or by using helper macro `__LL_ADC_COMMON_INSTANCE()`)
- **MultiDMATransfer:** This parameter can be one of the following values:
 - LL_ADC_MULTI_REG_DMA_EACH_ADC
 - LL_ADC_MULTI_REG_DMA_LIMIT_1
 - LL_ADC_MULTI_REG_DMA_LIMIT_2
 - LL_ADC_MULTI_REG_DMA_LIMIT_3
 - LL_ADC_MULTI_REG_DMA_UNLMT_1
 - LL_ADC_MULTI_REG_DMA_UNLMT_2
 - LL_ADC_MULTI_REG_DMA_UNLMT_3

Return values

- **None:**

Notes

- If ADC multimode transfer by DMA is not selected: each ADC uses its own DMA channel, with its individual DMA transfer settings. If ADC multimode transfer by DMA is selected: One DMA channel is used for both ADC (DMA of ADC master) Specifies the DMA requests mode: Limited mode (One shot mode): DMA transfer requests are stopped when number of DMA data transfers (number of ADC conversions) is reached. This ADC mode is intended to be used with DMA mode non-circular. Unlimited mode: DMA transfer requests are unlimited, whatever number of DMA data transfers (number of ADC conversions). This ADC mode is intended to be used with DMA mode circular.
- If ADC DMA requests mode is set to unlimited and DMA is set to mode non-circular: when DMA transfers size will be reached, DMA will stop transfers of ADC conversions data ADC will raise an overrun error (overrun flag and interruption if enabled).
- How to retrieve multimode conversion data: Whatever multimode transfer by DMA setting: using function `LL_ADC_REG_ReadMultiConversionData32()`. If ADC multimode transfer by DMA is selected: conversion data is a raw data with ADC master and slave concatenated. A macro is available to get the conversion data of ADC master or ADC slave: see helper macro `__LL_ADC_MULTI_CONV_DATA_MASTER_SLAVE()`.

Reference Manual to LL API cross reference:

- CCR MDMA `LL_ADC_SetMultiDMATransfer`
- CCR DDS `LL_ADC_SetMultiDMATransfer`

LL_ADC_GetMultiDMATransfer

Function name

`__STATIC_INLINE uint32_t LL_ADC_GetMultiDMATransfer (ADC_Common_TypeDef * ADCxy_COMMON)`

Function description

Get ADC multimode conversion data transfer: no transfer or transfer by DMA.

Parameters

- **ADCxy_COMMON:** ADC common instance (can be set directly from CMSIS definition or by using helper macro `__LL_ADC_COMMON_INSTANCE()`)

Return values

- **Returned:** value can be one of the following values:
 - `LL_ADC_MULTI_REG_DMA_EACH_ADC`
 - `LL_ADC_MULTI_REG_DMA_LIMIT_1`
 - `LL_ADC_MULTI_REG_DMA_LIMIT_2`
 - `LL_ADC_MULTI_REG_DMA_LIMIT_3`
 - `LL_ADC_MULTI_REG_DMA_UNLMT_1`
 - `LL_ADC_MULTI_REG_DMA_UNLMT_2`
 - `LL_ADC_MULTI_REG_DMA_UNLMT_3`

Notes

- If ADC multimode transfer by DMA is not selected: each ADC uses its own DMA channel, with its individual DMA transfer settings. If ADC multimode transfer by DMA is selected: One DMA channel is used for both ADC (DMA of ADC master) Specifies the DMA requests mode: Limited mode (One shot mode): DMA transfer requests are stopped when number of DMA data transfers (number of ADC conversions) is reached. This ADC mode is intended to be used with DMA mode non-circular. Unlimited mode: DMA transfer requests are unlimited, whatever number of DMA data transfers (number of ADC conversions). This ADC mode is intended to be used with DMA mode circular.
- If ADC DMA requests mode is set to unlimited and DMA is set to mode non-circular: when DMA transfers size will be reached, DMA will stop transfers of ADC conversions data ADC will raise an overrun error (overrun flag and interruption if enabled).
- How to retrieve multimode conversion data: Whatever multimode transfer by DMA setting: using function `LL_ADC_REG_ReadMultiConversionData32()`. If ADC multimode transfer by DMA is selected: conversion data is a raw data with ADC master and slave concatenated. A macro is available to get the conversion data of ADC master or ADC slave: see helper macro `__LL_ADC_MULTI_CONV_DATA_MASTER_SLAVE()`.

Reference Manual to LL API cross reference:

- CCR MDMA `LL_ADC_GetMultiDMATransfer`
- CCR DDS `LL_ADC_GetMultiDMATransfer`

LL_ADC_SetMultiTwoSamplingDelay

Function name

`__STATIC_INLINE void LL_ADC_SetMultiTwoSamplingDelay (ADC_Common_TypeDef * ADCxy_COMMON, uint32_t MultiTwoSamplingDelay)`

Function description

Set ADC multimode delay between 2 sampling phases.

Parameters

- **ADCxy_COMMON:** ADC common instance (can be set directly from CMSIS definition or by using helper macro `__LL_ADC_COMMON_INSTANCE()`)
- **MultiTwoSamplingDelay:** This parameter can be one of the following values:
 - `LL_ADC_MULTI_TWOSMP_DELAY_5CYCLES`
 - `LL_ADC_MULTI_TWOSMP_DELAY_6CYCLES`
 - `LL_ADC_MULTI_TWOSMP_DELAY_7CYCLES`
 - `LL_ADC_MULTI_TWOSMP_DELAY_8CYCLES`
 - `LL_ADC_MULTI_TWOSMP_DELAY_9CYCLES`
 - `LL_ADC_MULTI_TWOSMP_DELAY_10CYCLES`
 - `LL_ADC_MULTI_TWOSMP_DELAY_11CYCLES`
 - `LL_ADC_MULTI_TWOSMP_DELAY_12CYCLES`
 - `LL_ADC_MULTI_TWOSMP_DELAY_13CYCLES`
 - `LL_ADC_MULTI_TWOSMP_DELAY_14CYCLES`
 - `LL_ADC_MULTI_TWOSMP_DELAY_15CYCLES`
 - `LL_ADC_MULTI_TWOSMP_DELAY_16CYCLES`
 - `LL_ADC_MULTI_TWOSMP_DELAY_17CYCLES`
 - `LL_ADC_MULTI_TWOSMP_DELAY_18CYCLES`
 - `LL_ADC_MULTI_TWOSMP_DELAY_19CYCLES`
 - `LL_ADC_MULTI_TWOSMP_DELAY_20CYCLES`

Return values

- **None:**

Notes

- The sampling delay range depends on ADC resolution: ADC resolution 12 bits can have maximum delay of 12 cycles. ADC resolution 10 bits can have maximum delay of 10 cycles. ADC resolution 8 bits can have maximum delay of 8 cycles. ADC resolution 6 bits can have maximum delay of 6 cycles.

Reference Manual to LL API cross reference:

- CCR DELAY LL_ADC_SetMultiTwoSamplingDelay

LL_ADC_GetMultiTwoSamplingDelay

Function name

__STATIC_INLINE uint32_t LL_ADC_GetMultiTwoSamplingDelay (ADC_Common_TypeDef * ADCxy_COMMON)

Function description

Get ADC multimode delay between 2 sampling phases.

Parameters

- ADCxy_COMMON:** ADC common instance (can be set directly from CMSIS definition or by using helper macro `__LL_ADC_COMMON_INSTANCE()`)

Return values

- Returned:** value can be one of the following values:
 - LL_ADC_MULTI_TWOSMP_DELAY_5CYCLES
 - LL_ADC_MULTI_TWOSMP_DELAY_6CYCLES
 - LL_ADC_MULTI_TWOSMP_DELAY_7CYCLES
 - LL_ADC_MULTI_TWOSMP_DELAY_8CYCLES
 - LL_ADC_MULTI_TWOSMP_DELAY_9CYCLES
 - LL_ADC_MULTI_TWOSMP_DELAY_10CYCLES
 - LL_ADC_MULTI_TWOSMP_DELAY_11CYCLES
 - LL_ADC_MULTI_TWOSMP_DELAY_12CYCLES
 - LL_ADC_MULTI_TWOSMP_DELAY_13CYCLES
 - LL_ADC_MULTI_TWOSMP_DELAY_14CYCLES
 - LL_ADC_MULTI_TWOSMP_DELAY_15CYCLES
 - LL_ADC_MULTI_TWOSMP_DELAY_16CYCLES
 - LL_ADC_MULTI_TWOSMP_DELAY_17CYCLES
 - LL_ADC_MULTI_TWOSMP_DELAY_18CYCLES
 - LL_ADC_MULTI_TWOSMP_DELAY_19CYCLES
 - LL_ADC_MULTI_TWOSMP_DELAY_20CYCLES

Reference Manual to LL API cross reference:

- CCR DELAY LL_ADC_GetMultiTwoSamplingDelay

LL_ADC_Enable

Function name

__STATIC_INLINE void LL_ADC_Enable (ADC_TypeDef * ADCx)

Function description

Enable the selected ADC instance.

Parameters

- ADCx:** ADC instance

Return values

- **None:**

Notes

- On this STM32 series, after ADC enable, a delay for ADC internal analog stabilization is required before performing a ADC conversion start. Refer to device datasheet, parameter tSTAB.

Reference Manual to LL API cross reference:

- CR2 ADON LL_ADC_Enable

LL_ADC_Disable

Function name

```
__STATIC_INLINE void LL_ADC_Disable (ADC_TypeDef * ADCx)
```

Function description

Disable the selected ADC instance.

Parameters

- **ADCx:** ADC instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR2 ADON LL_ADC_Disable

LL_ADC_IsEnabled

Function name

```
__STATIC_INLINE uint32_t LL_ADC_IsEnabled (ADC_TypeDef * ADCx)
```

Function description

Get the selected ADC instance enable state.

Parameters

- **ADCx:** ADC instance

Return values

- **0:** ADC is disabled, **1:** ADC is enabled.

Reference Manual to LL API cross reference:

- CR2 ADON LL_ADC_IsEnabled

LL_ADC_REG_StartConversionSWStart

Function name

```
__STATIC_INLINE void LL_ADC_REG_StartConversionSWStart (ADC_TypeDef * ADCx)
```

Function description

Start ADC group regular conversion.

Parameters

- **ADCx:** ADC instance

Return values

- **None:**

Notes

- On this STM32 series, this function is relevant only for internal trigger (SW start), not for external trigger: If ADC trigger has been set to software start, ADC conversion starts immediately. If ADC trigger has been set to external trigger, ADC conversion start must be performed using function LL_ADC_REG_StartConversionExtTrig(). (if external trigger edge would have been set during ADC other settings, ADC conversion would start at trigger event as soon as ADC is enabled).

Reference Manual to LL API cross reference:

- CR2 SWSTART LL_ADC_REG_StartConversionSWStart

LL_ADC_REG_StartConversionExtTrig

Function name

```
__STATIC_INLINE void LL_ADC_REG_StartConversionExtTrig (ADC_TypeDef * ADCx, uint32_t ExternalTriggerEdge)
```

Function description

Start ADC group regular conversion from external trigger.

Parameters

- ExternalTriggerEdge:** This parameter can be one of the following values:
 - LL_ADC_REG_TRIG_EXT_RISING
 - LL_ADC_REG_TRIG_EXT_FALLING
 - LL_ADC_REG_TRIG_EXT_RISINGFALLING
- ADCx:** ADC instance

Return values

- None:**

Notes

- ADC conversion will start at next trigger event (on the selected trigger edge) following the ADC start conversion command.
- On this STM32 series, this function is relevant for ADC conversion start from external trigger. If internal trigger (SW start) is needed, perform ADC conversion start using function LL_ADC_REG_StartConversionSWStart().

Reference Manual to LL API cross reference:

- CR2 EXTEN LL_ADC_REG_StartConversionExtTrig

LL_ADC_REG_StopConversionExtTrig

Function name

```
__STATIC_INLINE void LL_ADC_REG_StopConversionExtTrig (ADC_TypeDef * ADCx)
```

Function description

Stop ADC group regular conversion from external trigger.

Parameters

- ADCx:** ADC instance

Return values

- None:**

Notes

- No more ADC conversion will start at next trigger event following the ADC stop conversion command. If a conversion is on-going, it will be completed.
- On this STM32 series, there is no specific command to stop a conversion on-going or to stop ADC converting in continuous mode. These actions can be performed using function LL_ADC_Disable().

Reference Manual to LL API cross reference:

- CR2_EXTEN LL_ADC_REG_StopConversionExtTrig

LL_ADC_REG_ReadConversionData32

Function name

__STATIC_INLINE uint32_t LL_ADC_REG_ReadConversionData32 (ADC_TypeDef * ADCx)

Function description

Get ADC group regular conversion data, range fit for all ADC configurations: all ADC resolutions and all oversampling increased data width (for devices with feature oversampling).

Parameters

- **ADCx**: ADC instance

Return values

- **Value**: between Min_Data=0x00000000 and Max_Data=0xFFFFFFFF

Reference Manual to LL API cross reference:

- DR_RDATA LL_ADC_REG_ReadConversionData32

LL_ADC_REG_ReadConversionData12

Function name

__STATIC_INLINE uint16_t LL_ADC_REG_ReadConversionData12 (ADC_TypeDef * ADCx)

Function description

Get ADC group regular conversion data, range fit for ADC resolution 12 bits.

Parameters

- **ADCx**: ADC instance

Return values

- **Value**: between Min_Data=0x000 and Max_Data=0xFFF

Notes

- For devices with feature oversampling: Oversampling can increase data width, function for extended range may be needed: LL_ADC_REG_ReadConversionData32.

Reference Manual to LL API cross reference:

- DR_RDATA LL_ADC_REG_ReadConversionData12

LL_ADC_REG_ReadConversionData10

Function name

__STATIC_INLINE uint16_t LL_ADC_REG_ReadConversionData10 (ADC_TypeDef * ADCx)

Function description

Get ADC group regular conversion data, range fit for ADC resolution 10 bits.

Parameters

- **ADCx**: ADC instance

Return values

- **Value:** between Min_Data=0x000 and Max_Data=0x3FF

Notes

- For devices with feature oversampling: Oversampling can increase data width, function for extended range may be needed: LL_ADC_REG_ReadConversionData32.

Reference Manual to LL API cross reference:

- DR RDATA LL_ADC_REG_ReadConversionData10

LL_ADC_REG_ReadConversionData8

Function name

```
__STATIC_INLINE uint8_t LL_ADC_REG_ReadConversionData8 (ADC_TypeDef * ADCx)
```

Function description

Get ADC group regular conversion data, range fit for ADC resolution 8 bits.

Parameters

- **ADCx:** ADC instance

Return values

- **Value:** between Min_Data=0x00 and Max_Data=0xFF

Notes

- For devices with feature oversampling: Oversampling can increase data width, function for extended range may be needed: LL_ADC_REG_ReadConversionData32.

Reference Manual to LL API cross reference:

- DR RDATA LL_ADC_REG_ReadConversionData8

LL_ADC_REG_ReadConversionData6

Function name

```
__STATIC_INLINE uint8_t LL_ADC_REG_ReadConversionData6 (ADC_TypeDef * ADCx)
```

Function description

Get ADC group regular conversion data, range fit for ADC resolution 6 bits.

Parameters

- **ADCx:** ADC instance

Return values

- **Value:** between Min_Data=0x00 and Max_Data=0x3F

Notes

- For devices with feature oversampling: Oversampling can increase data width, function for extended range may be needed: LL_ADC_REG_ReadConversionData32.

Reference Manual to LL API cross reference:

- DR RDATA LL_ADC_REG_ReadConversionData6

LL_ADC_REG_ReadMultiConversionData32

Function name

```
__STATIC_INLINE uint32_t LL_ADC_REG_ReadMultiConversionData32 (ADC_Common_TypeDef *  
ADCxy_COMMON, uint32_t ConversionData)
```

Function description

Get ADC multimode conversion data of ADC master, ADC slave or raw data with ADC master and slave concatenated.

Parameters

- **ADCxy_COMMON:** ADC common instance (can be set directly from CMSIS definition or by using helper macro `__LL_ADC_COMMON_INSTANCE()`)
- **ConversionData:** This parameter can be one of the following values:
 - `LL_ADC_MULTI_MASTER`
 - `LL_ADC_MULTI_SLAVE`
 - `LL_ADC_MULTI_MASTER_SLAVE`

Return values

- **Value:** between `Min_Data=0x00000000` and `Max_Data=0xFFFFFFFF`

Notes

- If raw data with ADC master and slave concatenated is retrieved, a macro is available to get the conversion data of ADC master or ADC slave: see helper macro `__LL_ADC_MULTI_CONV_DATA_MASTER_SLAVE()`. (however this macro is mainly intended for multimode transfer by DMA, because this function can do the same by getting multimode conversion data of ADC master or ADC slave separately).

Reference Manual to LL API cross reference:

- CDR DATA1 `LL_ADC_REG_ReadMultiConversionData32`
- CDR DATA2 `LL_ADC_REG_ReadMultiConversionData32`

LL_ADC_INJ_StartConversionSWStart

Function name

`__STATIC_INLINE void LL_ADC_INJ_StartConversionSWStart (ADC_TypeDef * ADCx)`

Function description

Start ADC group injected conversion.

Parameters

- **ADCx:** ADC instance

Return values

- **None:**

Notes

- On this STM32 series, this function is relevant only for internal trigger (SW start), not for external trigger: If ADC trigger has been set to software start, ADC conversion starts immediately. If ADC trigger has been set to external trigger, ADC conversion start must be performed using function `LL_ADC_INJ_StartConversionExtTrig()`. (if external trigger edge would have been set during ADC other settings, ADC conversion would start at trigger event as soon as ADC is enabled).

Reference Manual to LL API cross reference:

- CR2 JSWSTART `LL_ADC_INJ_StartConversionSWStart`

LL_ADC_INJ_StartConversionExtTrig

Function name

`__STATIC_INLINE void LL_ADC_INJ_StartConversionExtTrig (ADC_TypeDef * ADCx, uint32_t ExternalTriggerEdge)`

Function description

Start ADC group injected conversion from external trigger.

Parameters

- **ExternalTriggerEdge:** This parameter can be one of the following values:
 - LL_ADC_INJ_TRIG_EXT_RISING
 - LL_ADC_INJ_TRIG_EXT_FALLING
 - LL_ADC_INJ_TRIG_EXT_RISINGFALLING
- **ADCx:** ADC instance

Return values

- **None:**

Notes

- ADC conversion will start at next trigger event (on the selected trigger edge) following the ADC start conversion command.
- On this STM32 series, this function is relevant for ADC conversion start from external trigger. If internal trigger (SW start) is needed, perform ADC conversion start using function LL_ADC_INJ_StartConversionSWStart().

Reference Manual to LL API cross reference:

- CR2 JEXTEN LL_ADC_INJ_StartConversionExtTrig

LL_ADC_INJ_StopConversionExtTrig

Function name

__STATIC_INLINE void LL_ADC_INJ_StopConversionExtTrig (ADC_TypeDef * ADCx)

Function description

Stop ADC group injected conversion from external trigger.

Parameters

- **ADCx:** ADC instance

Return values

- **None:**

Notes

- No more ADC conversion will start at next trigger event following the ADC stop conversion command. If a conversion is on-going, it will be completed.
- On this STM32 series, there is no specific command to stop a conversion on-going or to stop ADC converting in continuous mode. These actions can be performed using function LL_ADC_Disable().

Reference Manual to LL API cross reference:

- CR2 JEXTEN LL_ADC_INJ_StopConversionExtTrig

LL_ADC_INJ_ReadConversionData32

Function name

__STATIC_INLINE uint32_t LL_ADC_INJ_ReadConversionData32 (ADC_TypeDef * ADCx, uint32_t Rank)

Function description

Get ADC group regular conversion data, range fit for all ADC configurations: all ADC resolutions and all oversampling increased data width (for devices with feature oversampling).

Parameters

- **ADCx:** ADC instance
- **Rank:** This parameter can be one of the following values:
 - LL_ADC_INJ_RANK_1
 - LL_ADC_INJ_RANK_2
 - LL_ADC_INJ_RANK_3
 - LL_ADC_INJ_RANK_4

Return values

- **Value:** between Min_Data=0x00000000 and Max_Data=0xFFFFFFFF

Reference Manual to LL API cross reference:

- JDR1 JDATA LL_ADC_INJ_ReadConversionData32
- JDR2 JDATA LL_ADC_INJ_ReadConversionData32
- JDR3 JDATA LL_ADC_INJ_ReadConversionData32
- JDR4 JDATA LL_ADC_INJ_ReadConversionData32

LL_ADC_INJ_ReadConversionData12

Function name

```
__STATIC_INLINE uint16_t LL_ADC_INJ_ReadConversionData12 (ADC_TypeDef * ADCx, uint32_t Rank)
```

Function description

Get ADC group injected conversion data, range fit for ADC resolution 12 bits.

Parameters

- **ADCx:** ADC instance
- **Rank:** This parameter can be one of the following values:
 - LL_ADC_INJ_RANK_1
 - LL_ADC_INJ_RANK_2
 - LL_ADC_INJ_RANK_3
 - LL_ADC_INJ_RANK_4

Return values

- **Value:** between Min_Data=0x000 and Max_Data=0xFFF

Notes

- For devices with feature oversampling: Oversampling can increase data width, function for extended range may be needed: LL_ADC_INJ_ReadConversionData32.

Reference Manual to LL API cross reference:

- JDR1 JDATA LL_ADC_INJ_ReadConversionData12
- JDR2 JDATA LL_ADC_INJ_ReadConversionData12
- JDR3 JDATA LL_ADC_INJ_ReadConversionData12
- JDR4 JDATA LL_ADC_INJ_ReadConversionData12

LL_ADC_INJ_ReadConversionData10

Function name

```
__STATIC_INLINE uint16_t LL_ADC_INJ_ReadConversionData10 (ADC_TypeDef * ADCx, uint32_t Rank)
```

Function description

Get ADC group injected conversion data, range fit for ADC resolution 10 bits.

Parameters

- **ADCx:** ADC instance
- **Rank:** This parameter can be one of the following values:
 - LL_ADC_INJ_RANK_1
 - LL_ADC_INJ_RANK_2
 - LL_ADC_INJ_RANK_3
 - LL_ADC_INJ_RANK_4

Return values

- **Value:** between Min_Data=0x000 and Max_Data=0x3FF

Notes

- For devices with feature oversampling: Oversampling can increase data width, function for extended range may be needed: LL_ADC_INJ_ReadConversionData32.

Reference Manual to LL API cross reference:

- JDR1 JDATA LL_ADC_INJ_ReadConversionData10
- JDR2 JDATA LL_ADC_INJ_ReadConversionData10
- JDR3 JDATA LL_ADC_INJ_ReadConversionData10
- JDR4 JDATA LL_ADC_INJ_ReadConversionData10

LL_ADC_INJ_ReadConversionData8

Function name

__STATIC_INLINE uint8_t LL_ADC_INJ_ReadConversionData8 (ADC_TypeDef * ADCx, uint32_t Rank)

Function description

Get ADC group injected conversion data, range fit for ADC resolution 8 bits.

Parameters

- **ADCx:** ADC instance
- **Rank:** This parameter can be one of the following values:
 - LL_ADC_INJ_RANK_1
 - LL_ADC_INJ_RANK_2
 - LL_ADC_INJ_RANK_3
 - LL_ADC_INJ_RANK_4

Return values

- **Value:** between Min_Data=0x00 and Max_Data=0xFF

Notes

- For devices with feature oversampling: Oversampling can increase data width, function for extended range may be needed: LL_ADC_INJ_ReadConversionData32.

Reference Manual to LL API cross reference:

- JDR1 JDATA LL_ADC_INJ_ReadConversionData8
- JDR2 JDATA LL_ADC_INJ_ReadConversionData8
- JDR3 JDATA LL_ADC_INJ_ReadConversionData8
- JDR4 JDATA LL_ADC_INJ_ReadConversionData8

LL_ADC_INJ_ReadConversionData6

Function name

__STATIC_INLINE uint8_t LL_ADC_INJ_ReadConversionData6 (ADC_TypeDef * ADCx, uint32_t Rank)

Function description

Get ADC group injected conversion data, range fit for ADC resolution 6 bits.

Parameters

- **ADCx:** ADC instance
- **Rank:** This parameter can be one of the following values:
 - LL_ADC_INJ_RANK_1
 - LL_ADC_INJ_RANK_2
 - LL_ADC_INJ_RANK_3
 - LL_ADC_INJ_RANK_4

Return values

- **Value:** between Min_Data=0x00 and Max_Data=0x3F

Notes

- For devices with feature oversampling: Oversampling can increase data width, function for extended range may be needed: LL_ADC_INJ_ReadConversionData32.

Reference Manual to LL API cross reference:

- JDR1 JDATA LL_ADC_INJ_ReadConversionData6
- JDR2 JDATA LL_ADC_INJ_ReadConversionData6
- JDR3 JDATA LL_ADC_INJ_ReadConversionData6
- JDR4 JDATA LL_ADC_INJ_ReadConversionData6

LL_ADC_IsActiveFlag_EOCS

Function name

__STATIC_INLINE uint32_t LL_ADC_IsActiveFlag_EOCS (ADC_TypeDef * ADCx)

Function description

Get flag ADC group regular end of unitary conversion or end of sequence conversions, depending on ADC configuration.

Parameters

- **ADCx:** ADC instance

Return values

- **State:** of bit (1 or 0).

Notes

- To configure flag of end of conversion, use function LL_ADC_REG_SetFlagEndOfConversion().

Reference Manual to LL API cross reference:

- SR EOC LL_ADC_IsActiveFlag_EOCS

LL_ADC_IsActiveFlag_OVR

Function name

__STATIC_INLINE uint32_t LL_ADC_IsActiveFlag_OVR (ADC_TypeDef * ADCx)

Function description

Get flag ADC group regular overrun.

Parameters

- **ADCx:** ADC instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- SR OVR LL_ADC_IsActiveFlag_OVR

LL_ADC_IsActiveFlag_JEOS

Function name

```
__STATIC_INLINE uint32_t LL_ADC_IsActiveFlag_JEOS (ADC_TypeDef * ADCx)
```

Function description

Get flag ADC group injected end of sequence conversions.

Parameters

- **ADCx:** ADC instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- SR JEOC LL_ADC_IsActiveFlag_JEOS

LL_ADC_IsActiveFlag_AWD1

Function name

```
__STATIC_INLINE uint32_t LL_ADC_IsActiveFlag_AWD1 (ADC_TypeDef * ADCx)
```

Function description

Get flag ADC analog watchdog 1 flag.

Parameters

- **ADCx:** ADC instance

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- SR AWD LL_ADC_IsActiveFlag_AWD1

LL_ADC_ClearFlag_EOCS

Function name

```
__STATIC_INLINE void LL_ADC_ClearFlag_EOCS (ADC_TypeDef * ADCx)
```

Function description

Clear flag ADC group regular end of unitary conversion or end of sequence conversions, depending on ADC configuration.

Parameters

- **ADCx:** ADC instance

Return values

- **None:**

Notes

- To configure flag of end of conversion, use function LL_ADC_REG_SetFlagEndOfConversion().

Reference Manual to LL API cross reference:

- SR EOC LL_ADC_ClearFlag_EOCS

LL_ADC_ClearFlag_OVR**Function name**

__STATIC_INLINE void LL_ADC_ClearFlag_OVR (ADC_TypeDef * ADCx)

Function description

Clear flag ADC group regular overrun.

Parameters

- **ADCx**: ADC instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- SR OVR LL_ADC_ClearFlag_OVR

LL_ADC_ClearFlag_JEOS**Function name**

__STATIC_INLINE void LL_ADC_ClearFlag_JEOS (ADC_TypeDef * ADCx)

Function description

Clear flag ADC group injected end of sequence conversions.

Parameters

- **ADCx**: ADC instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- SR JEOC LL_ADC_ClearFlag_JEOS

LL_ADC_ClearFlag_AWD1**Function name**

__STATIC_INLINE void LL_ADC_ClearFlag_AWD1 (ADC_TypeDef * ADCx)

Function description

Clear flag ADC analog watchdog 1.

Parameters

- **ADCx**: ADC instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- SR AWD LL_ADC_ClearFlag_AWD1

LL_ADC_IsActiveFlag_MST_EOCS

Function name

```
__STATIC_INLINE uint32_t LL_ADC_IsActiveFlag_MST_EOCS (ADC_Common_TypeDef *  
ADCxy_COMMON)
```

Function description

Get flag multimode ADC group regular end of unitary conversion or end of sequence conversions, depending on ADC configuration, of the ADC master.

Parameters

- **ADCxy_COMMON:** ADC common instance (can be set directly from CMSIS definition or by using helper macro `__LL_ADC_COMMON_INSTANCE()`)

Return values

- **State:** of bit (1 or 0).

Notes

- To configure flag of end of conversion, use function `LL_ADC_REG_SetFlagEndOfConversion()`.

Reference Manual to LL API cross reference:

- CSR EOC1 `LL_ADC_IsActiveFlag_MST_EOCS`

LL_ADC_IsActiveFlag_SLV1_EOCS

Function name

```
__STATIC_INLINE uint32_t LL_ADC_IsActiveFlag_SLV1_EOCS (ADC_Common_TypeDef *  
ADCxy_COMMON)
```

Function description

Get flag multimode ADC group regular end of unitary conversion or end of sequence conversions, depending on ADC configuration, of the ADC slave 1.

Parameters

- **ADCxy_COMMON:** ADC common instance (can be set directly from CMSIS definition or by using helper macro `__LL_ADC_COMMON_INSTANCE()`)

Return values

- **State:** of bit (1 or 0).

Notes

- To configure flag of end of conversion, use function `LL_ADC_REG_SetFlagEndOfConversion()`.

Reference Manual to LL API cross reference:

- CSR EOC2 `LL_ADC_IsActiveFlag_SLV1_EOCS`

LL_ADC_IsActiveFlag_SLV2_EOCS

Function name

```
__STATIC_INLINE uint32_t LL_ADC_IsActiveFlag_SLV2_EOCS (ADC_Common_TypeDef *  
ADCxy_COMMON)
```

Function description

Get flag multimode ADC group regular end of unitary conversion or end of sequence conversions, depending on ADC configuration, of the ADC slave 2.

Parameters

- **ADCxy_COMMON:** ADC common instance (can be set directly from CMSIS definition or by using helper macro `__LL_ADC_COMMON_INSTANCE()`)

Return values

- **State:** of bit (1 or 0).

Notes

- To configure flag of end of conversion, use function `LL_ADC_REG_SetFlagEndOfConversion()`.

Reference Manual to LL API cross reference:

- CSR EOC3 `LL_ADC_IsActiveFlag_SLV2_EOCS`

LL_ADC_IsActiveFlag_MST_OVR

Function name

```
__STATIC_INLINE uint32_t LL_ADC_IsActiveFlag_MST_OVR (ADC_Common_TypeDef *
ADCxy_COMMON)
```

Function description

Get flag multimode ADC group regular overrun of the ADC master.

Parameters

- **ADCxy_COMMON:** ADC common instance (can be set directly from CMSIS definition or by using helper macro `__LL_ADC_COMMON_INSTANCE()`)

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CSR OVR1 `LL_ADC_IsActiveFlag_MST_OVR`

LL_ADC_IsActiveFlag_SLV1_OVR

Function name

```
__STATIC_INLINE uint32_t LL_ADC_IsActiveFlag_SLV1_OVR (ADC_Common_TypeDef *
ADCxy_COMMON)
```

Function description

Get flag multimode ADC group regular overrun of the ADC slave 1.

Parameters

- **ADCxy_COMMON:** ADC common instance (can be set directly from CMSIS definition or by using helper macro `__LL_ADC_COMMON_INSTANCE()`)

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CSR OVR2 `LL_ADC_IsActiveFlag_SLV1_OVR`

LL_ADC_IsActiveFlag_SLV2_OVR

Function name

```
__STATIC_INLINE uint32_t LL_ADC_IsActiveFlag_SLV2_OVR (ADC_Common_TypeDef *
ADCxy_COMMON)
```

Function description

Get flag multimode ADC group regular overrun of the ADC slave 2.

Parameters

- **ADCxy_COMMON:** ADC common instance (can be set directly from CMSIS definition or by using helper macro `__LL_ADC_COMMON_INSTANCE()`)

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CSR OVR3 `LL_ADC_IsActiveFlag_SLV2_OVR`

LL_ADC_IsActiveFlag_MST_JEOS

Function name

```
__STATIC_INLINE uint32_t LL_ADC_IsActiveFlag_MST_JEOS (ADC_Common_TypeDef *
ADCxy_COMMON)
```

Function description

Get flag multimode ADC group injected end of sequence conversions of the ADC master.

Parameters

- **ADCxy_COMMON:** ADC common instance (can be set directly from CMSIS definition or by using helper macro `__LL_ADC_COMMON_INSTANCE()`)

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CSR JEOC `LL_ADC_IsActiveFlag_MST_JEOS`

LL_ADC_IsActiveFlag_SLV1_JEOS

Function name

```
__STATIC_INLINE uint32_t LL_ADC_IsActiveFlag_SLV1_JEOS (ADC_Common_TypeDef *
ADCxy_COMMON)
```

Function description

Get flag multimode ADC group injected end of sequence conversions of the ADC slave 1.

Parameters

- **ADCxy_COMMON:** ADC common instance (can be set directly from CMSIS definition or by using helper macro `__LL_ADC_COMMON_INSTANCE()`)

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CSR JEOC2 `LL_ADC_IsActiveFlag_SLV1_JEOS`

LL_ADC_IsActiveFlag_SLV2_JEOS

Function name

```
__STATIC_INLINE uint32_t LL_ADC_IsActiveFlag_SLV2_JEOS (ADC_Common_TypeDef *
ADCxy_COMMON)
```

Function description

Get flag multimode ADC group injected end of sequence conversions of the ADC slave 2.

Parameters

- **ADCxy_COMMON:** ADC common instance (can be set directly from CMSIS definition or by using helper macro `__LL_ADC_COMMON_INSTANCE()`)

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CSR JE0C3 LL_ADC_IsActiveFlag_SLV2_JEOS

LL_ADC_IsActiveFlag_MST_AWD1

Function name

```
__STATIC_INLINE uint32_t LL_ADC_IsActiveFlag_MST_AWD1 (ADC_Common_TypeDef *
ADCxy_COMMON)
```

Function description

Get flag multimode ADC analog watchdog 1 of the ADC master.

Parameters

- **ADCxy_COMMON:** ADC common instance (can be set directly from CMSIS definition or by using helper macro `__LL_ADC_COMMON_INSTANCE()`)

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CSR AWD1 LL_ADC_IsActiveFlag_MST_AWD1

LL_ADC_IsActiveFlag_SLV1_AWD1

Function name

```
__STATIC_INLINE uint32_t LL_ADC_IsActiveFlag_SLV1_AWD1 (ADC_Common_TypeDef *
ADCxy_COMMON)
```

Function description

Get flag multimode analog watchdog 1 of the ADC slave 1.

Parameters

- **ADCxy_COMMON:** ADC common instance (can be set directly from CMSIS definition or by using helper macro `__LL_ADC_COMMON_INSTANCE()`)

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CSR AWD2 LL_ADC_IsActiveFlag_SLV1_AWD1

LL_ADC_IsActiveFlag_SLV2_AWD1

Function name

```
__STATIC_INLINE uint32_t LL_ADC_IsActiveFlag_SLV2_AWD1 (ADC_Common_TypeDef *
ADCxy_COMMON)
```

Function description

Get flag multimode analog watchdog 1 of the ADC slave 2.

Parameters

- **ADCxy_COMMON:** ADC common instance (can be set directly from CMSIS definition or by using helper macro `__LL_ADC_COMMON_INSTANCE()`)

Return values

- **State:** of bit (1 or 0).

Reference Manual to LL API cross reference:

- CSR AWD3 `LL_ADC_IsActiveFlag_SLV2_AWD1`

LL_ADC_EnableIT_EOCS

Function name

```
__STATIC_INLINE void LL_ADC_EnableIT_EOCS (ADC_TypeDef * ADCx)
```

Function description

Enable interruption ADC group regular end of unitary conversion or end of sequence conversions, depending on ADC configuration.

Parameters

- **ADCx:** ADC instance

Return values

- **None:**

Notes

- To configure flag of end of conversion, use function `LL_ADC_REG_SetFlagEndOfConversion()`.

Reference Manual to LL API cross reference:

- CR1 EOCIE `LL_ADC_EnableIT_EOCS`

LL_ADC_EnableIT_OVR

Function name

```
__STATIC_INLINE void LL_ADC_EnableIT_OVR (ADC_TypeDef * ADCx)
```

Function description

Enable ADC group regular interruption overrun.

Parameters

- **ADCx:** ADC instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR1 OVRIE `LL_ADC_EnableIT_OVR`

LL_ADC_EnableIT_JEOS

Function name

```
__STATIC_INLINE void LL_ADC_EnableIT_JEOS (ADC_TypeDef * ADCx)
```

Function description

Enable interruption ADC group injected end of sequence conversions.

Parameters

- **ADCx**: ADC instance

Return values

- **None**:

Reference Manual to LL API cross reference:

- CR1 JEOCIE LL_ADC_EnableIT_JEOS

LL_ADC_EnableIT_AWD1

Function name

__STATIC_INLINE void LL_ADC_EnableIT_AWD1 (ADC_TypeDef * ADCx)

Function description

Enable interruption ADC analog watchdog 1.

Parameters

- **ADCx**: ADC instance

Return values

- **None**:

Reference Manual to LL API cross reference:

- CR1 AWDIE LL_ADC_EnableIT_AWD1

LL_ADC_DisableIT_EOCS

Function name

__STATIC_INLINE void LL_ADC_DisableIT_EOCS (ADC_TypeDef * ADCx)

Function description

Disable interruption ADC group regular end of unitary conversion or end of sequence conversions, depending on ADC configuration.

Parameters

- **ADCx**: ADC instance

Return values

- **None**:

Notes

- To configure flag of end of conversion, use function LL_ADC_REG_SetFlagEndOfConversion().

Reference Manual to LL API cross reference:

- CR1 EOCIE LL_ADC_DisableIT_EOCS

LL_ADC_DisableIT_OVR

Function name

__STATIC_INLINE void LL_ADC_DisableIT_OVR (ADC_TypeDef * ADCx)

Function description

Disable interruption ADC group regular overrun.

Parameters

- **ADCx**: ADC instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR1 OVR1E LL_ADC_DisableIT_OVR

LL_ADC_DisableIT_JEOS

Function name

```
__STATIC_INLINE void LL_ADC_DisableIT_JEOS (ADC_TypeDef * ADCx)
```

Function description

Disable interruption ADC group injected end of sequence conversions.

Parameters

- **ADCx:** ADC instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR1 JEOC1E LL_ADC_EnableIT_JEOS

LL_ADC_DisableIT_AWD1

Function name

```
__STATIC_INLINE void LL_ADC_DisableIT_AWD1 (ADC_TypeDef * ADCx)
```

Function description

Disable interruption ADC analog watchdog 1.

Parameters

- **ADCx:** ADC instance

Return values

- **None:**

Reference Manual to LL API cross reference:

- CR1 AWD1E LL_ADC_EnableIT_AWD1

LL_ADC_IsEnabledIT_EOCS

Function name

```
__STATIC_INLINE uint32_t LL_ADC_IsEnabledIT_EOCS (ADC_TypeDef * ADCx)
```

Function description

Get state of interruption ADC group regular end of unitary conversion or end of sequence conversions, depending on ADC configuration.

Parameters

- **ADCx:** ADC instance

Return values

- **State:** of bit (1 or 0).

Notes

- To configure flag of end of conversion, use function LL_ADC_REG_SetFlagEndOfConversion(). (0: interrupt disabled, 1: interrupt enabled)

Reference Manual to LL API cross reference:

- CR1 EOCIE LL_ADC_IsEnabledIT_EOCS

LL_ADC_IsEnabledIT_OVR

Function name

```
__STATIC_INLINE uint32_t LL_ADC_IsEnabledIT_OVR (ADC_TypeDef * ADCx)
```

Function description

Get state of interruption ADC group regular overrun (0: interrupt disabled, 1: interrupt enabled).

Parameters

- **ADCx**: ADC instance

Return values

- **State**: of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR1 OVRIE LL_ADC_IsEnabledIT_OVR

LL_ADC_IsEnabledIT_JEOS

Function name

```
__STATIC_INLINE uint32_t LL_ADC_IsEnabledIT_JEOS (ADC_TypeDef * ADCx)
```

Function description

Get state of interruption ADC group injected end of sequence conversions (0: interrupt disabled, 1: interrupt enabled).

Parameters

- **ADCx**: ADC instance

Return values

- **State**: of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR1 JEOCIE LL_ADC_EnableIT_JEOS

LL_ADC_IsEnabledIT_AWD1

Function name

```
__STATIC_INLINE uint32_t LL_ADC_IsEnabledIT_AWD1 (ADC_TypeDef * ADCx)
```

Function description

Get state of interruption ADC analog watchdog 1 (0: interrupt disabled, 1: interrupt enabled).

Parameters

- **ADCx**: ADC instance

Return values

- **State**: of bit (1 or 0).

Reference Manual to LL API cross reference:

- CR1 AWDIE LL_ADC_EnableIT_AWD1

LL_ADC_CommonDeInit

Function name

ErrorStatus LL_ADC_CommonDeInit (ADC_Common_TypeDef * ADCxy_COMMON)

Function description

De-initialize registers of all ADC instances belonging to the same ADC common instance to their default reset values.

Parameters

- **ADCxy_COMMON:** ADC common instance (can be set directly from CMSIS definition or by using helper macro `__LL_ADC_COMMON_INSTANCE()`)

Return values

- **An:** ErrorStatus enumeration value:
 - SUCCESS: ADC common registers are de-initialized
 - ERROR: not applicable

LL_ADC_CommonInit

Function name

ErrorStatus LL_ADC_CommonInit (ADC_Common_TypeDef * ADCxy_COMMON, LL_ADC_CommonInitTypeDef * ADC_CommonInitStruct)

Function description

Initialize some features of ADC common parameters (all ADC instances belonging to the same ADC common instance) and multimode (for devices with several ADC instances available).

Parameters

- **ADCxy_COMMON:** ADC common instance (can be set directly from CMSIS definition or by using helper macro `__LL_ADC_COMMON_INSTANCE()`)
- **ADC_CommonInitStruct:** Pointer to a LL_ADC_CommonInitTypeDef structure

Return values

- **An:** ErrorStatus enumeration value:
 - SUCCESS: ADC common registers are initialized
 - ERROR: ADC common registers are not initialized

Notes

- The setting of ADC common parameters is conditioned to ADC instances state: All ADC instances belonging to the same ADC common instance must be disabled.

LL_ADC_CommonStructInit

Function name

void LL_ADC_CommonStructInit (LL_ADC_CommonInitTypeDef * ADC_CommonInitStruct)

Function description

Set each LL_ADC_CommonInitTypeDef field to default value.

Parameters

- **ADC_CommonInitStruct:** Pointer to a LL_ADC_CommonInitTypeDef structure whose fields will be set to default values.

Return values

- **None:**

LL_ADC_DeInit

Function name

ErrorStatus LL_ADC_DeInit (ADC_TypeDef * ADCx)

Function description

De-initialize registers of the selected ADC instance to their default reset values.

Parameters

- **ADCx:** ADC instance

Return values

- **An:** ErrorStatus enumeration value:
 - SUCCESS: ADC registers are de-initialized
 - ERROR: ADC registers are not de-initialized

Notes

- To reset all ADC instances quickly (perform a hard reset), use function LL_ADC_CommonDeInit().

LL_ADC_Init

Function name

ErrorStatus LL_ADC_Init (ADC_TypeDef * ADCx, LL_ADC_InitTypeDef * ADC_InitStruct)

Function description

Initialize some features of ADC instance.

Parameters

- **ADCx:** ADC instance
- **ADC_InitStruct:** Pointer to a LL_ADC_REG_InitTypeDef structure

Return values

- **An:** ErrorStatus enumeration value:
 - SUCCESS: ADC registers are initialized
 - ERROR: ADC registers are not initialized

Notes

- These parameters have an impact on ADC scope: ADC instance. Affects both group regular and group injected (availability of ADC group injected depends on STM32 families). Refer to corresponding unitary functions into Configuration of ADC hierarchical scope: ADC instance .
- The setting of these parameters by function LL_ADC_Init() is conditioned to ADC state: ADC instance must be disabled. This condition is applied to all ADC features, for efficiency and compatibility over all STM32 families. However, the different features can be set under different ADC state conditions (setting possible with ADC enabled without conversion on going, ADC enabled with conversion on going, ...) Each feature can be updated afterwards with a unitary function and potentially with ADC in a different state than disabled, refer to description of each function for setting conditioned to ADC state.
- After using this function, some other features must be configured using LL unitary functions. The minimum configuration remaining to be done is: Set ADC group regular or group injected sequencer: map channel on the selected sequencer rank. Refer to function LL_ADC_REG_SetSequencerRanks().Set ADC channel sampling time Refer to function LL_ADC_SetChannelSamplingTime();

LL_ADC_StructInit

Function name

void LL_ADC_StructInit (LL_ADC_InitTypeDef * ADC_InitStruct)

Function description

Set each LL_ADC_InitTypeDef field to default value.

Parameters

- **ADC_InitStruct:** Pointer to a LL_ADC_InitTypeDef structure whose fields will be set to default values.

Return values

- **None:**

LL_ADC_REG_Init

Function name

ErrorStatus LL_ADC_REG_Init (ADC_TypeDef * ADCx, LL_ADC_REG_InitTypeDef * ADC_REG_InitStruct)

Function description

Initialize some features of ADC group regular.

Parameters

- **ADCx:** ADC instance
- **ADC_REG_InitStruct:** Pointer to a LL_ADC_REG_InitTypeDef structure

Return values

- **An:** ErrorStatus enumeration value:
 - SUCCESS: ADC registers are initialized
 - ERROR: ADC registers are not initialized

Notes

- These parameters have an impact on ADC scope: ADC group regular. Refer to corresponding unitary functions into Configuration of ADC hierarchical scope: group regular (functions with prefix "REG").
- The setting of these parameters by function LL_ADC_Init() is conditioned to ADC state: ADC instance must be disabled. This condition is applied to all ADC features, for efficiency and compatibility over all STM32 families. However, the different features can be set under different ADC state conditions (setting possible with ADC enabled without conversion on going, ADC enabled with conversion on going, ...) Each feature can be updated afterwards with a unitary function and potentially with ADC in a different state than disabled, refer to description of each function for setting conditioned to ADC state.
- After using this function, other features must be configured using LL unitary functions. The minimum configuration remaining to be done is: Set ADC group regular or group injected sequencer: map channel on the selected sequencer rank. Refer to function LL_ADC_REG_SetSequencerRanks(). Set ADC channel sampling time Refer to function LL_ADC_SetChannelSamplingTime();

LL_ADC_REG_StructInit

Function name

void LL_ADC_REG_StructInit (LL_ADC_REG_InitTypeDef * ADC_REG_InitStruct)

Function description

Set each LL_ADC_REG_InitTypeDef field to default value.

Parameters

- **ADC_REG_InitStruct:** Pointer to a LL_ADC_REG_InitTypeDef structure whose fields will be set to default values.

Return values

- **None:**

LL_ADC_INJ_Init

Function name

ErrorStatus LL_ADC_INJ_Init (ADC_TypeDef * ADCx, LL_ADC_INJ_InitTypeDef * ADC_INJ_InitStruct)

Function description

Initialize some features of ADC group injected.

Parameters

- **ADCx**: ADC instance
- **ADC_INJ_InitStruct**: Pointer to a LL_ADC_INJ_InitTypeDef structure

Return values

- **An**: ErrorStatus enumeration value:
 - SUCCESS: ADC registers are initialized
 - ERROR: ADC registers are not initialized

Notes

- These parameters have an impact on ADC scope: ADC group injected. Refer to corresponding unitary functions into Configuration of ADC hierarchical scope: group regular (functions with prefix "INJ").
- The setting of these parameters by function LL_ADC_Init() is conditioned to ADC state: ADC instance must be disabled. This condition is applied to all ADC features, for efficiency and compatibility over all STM32 families. However, the different features can be set under different ADC state conditions (setting possible with ADC enabled without conversion on going, ADC enabled with conversion on going, ...) Each feature can be updated afterwards with a unitary function and potentially with ADC in a different state than disabled, refer to description of each function for setting conditioned to ADC state.
- After using this function, other features must be configured using LL unitary functions. The minimum configuration remaining to be done is: Set ADC group injected sequencer: map channel on the selected sequencer rank. Refer to function LL_ADC_INJ_SetSequencerRanks(). Set ADC channel sampling time. Refer to function LL_ADC_SetChannelSamplingTime();

LL_ADC_INJ_StructInit

Function name

void LL_ADC_INJ_StructInit (LL_ADC_INJ_InitTypeDef * ADC_INJ_InitStruct)

Function description

Set each LL_ADC_INJ_InitTypeDef field to default value.

Parameters

- **ADC_INJ_InitStruct**: Pointer to a LL_ADC_INJ_InitTypeDef structure whose fields will be set to default values.

Return values

- **None**:

73.3 ADC Firmware driver defines

The following section lists the various define and macros of the module.

73.3.1 ADC

ADC

Analog watchdog - Monitored channels

LL_ADC_AWD_DISABLE

ADC analog watchdog monitoring disabled

LL_ADC_AWD_ALL_CHANNELS_REG

ADC analog watchdog monitoring of all channels, converted by group regular only

LL_ADC_AWD_ALL_CHANNELS_INJ

ADC analog watchdog monitoring of all channels, converted by group injected only

LL_ADC_AWD_ALL_CHANNELS_REG_INJ

ADC analog watchdog monitoring of all channels, converted by either group regular or injected

LL_ADC_AWD_CHANNEL_0_REG

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN0, converted by group regular only

LL_ADC_AWD_CHANNEL_0_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN0, converted by group injected only

LL_ADC_AWD_CHANNEL_0_REG_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN0, converted by either group regular or injected

LL_ADC_AWD_CHANNEL_1_REG

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN1, converted by group regular only

LL_ADC_AWD_CHANNEL_1_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN1, converted by group injected only

LL_ADC_AWD_CHANNEL_1_REG_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN1, converted by either group regular or injected

LL_ADC_AWD_CHANNEL_2_REG

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN2, converted by group regular only

LL_ADC_AWD_CHANNEL_2_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN2, converted by group injected only

LL_ADC_AWD_CHANNEL_2_REG_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN2, converted by either group regular or injected

LL_ADC_AWD_CHANNEL_3_REG

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN3, converted by group regular only

LL_ADC_AWD_CHANNEL_3_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN3, converted by group injected only

LL_ADC_AWD_CHANNEL_3_REG_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN3, converted by either group regular or injected

LL_ADC_AWD_CHANNEL_4_REG

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN4, converted by group regular only

LL_ADC_AWD_CHANNEL_4_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN4, converted by group injected only

LL_ADC_AWD_CHANNEL_4_REG_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN4, converted by either group regular or injected

LL_ADC_AWD_CHANNEL_5_REG

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN5, converted by group regular only

LL_ADC_AWD_CHANNEL_5_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN5, converted by group injected only

LL_ADC_AWD_CHANNEL_5_REG_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN5, converted by either group regular or injected

LL_ADC_AWD_CHANNEL_6_REG

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN6, converted by group regular only

LL_ADC_AWD_CHANNEL_6_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN6, converted by group injected only

LL_ADC_AWD_CHANNEL_6_REG_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN6, converted by either group regular or injected

LL_ADC_AWD_CHANNEL_7_REG

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN7, converted by group regular only

LL_ADC_AWD_CHANNEL_7_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN7, converted by group injected only

LL_ADC_AWD_CHANNEL_7_REG_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN7, converted by either group regular or injected

LL_ADC_AWD_CHANNEL_8_REG

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN8, converted by group regular only

LL_ADC_AWD_CHANNEL_8_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN8, converted by group injected only

LL_ADC_AWD_CHANNEL_8_REG_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN8, converted by either group regular or injected

LL_ADC_AWD_CHANNEL_9_REG

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN9, converted by group regular only

LL_ADC_AWD_CHANNEL_9_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN9, converted by group injected only

LL_ADC_AWD_CHANNEL_9_REG_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN9, converted by either group regular or injected

LL_ADC_AWD_CHANNEL_10_REG

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN10, converted by group regular only

LL_ADC_AWD_CHANNEL_10_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN10, converted by group injected only

LL_ADC_AWD_CHANNEL_10_REG_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN10, converted by either group regular or injected

LL_ADC_AWD_CHANNEL_11_REG

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN11, converted by group regular only

LL_ADC_AWD_CHANNEL_11_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN11, converted by group injected only

LL_ADC_AWD_CHANNEL_11_REG_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN11, converted by either group regular or injected

LL_ADC_AWD_CHANNEL_12_REG

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN12, converted by group regular only

LL_ADC_AWD_CHANNEL_12_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN12, converted by group injected only

LL_ADC_AWD_CHANNEL_12_REG_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN12, converted by either group regular or injected

LL_ADC_AWD_CHANNEL_13_REG

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN13, converted by group regular only

LL_ADC_AWD_CHANNEL_13_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN13, converted by group injected only

LL_ADC_AWD_CHANNEL_13_REG_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN13, converted by either group regular or injected

LL_ADC_AWD_CHANNEL_14_REG

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN14, converted by group regular only

LL_ADC_AWD_CHANNEL_14_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN14, converted by group injected only

LL_ADC_AWD_CHANNEL_14_REG_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN14, converted by either group regular or injected

LL_ADC_AWD_CHANNEL_15_REG

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN15, converted by group regular only

LL_ADC_AWD_CHANNEL_15_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN15, converted by group injected only

LL_ADC_AWD_CHANNEL_15_REG_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN15, converted by either group regular or injected

LL_ADC_AWD_CHANNEL_16_REG

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN16, converted by group regular only

LL_ADC_AWD_CHANNEL_16_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN16, converted by group injected only

LL_ADC_AWD_CHANNEL_16_REG_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN16, converted by either group regular or injected

LL_ADC_AWD_CHANNEL_17_REG

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN17, converted by group regular only

LL_ADC_AWD_CHANNEL_17_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN17, converted by group injected only

LL_ADC_AWD_CHANNEL_17_REG_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN17, converted by either group regular or injected

LL_ADC_AWD_CHANNEL_18_REG

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN18, converted by group regular only

LL_ADC_AWD_CHANNEL_18_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN18, converted by group injected only

LL_ADC_AWD_CHANNEL_18_REG_INJ

ADC analog watchdog monitoring of ADC external channel (channel connected to GPIO pin) ADCx_IN18, converted by either group regular or injected

LL_ADC_AWD_CH_VREFINT_REG

ADC analog watchdog monitoring of ADC internal channel connected to VrefInt: Internal voltage reference, converted by group regular only

LL_ADC_AWD_CH_VREFINT_INJ

ADC analog watchdog monitoring of ADC internal channel connected to VrefInt: Internal voltage reference, converted by group injected only

LL_ADC_AWD_CH_VREFINT_REG_INJ

ADC analog watchdog monitoring of ADC internal channel connected to VrefInt: Internal voltage reference, converted by either group regular or injected

LL_ADC_AWD_CH_VBAT_REG

ADC analog watchdog monitoring of ADC internal channel connected to Vbat/3: Vbat voltage through a divider ladder of factor 1/3 to have Vbat always below Vdda, converted by group regular only

LL_ADC_AWD_CH_VBAT_INJ

ADC analog watchdog monitoring of ADC internal channel connected to Vbat/3: Vbat voltage through a divider ladder of factor 1/3 to have Vbat always below Vdda, converted by group injected only

LL_ADC_AWD_CH_VBAT_REG_INJ

ADC analog watchdog monitoring of ADC internal channel connected to Vbat/3: Vbat voltage through a divider ladder of factor 1/3 to have Vbat always below Vdda

LL_ADC_AWD_CH_TEMPSENSOR_REG

ADC analog watchdog monitoring of ADC internal channel connected to Temperature sensor, converted by group regular only. This internal channel is shared between temperature sensor and Vbat, only 1 measurement path must be enabled.

LL_ADC_AWD_CH_TEMPSENSOR_INJ

ADC analog watchdog monitoring of ADC internal channel connected to Temperature sensor, converted by group injected only. This internal channel is shared between temperature sensor and Vbat, only 1 measurement path must be enabled.

LL_ADC_AWD_CH_TEMPSENSOR_REG_INJ

ADC analog watchdog monitoring of ADC internal channel connected to Temperature sensor, converted by either group regular or injected. This internal channel is shared between temperature sensor and Vbat, only 1 measurement path must be enabled.

Analog watchdog - Analog watchdog number

LL_ADC_AWD1

ADC analog watchdog number 1

Analog watchdog - Thresholds

LL_ADC_AWD_THRESHOLD_HIGH

ADC analog watchdog threshold high

LL_ADC_AWD_THRESHOLD_LOW

ADC analog watchdog threshold low

ADC instance - Channel number

LL_ADC_CHANNEL_0

ADC external channel (channel connected to GPIO pin) ADCx_IN0

LL_ADC_CHANNEL_1

ADC external channel (channel connected to GPIO pin) ADCx_IN1

LL_ADC_CHANNEL_2

ADC external channel (channel connected to GPIO pin) ADCx_IN2

LL_ADC_CHANNEL_3

ADC external channel (channel connected to GPIO pin) ADCx_IN3

LL_ADC_CHANNEL_4

ADC external channel (channel connected to GPIO pin) ADCx_IN4

LL_ADC_CHANNEL_5

ADC external channel (channel connected to GPIO pin) ADCx_IN5

LL_ADC_CHANNEL_6

ADC external channel (channel connected to GPIO pin) ADCx_IN6

LL_ADC_CHANNEL_7

ADC external channel (channel connected to GPIO pin) ADCx_IN7

LL_ADC_CHANNEL_8

ADC external channel (channel connected to GPIO pin) ADCx_IN8

LL_ADC_CHANNEL_9

ADC external channel (channel connected to GPIO pin) ADCx_IN9

LL_ADC_CHANNEL_10

ADC external channel (channel connected to GPIO pin) ADCx_IN10

LL_ADC_CHANNEL_11

ADC external channel (channel connected to GPIO pin) ADCx_IN11

LL_ADC_CHANNEL_12

ADC external channel (channel connected to GPIO pin) ADCx_IN12

LL_ADC_CHANNEL_13

ADC external channel (channel connected to GPIO pin) ADCx_IN13

LL_ADC_CHANNEL_14

ADC external channel (channel connected to GPIO pin) ADCx_IN14

LL_ADC_CHANNEL_15

ADC external channel (channel connected to GPIO pin) ADCx_IN15

LL_ADC_CHANNEL_16

ADC external channel (channel connected to GPIO pin) ADCx_IN16

LL_ADC_CHANNEL_17

ADC external channel (channel connected to GPIO pin) ADCx_IN17

LL_ADC_CHANNEL_18

ADC external channel (channel connected to GPIO pin) ADCx_IN18

LL_ADC_CHANNEL_VREFINT

ADC internal channel connected to VrefInt: Internal voltage reference. On STM32F4, ADC channel available only on ADC instance: ADC1.

LL_ADC_CHANNEL_VBAT

ADC internal channel connected to Vbat/3: Vbat voltage through a divider ladder of factor 1/3 to have Vbat always below Vdda. On STM32F4, ADC channel available only on ADC instance: ADC1.

LL_ADC_CHANNEL_TEMPSENSOR

ADC internal channel connected to Temperature sensor. On STM32F4, ADC channel available only on ADC instance: ADC1. This internal channel is shared between temperature sensor and Vbat, only 1 measurement path must be enabled.

Channel - Sampling time

LL_ADC_SAMPLINGTIME_3CYCLES

Sampling time 3 ADC clock cycles

LL_ADC_SAMPLINGTIME_15CYCLES

Sampling time 15 ADC clock cycles

LL_ADC_SAMPLINGTIME_28CYCLES

Sampling time 28 ADC clock cycles

LL_ADC_SAMPLINGTIME_56CYCLES

Sampling time 56 ADC clock cycles

LL_ADC_SAMPLINGTIME_84CYCLES

Sampling time 84 ADC clock cycles

LL_ADC_SAMPLINGTIME_112CYCLES

Sampling time 112 ADC clock cycles

LL_ADC_SAMPLINGTIME_144CYCLES

Sampling time 144 ADC clock cycles

LL_ADC_SAMPLINGTIME_480CYCLES

Sampling time 480 ADC clock cycles

ADC common - Clock source

LL_ADC_CLOCK_SYNC_PCLK_DIV2

ADC synchronous clock derived from AHB clock with prescaler division by 2

LL_ADC_CLOCK_SYNC_PCLK_DIV4

ADC synchronous clock derived from AHB clock with prescaler division by 4

LL_ADC_CLOCK_SYNC_PCLK_DIV6

ADC synchronous clock derived from AHB clock with prescaler division by 6

LL_ADC_CLOCK_SYNC_PCLK_DIV8

ADC synchronous clock derived from AHB clock with prescaler division by 8

ADC common - Measurement path to internal channels

LL_ADC_PATH_INTERNAL_NONE

ADC measurement paths all disabled

LL_ADC_PATH_INTERNAL_VREFINT

ADC measurement path to internal channel VrefInt

LL_ADC_PATH_INTERNAL_TEMPSENSOR

ADC measurement path to internal channel temperature sensor

LL_ADC_PATH_INTERNAL_VBAT

ADC measurement path to internal channel Vbat

ADC instance - Data alignment**LL_ADC_DATA_ALIGN_RIGHT**

ADC conversion data alignment: right aligned (alignment on data register LSB bit 0)

LL_ADC_DATA_ALIGN_LEFT

ADC conversion data alignment: left aligned (alignment on data register MSB bit 15)

ADC flags**LL_ADC_FLAG_STRT**

ADC flag ADC group regular conversion start

LL_ADC_FLAG_EOCS

ADC flag ADC group regular end of unitary conversion or sequence conversions (to configure flag of end of conversion, use function

LL_ADC_FLAG_OVR

ADC flag ADC group regular overrun

LL_ADC_FLAG_JSTRT

ADC flag ADC group injected conversion start

LL_ADC_FLAG_JEOS

ADC flag ADC group injected end of sequence conversions (Note: on this STM32 series, there is no flag ADC group injected end of unitary conversion. Flag noted as "JEOC" is corresponding to flag "JEOS" in other STM32 families)

LL_ADC_FLAG_AWD1

ADC flag ADC analog watchdog 1

LL_ADC_FLAG_EOCS_MST

ADC flag ADC multimode master group regular end of unitary conversion or sequence conversions (to configure flag of end of conversion, use function

LL_ADC_FLAG_EOCS_SLV1

ADC flag ADC multimode slave 1 group regular end of unitary conversion or sequence conversions (to configure flag of end of conversion, use function

LL_ADC_FLAG_EOCS_SLV2

ADC flag ADC multimode slave 2 group regular end of unitary conversion or sequence conversions (to configure flag of end of conversion, use function

LL_ADC_FLAG_OVR_MST

ADC flag ADC multimode master group regular overrun

LL_ADC_FLAG_OVR_SLV1

ADC flag ADC multimode slave 1 group regular overrun

LL_ADC_FLAG_OVR_SLV2

ADC flag ADC multimode slave 2 group regular overrun

LL_ADC_FLAG_JEOS_MST

ADC flag ADC multimode master group injected end of sequence conversions (Note: on this STM32 series, there is no flag ADC group injected end of unitary conversion. Flag noted as "JEOC" is corresponding to flag "JEOS" in other STM32 families)

LL_ADC_FLAG_JEOS_SLV1

ADC flag ADC multimode slave 1 group injected end of sequence conversions (Note: on this STM32 series, there is no flag ADC group injected end of unitary conversion. Flag noted as "JEOC" is corresponding to flag "JEOS" in other STM32 families)

LL_ADC_FLAG_JEOS_SLV2

ADC flag ADC multimode slave 2 group injected end of sequence conversions (Note: on this STM32 series, there is no flag ADC group injected end of unitary conversion. Flag noted as "JEOC" is corresponding to flag "JEOS" in other STM32 families)

LL_ADC_FLAG_AWD1_MST

ADC flag ADC multimode master analog watchdog 1 of the ADC master

LL_ADC_FLAG_AWD1_SLV1

ADC flag ADC multimode slave 1 analog watchdog 1

LL_ADC_FLAG_AWD1_SLV2

ADC flag ADC multimode slave 2 analog watchdog 1

ADC instance - Groups**LL_ADC_GROUP_REGULAR**

ADC group regular (available on all STM32 devices)

LL_ADC_GROUP_INJECTED

ADC group injected (not available on all STM32 devices)

LL_ADC_GROUP_REGULAR_INJECTED

ADC both groups regular and injected

Definitions of ADC hardware constraints delays**LL_ADC_DELAY_VREFINT_STAB_US**

Delay for internal voltage reference stabilization time

LL_ADC_DELAY_TEMPSENSOR_STAB_US

Delay for internal voltage reference stabilization time

ADC group injected - Sequencer discontinuous mode**LL_ADC_INJ_SEQ_DISCONT_DISABLE**

ADC group injected sequencer discontinuous mode disable

LL_ADC_INJ_SEQ_DISCONT_1RANK

ADC group injected sequencer discontinuous mode enable with sequence interruption every rank

ADC group injected - Sequencer ranks**LL_ADC_INJ_RANK_1**

ADC group injected sequencer rank 1

LL_ADC_INJ_RANK_2

ADC group injected sequencer rank 2

LL_ADC_INJ_RANK_3

ADC group injected sequencer rank 3

LL_ADC_INJ_RANK_4

ADC group injected sequencer rank 4

ADC group injected - Sequencer scan length**LL_ADC_INJ_SEQ_SCAN_DISABLE**

ADC group injected sequencer disable (equivalent to sequencer of 1 rank: ADC conversion on only 1 channel)

LL_ADC_INJ_SEQ_SCAN_ENABLE_2RANKS

ADC group injected sequencer enable with 2 ranks in the sequence

LL_ADC_INJ_SEQ_SCAN_ENABLE_3RANKS

ADC group injected sequencer enable with 3 ranks in the sequence

LL_ADC_INJ_SEQ_SCAN_ENABLE_4RANKS

ADC group injected sequencer enable with 4 ranks in the sequence

ADC group injected - Trigger edge**LL_ADC_INJ_TRIG_EXT_RISING**

ADC group injected conversion trigger polarity set to rising edge

LL_ADC_INJ_TRIG_EXT_FALLING

ADC group injected conversion trigger polarity set to falling edge

LL_ADC_INJ_TRIG_EXT_RISINGFALLING

ADC group injected conversion trigger polarity set to both rising and falling edges

ADC group injected - Trigger source**LL_ADC_INJ_TRIG_SOFTWARE**

ADC group injected conversion trigger internal: SW start.

LL_ADC_INJ_TRIG_EXT_TIM1_CH4

ADC group injected conversion trigger from external IP: TIM1 channel 4 event (capture compare: input capture or output capture). Trigger edge set to rising edge (default setting).

LL_ADC_INJ_TRIG_EXT_TIM1_TRGO

ADC group injected conversion trigger from external IP: TIM1 TRGO. Trigger edge set to rising edge (default setting).

LL_ADC_INJ_TRIG_EXT_TIM2_CH1

ADC group injected conversion trigger from external IP: TIM2 channel 1 event (capture compare: input capture or output capture). Trigger edge set to rising edge (default setting).

LL_ADC_INJ_TRIG_EXT_TIM2_TRGO

ADC group injected conversion trigger from external IP: TIM2 TRGO. Trigger edge set to rising edge (default setting).

LL_ADC_INJ_TRIG_EXT_TIM3_CH2

ADC group injected conversion trigger from external IP: TIM3 channel 2 event (capture compare: input capture or output capture). Trigger edge set to rising edge (default setting).

LL_ADC_INJ_TRIG_EXT_TIM3_CH4

ADC group injected conversion trigger from external IP: TIM3 channel 4 event (capture compare: input capture or output capture). Trigger edge set to rising edge (default setting).

LL_ADC_INJ_TRIG_EXT_TIM4_CH1

ADC group injected conversion trigger from external IP: TIM4 channel 1 event (capture compare: input capture or output capture). Trigger edge set to rising edge (default setting).

LL_ADC_INJ_TRIG_EXT_TIM4_CH2

ADC group injected conversion trigger from external IP: TIM4 channel 2 event (capture compare: input capture or output capture). Trigger edge set to rising edge (default setting).

LL_ADC_INJ_TRIG_EXT_TIM4_CH3

ADC group injected conversion trigger from external IP: TIM4 channel 3 event (capture compare: input capture or output capture). Trigger edge set to rising edge (default setting).

LL_ADC_INJ_TRIG_EXT_TIM4_TRGO

ADC group injected conversion trigger from external IP: TIM4 TRGO. Trigger edge set to rising edge (default setting).

LL_ADC_INJ_TRIG_EXT_TIM5_CH4

ADC group injected conversion trigger from external IP: TIM5 channel 4 event (capture compare: input capture or output capture). Trigger edge set to rising edge (default setting).

LL_ADC_INJ_TRIG_EXT_TIM5_TRGO

ADC group injected conversion trigger from external IP: TIM5 TRGO. Trigger edge set to rising edge (default setting).

LL_ADC_INJ_TRIG_EXT_TIM8_CH2

ADC group injected conversion trigger from external IP: TIM8 channel 2 event (capture compare: input capture or output capture). Trigger edge set to rising edge (default setting).

LL_ADC_INJ_TRIG_EXT_TIM8_CH3

ADC group injected conversion trigger from external IP: TIM8 channel 3 event (capture compare: input capture or output capture). Trigger edge set to rising edge (default setting).

LL_ADC_INJ_TRIG_EXT_TIM8_CH4

ADC group injected conversion trigger from external IP: TIM8 channel 4 event (capture compare: input capture or output capture). Trigger edge set to rising edge (default setting).

LL_ADC_INJ_TRIG_EXT_EXTI_LINE15

ADC group injected conversion trigger from external IP: external interrupt line 15. Trigger edge set to rising edge (default setting).

ADC group injected - Automatic trigger mode

LL_ADC_INJ_TRIG_INDEPENDENT

ADC group injected conversion trigger independent. Setting mandatory if ADC group injected injected trigger source is set to an external trigger.

LL_ADC_INJ_TRIG_FROM_GRP_REGULAR

ADC group injected conversion trigger from ADC group regular. Setting compliant only with group injected trigger source set to SW start, without any further action on ADC group injected conversion start or stop: in this case, ADC group injected is controlled only from ADC group regular.

ADC interruptions for configuration (interruption enable or disable)

LL_ADC_IT_EOCS

ADC interruption ADC group regular end of unitary conversion or sequence conversions (to configure flag of end of conversion, use function

LL_ADC_IT_OVR

ADC interruption ADC group regular overrun

LL_ADC_IT_JEOS

ADC interruption ADC group injected end of sequence conversions (Note: on this STM32 series, there is no flag ADC group injected end of unitary conversion. Flag noted as "JEOC" is corresponding to flag "JEOS" in other STM32 families)

LL_ADC_IT_AWD1

ADC interruption ADC analog watchdog 1

Multimode - DMA transfer

LL_ADC_MULTI_REG_DMA_EACH_ADC

ADC multimode group regular conversions are transferred by DMA: each ADC uses its own DMA channel, with its individual DMA transfer settings

LL_ADC_MULTI_REG_DMA_LIMIT_1

ADC multimode group regular conversions are transferred by DMA, one DMA channel for all ADC instances (DMA of ADC master), in limited mode (one shot mode): DMA transfer requests are stopped when number of DMA data transfers (number of ADC conversions) is reached. This ADC mode is intended to be used with DMA mode non-circular. Setting of DMA mode 1: 2 or 3 (dual or triple mode) half-words one by one, ADC1 then ADC2 then ADC3.

LL_ADC_MULTI_REG_DMA_LIMIT_2

ADC multimode group regular conversions are transferred by DMA, one DMA channel for all ADC instances (DMA of ADC master), in limited mode (one shot mode): DMA transfer requests are stopped when number of DMA data transfers (number of ADC conversions) is reached. This ADC mode is intended to be used with DMA mode non-circular. Setting of DMA mode 2: 2 or 3 (dual or triple mode) half-words one by one, ADC2&1 then ADC1&3 then ADC3&2.

LL_ADC_MULTI_REG_DMA_LIMIT_3

ADC multimode group regular conversions are transferred by DMA, one DMA channel for all ADC instances (DMA of ADC master), in limited mode (one shot mode): DMA transfer requests are stopped when number of DMA data transfers (number of ADC conversions) is reached. This ADC mode is intended to be used with DMA mode non-circular. Setting of DMA mode 3: 2 or 3 (dual or triple mode) bytes one by one, ADC2&1 then ADC1&3 then ADC3&2.

LL_ADC_MULTI_REG_DMA_UNLMT_1

ADC multimode group regular conversions are transferred by DMA, one DMA channel for all ADC instances (DMA of ADC master), in unlimited mode: DMA transfer requests are unlimited, whatever number of DMA data transferred (number of ADC conversions) is reached. This ADC mode is intended to be used with DMA mode non-circular. Setting of DMA mode 1: 2 or 3 (dual or triple mode) half-words one by one, ADC1 then ADC2 then ADC3.

LL_ADC_MULTI_REG_DMA_UNLMT_2

ADC multimode group regular conversions are transferred by DMA, one DMA channel for all ADC instances (DMA of ADC master), in unlimited mode: DMA transfer requests are unlimited, whatever number of DMA data transferred (number of ADC conversions) is reached. This ADC mode is intended to be used with DMA mode non-circular. Setting of DMA mode 2: 2 or 3 (dual or triple mode) half-words by pairs, ADC2&1 then ADC1&3 then ADC3&2.

LL_ADC_MULTI_REG_DMA_UNLMT_3

ADC multimode group regular conversions are transferred by DMA, one DMA channel for all ADC instances (DMA of ADC master), in unlimited mode: DMA transfer requests are unlimited, whatever number of DMA data transferred (number of ADC conversions) is reached. This ADC mode is intended to be used with DMA mode non-circular. Setting of DMA mode 3: 2 or 3 (dual or triple mode) bytes one by one, ADC2&1 then ADC1&3 then ADC3&2.

Multimode - ADC master or slave

LL_ADC_MULTI_MASTER

In multimode, selection among several ADC instances: ADC master

LL_ADC_MULTI_SLAVE

In multimode, selection among several ADC instances: ADC slave

LL_ADC_MULTI_MASTER_SLAVE

In multimode, selection among several ADC instances: both ADC master and ADC slave

Multimode - Mode

LL_ADC_MULTI_INDEPENDENT

ADC dual mode disabled (ADC independent mode)

LL_ADC_MULTI_DUAL_REG_SIMULT

ADC dual mode enabled: group regular simultaneous

LL_ADC_MULTI_DUAL_REG_INTERL

ADC dual mode enabled: Combined group regular interleaved

LL_ADC_MULTI_DUAL_INJ_SIMULT

ADC dual mode enabled: group injected simultaneous

LL_ADC_MULTI_DUAL_INJ_ALTERN

ADC dual mode enabled: group injected alternate trigger. Works only with external triggers (not internal SW start)

LL_ADC_MULTI_DUAL_REG_SIM_INJ_SIM

ADC dual mode enabled: Combined group regular simultaneous + group injected simultaneous

LL_ADC_MULTI_DUAL_REG_SIM_INJ_ALT

ADC dual mode enabled: Combined group regular simultaneous + group injected alternate trigger

LL_ADC_MULTI_DUAL_REG_INT_INJ_SIM

ADC dual mode enabled: Combined group regular interleaved + group injected simultaneous

LL_ADC_MULTI_TRIPLE_REG_SIM_INJ_SIM

ADC triple mode enabled: Combined group regular simultaneous + group injected simultaneous

LL_ADC_MULTI_TRIPLE_REG_SIM_INJ_ALT

ADC triple mode enabled: Combined group regular simultaneous + group injected alternate trigger

LL_ADC_MULTI_TRIPLE_INJ_SIMULT

ADC triple mode enabled: group injected simultaneous

LL_ADC_MULTI_TRIPLE_REG_SIMULT

ADC triple mode enabled: group regular simultaneous

LL_ADC_MULTI_TRIPLE_REG_INTERL

ADC triple mode enabled: Combined group regular interleaved

LL_ADC_MULTI_TRIPLE_INJ_ALTERN

ADC triple mode enabled: group injected alternate trigger. Works only with external triggers (not internal SW start)

Multimode - Delay between two sampling phases

LL_ADC_MULTI_TWOSMP_DELAY_5CYCLES

ADC multimode delay between two sampling phases: 5 ADC clock cycles

LL_ADC_MULTI_TWOSMP_DELAY_6CYCLES

ADC multimode delay between two sampling phases: 6 ADC clock cycles

LL_ADC_MULTI_TWOSMP_DELAY_7CYCLES

ADC multimode delay between two sampling phases: 7 ADC clock cycles

LL_ADC_MULTI_TWOSMP_DELAY_8CYCLES

ADC multimode delay between two sampling phases: 8 ADC clock cycles

LL_ADC_MULTI_TWOSMP_DELAY_9CYCLES

ADC multimode delay between two sampling phases: 9 ADC clock cycles

LL_ADC_MULTI_TWOSMP_DELAY_10CYCLES

ADC multimode delay between two sampling phases: 10 ADC clock cycles

LL_ADC_MULTI_TWOSMP_DELAY_11CYCLES

ADC multimode delay between two sampling phases: 11 ADC clock cycles

LL_ADC_MULTI_TWOSMP_DELAY_12CYCLES

ADC multimode delay between two sampling phases: 12 ADC clock cycles

LL_ADC_MULTI_TWOSMP_DELAY_13CYCLES

ADC multimode delay between two sampling phases: 13 ADC clock cycles

LL_ADC_MULTI_TWOSMP_DELAY_14CYCLES

ADC multimode delay between two sampling phases: 14 ADC clock cycles

LL_ADC_MULTI_TWOSMP_DELAY_15CYCLES

ADC multimode delay between two sampling phases: 15 ADC clock cycles

LL_ADC_MULTI_TWOSMP_DELAY_16CYCLES

ADC multimode delay between two sampling phases: 16 ADC clock cycles

LL_ADC_MULTI_TWOSMP_DELAY_17CYCLES

ADC multimode delay between two sampling phases: 17 ADC clock cycles

LL_ADC_MULTI_TWOSMP_DELAY_18CYCLES

ADC multimode delay between two sampling phases: 18 ADC clock cycles

LL_ADC_MULTI_TWOSMP_DELAY_19CYCLES

ADC multimode delay between two sampling phases: 19 ADC clock cycles

LL_ADC_MULTI_TWOSMP_DELAY_20CYCLES

ADC multimode delay between two sampling phases: 20 ADC clock cycles

ADC registers compliant with specific purpose**LL_ADC_DMA_REG_REGULAR_DATA****LL_ADC_DMA_REG_REGULAR_DATA_MULTI*****ADC group regular - Continuous mode*****LL_ADC_REG_CONV_SINGLE**

ADC conversions are performed in single mode: one conversion per trigger

LL_ADC_REG_CONV_CONTINUOUS

ADC conversions are performed in continuous mode: after the first trigger, following conversions launched successively automatically

ADC group regular - DMA transfer of ADC conversion data**LL_ADC_REG_DMA_TRANSFER_NONE**

ADC conversions are not transferred by DMA

LL_ADC_REG_DMA_TRANSFER_LIMITED

ADC conversion data are transferred by DMA, in limited mode (one shot mode): DMA transfer requests are stopped when number of DMA data transfers (number of ADC conversions) is reached. This ADC mode is intended to be used with DMA mode non-circular.

LL_ADC_REG_DMA_TRANSFER_UNLIMITED

ADC conversion data are transferred by DMA, in unlimited mode: DMA transfer requests are unlimited, whatever number of DMA data transferred (number of ADC conversions). This ADC mode is intended to be used with DMA mode circular.

ADC group regular - Flag EOC selection (unitary or sequence conversions)**LL_ADC_REG_FLAG_EOC_SEQUENCE_CONV**

ADC flag EOC (end of unitary conversion) selected

LL_ADC_REG_FLAG_EOC_UNITARY_CONV

ADC flag EOS (end of sequence conversions) selected

ADC group regular - Sequencer discontinuous mode**LL_ADC_REG_SEQ_DISCONT_DISABLE**

ADC group regular sequencer discontinuous mode disable

LL_ADC_REG_SEQ_DISCONT_1RANK

ADC group regular sequencer discontinuous mode enable with sequence interruption every rank

LL_ADC_REG_SEQ_DISCONT_2RANKS

ADC group regular sequencer discontinuous mode enabled with sequence interruption every 2 ranks

LL_ADC_REG_SEQ_DISCONT_3RANKS

ADC group regular sequencer discontinuous mode enable with sequence interruption every 3 ranks

LL_ADC_REG_SEQ_DISCONT_4RANKS

ADC group regular sequencer discontinuous mode enable with sequence interruption every 4 ranks

LL_ADC_REG_SEQ_DISCONT_5RANKS

ADC group regular sequencer discontinuous mode enable with sequence interruption every 5 ranks

LL_ADC_REG_SEQ_DISCONT_6RANKS

ADC group regular sequencer discontinuous mode enable with sequence interruption every 6 ranks

LL_ADC_REG_SEQ_DISCONT_7RANKS

ADC group regular sequencer discontinuous mode enable with sequence interruption every 7 ranks

LL_ADC_REG_SEQ_DISCONT_8RANKS

ADC group regular sequencer discontinuous mode enable with sequence interruption every 8 ranks

ADC group regular - Sequencer ranks

LL_ADC_REG_RANK_1

ADC group regular sequencer rank 1

LL_ADC_REG_RANK_2

ADC group regular sequencer rank 2

LL_ADC_REG_RANK_3

ADC group regular sequencer rank 3

LL_ADC_REG_RANK_4

ADC group regular sequencer rank 4

LL_ADC_REG_RANK_5

ADC group regular sequencer rank 5

LL_ADC_REG_RANK_6

ADC group regular sequencer rank 6

LL_ADC_REG_RANK_7

ADC group regular sequencer rank 7

LL_ADC_REG_RANK_8

ADC group regular sequencer rank 8

LL_ADC_REG_RANK_9

ADC group regular sequencer rank 9

LL_ADC_REG_RANK_10

ADC group regular sequencer rank 10

LL_ADC_REG_RANK_11

ADC group regular sequencer rank 11

LL_ADC_REG_RANK_12

ADC group regular sequencer rank 12

LL_ADC_REG_RANK_13

ADC group regular sequencer rank 13

LL_ADC_REG_RANK_14

ADC group regular sequencer rank 14

LL_ADC_REG_RANK_15

ADC group regular sequencer rank 15

LL_ADC_REG_RANK_16

ADC group regular sequencer rank 16

ADC group regular - Sequencer scan length**LL_ADC_REG_SEQ_SCAN_DISABLE**

ADC group regular sequencer disable (equivalent to sequencer of 1 rank: ADC conversion on only 1 channel)

LL_ADC_REG_SEQ_SCAN_ENABLE_2RANKS

ADC group regular sequencer enable with 2 ranks in the sequence

LL_ADC_REG_SEQ_SCAN_ENABLE_3RANKS

ADC group regular sequencer enable with 3 ranks in the sequence

LL_ADC_REG_SEQ_SCAN_ENABLE_4RANKS

ADC group regular sequencer enable with 4 ranks in the sequence

LL_ADC_REG_SEQ_SCAN_ENABLE_5RANKS

ADC group regular sequencer enable with 5 ranks in the sequence

LL_ADC_REG_SEQ_SCAN_ENABLE_6RANKS

ADC group regular sequencer enable with 6 ranks in the sequence

LL_ADC_REG_SEQ_SCAN_ENABLE_7RANKS

ADC group regular sequencer enable with 7 ranks in the sequence

LL_ADC_REG_SEQ_SCAN_ENABLE_8RANKS

ADC group regular sequencer enable with 8 ranks in the sequence

LL_ADC_REG_SEQ_SCAN_ENABLE_9RANKS

ADC group regular sequencer enable with 9 ranks in the sequence

LL_ADC_REG_SEQ_SCAN_ENABLE_10RANKS

ADC group regular sequencer enable with 10 ranks in the sequence

LL_ADC_REG_SEQ_SCAN_ENABLE_11RANKS

ADC group regular sequencer enable with 11 ranks in the sequence

LL_ADC_REG_SEQ_SCAN_ENABLE_12RANKS

ADC group regular sequencer enable with 12 ranks in the sequence

LL_ADC_REG_SEQ_SCAN_ENABLE_13RANKS

ADC group regular sequencer enable with 13 ranks in the sequence

LL_ADC_REG_SEQ_SCAN_ENABLE_14RANKS

ADC group regular sequencer enable with 14 ranks in the sequence

LL_ADC_REG_SEQ_SCAN_ENABLE_15RANKS

ADC group regular sequencer enable with 15 ranks in the sequence

LL_ADC_REG_SEQ_SCAN_ENABLE_16RANKS

ADC group regular sequencer enable with 16 ranks in the sequence

ADC group regular - Trigger edge**LL_ADC_REG_TRIG_EXT_RISING**

ADC group regular conversion trigger polarity set to rising edge

LL_ADC_REG_TRIG_EXT_FALLING

ADC group regular conversion trigger polarity set to falling edge

LL_ADC_REG_TRIG_EXT_RISINGFALLING

ADC group regular conversion trigger polarity set to both rising and falling edges

ADC group regular - Trigger source**LL_ADC_REG_TRIG_SOFTWARE**

ADC group regular conversion trigger internal: SW start.

LL_ADC_REG_TRIG_EXT_TIM1_CH1

ADC group regular conversion trigger from external IP: TIM1 channel 1 event (capture compare: input capture or output capture). Trigger edge set to rising edge (default setting).

LL_ADC_REG_TRIG_EXT_TIM1_CH2

ADC group regular conversion trigger from external IP: TIM1 channel 2 event (capture compare: input capture or output capture). Trigger edge set to rising edge (default setting).

LL_ADC_REG_TRIG_EXT_TIM1_CH3

ADC group regular conversion trigger from external IP: TIM1 channel 3 event (capture compare: input capture or output capture). Trigger edge set to rising edge (default setting).

LL_ADC_REG_TRIG_EXT_TIM2_CH2

ADC group regular conversion trigger from external IP: TIM2 channel 2 event (capture compare: input capture or output capture). Trigger edge set to rising edge (default setting).

LL_ADC_REG_TRIG_EXT_TIM2_CH3

ADC group regular conversion trigger from external IP: TIM2 channel 3 event (capture compare: input capture or output capture). Trigger edge set to rising edge (default setting).

LL_ADC_REG_TRIG_EXT_TIM2_CH4

ADC group regular conversion trigger from external IP: TIM2 channel 4 event (capture compare: input capture or output capture). Trigger edge set to rising edge (default setting).

LL_ADC_REG_TRIG_EXT_TIM2_TRGO

ADC group regular conversion trigger from external IP: TIM2 TRGO. Trigger edge set to rising edge (default setting).

LL_ADC_REG_TRIG_EXT_TIM3_CH1

ADC group regular conversion trigger from external IP: TIM3 channel 1 event (capture compare: input capture or output capture). Trigger edge set to rising edge (default setting).

LL_ADC_REG_TRIG_EXT_TIM3_TRGO

ADC group regular conversion trigger from external IP: TIM3 TRGO. Trigger edge set to rising edge (default setting).

LL_ADC_REG_TRIG_EXT_TIM4_CH4

ADC group regular conversion trigger from external IP: TIM4 channel 4 event (capture compare: input capture or output capture). Trigger edge set to rising edge (default setting).

LL_ADC_REG_TRIG_EXT_TIM5_CH1

ADC group regular conversion trigger from external IP: TIM5 channel 1 event (capture compare: input capture or output capture). Trigger edge set to rising edge (default setting).

LL_ADC_REG_TRIG_EXT_TIM5_CH2

ADC group regular conversion trigger from external IP: TIM5 channel 2 event (capture compare: input capture or output capture). Trigger edge set to rising edge (default setting).

LL_ADC_REG_TRIG_EXT_TIM5_CH3

ADC group regular conversion trigger from external IP: TIM5 channel 3 event (capture compare: input capture or output capture). Trigger edge set to rising edge (default setting).

LL_ADC_REG_TRIG_EXT_TIM8_CH1

ADC group regular conversion trigger from external IP: TIM8 channel 1 event (capture compare: input capture or output capture). Trigger edge set to rising edge (default setting).

LL_ADC_REG_TRIG_EXT_TIM8_TRGO

ADC group regular conversion trigger from external IP: TIM8 TRGO. Trigger edge set to rising edge (default setting).

LL_ADC_REG_TRIG_EXT_EXTI_LINE11

ADC group regular conversion trigger from external IP: external interrupt line 11. Trigger edge set to rising edge (default setting).

ADC instance - Resolution**LL_ADC_RESOLUTION_12B**

ADC resolution 12 bits

LL_ADC_RESOLUTION_10B

ADC resolution 10 bits

LL_ADC_RESOLUTION_8B

ADC resolution 8 bits

LL_ADC_RESOLUTION_6B

ADC resolution 6 bits

ADC instance - Scan selection**LL_ADC_SEQ_SCAN_DISABLE**

ADC conversion is performed in unitary conversion mode (one channel converted, that defined in rank 1). Configuration of both groups regular and injected sequencers (sequence length, ...) is discarded: equivalent to length of 1 rank.

LL_ADC_SEQ_SCAN_ENABLE

ADC conversions are performed in sequence conversions mode, according to configuration of both groups regular and injected sequencers (sequence length, ...).

ADC helper macro

__LL_ADC_CHANNEL_TO_DECIMAL_NB

Description:

- Helper macro to get ADC channel number in decimal format from literals LL_ADC_CHANNEL_x.

Parameters:

- __CHANNEL__: This parameter can be one of the following values:
 - LL_ADC_CHANNEL_0
 - LL_ADC_CHANNEL_1
 - LL_ADC_CHANNEL_2
 - LL_ADC_CHANNEL_3
 - LL_ADC_CHANNEL_4
 - LL_ADC_CHANNEL_5
 - LL_ADC_CHANNEL_6
 - LL_ADC_CHANNEL_7
 - LL_ADC_CHANNEL_8
 - LL_ADC_CHANNEL_9
 - LL_ADC_CHANNEL_10
 - LL_ADC_CHANNEL_11
 - LL_ADC_CHANNEL_12
 - LL_ADC_CHANNEL_13
 - LL_ADC_CHANNEL_14
 - LL_ADC_CHANNEL_15
 - LL_ADC_CHANNEL_16
 - LL_ADC_CHANNEL_17
 - LL_ADC_CHANNEL_18
 - LL_ADC_CHANNEL_VREFINT (1)
 - LL_ADC_CHANNEL_TEMPSENSOR (1)(2)
 - LL_ADC_CHANNEL_VBAT (1)

Return value:

- Value: between Min_Data=0 and Max_Data=18

Notes:

- Example: __LL_ADC_CHANNEL_TO_DECIMAL_NB(LL_ADC_CHANNEL_4) will return decimal number "4". The input can be a value from functions where a channel number is returned, either defined with number or with bitfield (only one bit must be set).

__LL_ADC_DECIMAL_NB_TO_CHANNEL

Description:

- Helper macro to get ADC channel in literal format LL_ADC_CHANNEL_x from number in decimal format.

Parameters:

- __DECIMAL_NB__: Value between Min_Data=0 and Max_Data=18

Return value:

- Returned: value can be one of the following values:
 - LL_ADC_CHANNEL_0
 - LL_ADC_CHANNEL_1
 - LL_ADC_CHANNEL_2
 - LL_ADC_CHANNEL_3
 - LL_ADC_CHANNEL_4
 - LL_ADC_CHANNEL_5
 - LL_ADC_CHANNEL_6
 - LL_ADC_CHANNEL_7
 - LL_ADC_CHANNEL_8
 - LL_ADC_CHANNEL_9
 - LL_ADC_CHANNEL_10
 - LL_ADC_CHANNEL_11
 - LL_ADC_CHANNEL_12
 - LL_ADC_CHANNEL_13
 - LL_ADC_CHANNEL_14
 - LL_ADC_CHANNEL_15
 - LL_ADC_CHANNEL_16
 - LL_ADC_CHANNEL_17
 - LL_ADC_CHANNEL_18
 - LL_ADC_CHANNEL_VREFINT (1)
 - LL_ADC_CHANNEL_TEMPSENSOR (1)(2)
 - LL_ADC_CHANNEL_VBAT (1)

Notes:

- Example: __LL_ADC_DECIMAL_NB_TO_CHANNEL(4) will return a data equivalent to "LL_ADC_CHANNEL_4".

__LL_ADC_IS_CHANNEL_INTERNAL

Description:

- Helper macro to determine whether the selected channel corresponds to literal definitions of driver.

Parameters:

- **__CHANNEL__**: This parameter can be one of the following values:
 - LL_ADC_CHANNEL_0
 - LL_ADC_CHANNEL_1
 - LL_ADC_CHANNEL_2
 - LL_ADC_CHANNEL_3
 - LL_ADC_CHANNEL_4
 - LL_ADC_CHANNEL_5
 - LL_ADC_CHANNEL_6
 - LL_ADC_CHANNEL_7
 - LL_ADC_CHANNEL_8
 - LL_ADC_CHANNEL_9
 - LL_ADC_CHANNEL_10
 - LL_ADC_CHANNEL_11
 - LL_ADC_CHANNEL_12
 - LL_ADC_CHANNEL_13
 - LL_ADC_CHANNEL_14
 - LL_ADC_CHANNEL_15
 - LL_ADC_CHANNEL_16
 - LL_ADC_CHANNEL_17
 - LL_ADC_CHANNEL_18
 - LL_ADC_CHANNEL_VREFINT (1)
 - LL_ADC_CHANNEL_TEMPSENSOR (1)(2)
 - LL_ADC_CHANNEL_VBAT (1)

Return value:

- Value: "0" if the channel corresponds to a parameter definition of a ADC external channel (channel connected to a GPIO pin). Value "1" if the channel corresponds to a parameter definition of a ADC internal channel.

Notes:

- The different literal definitions of ADC channels are: ADC internal channel: LL_ADC_CHANNEL_VREFINT, LL_ADC_CHANNEL_TEMPSENSOR, ...ADC external channel (channel connected to a GPIO pin): LL_ADC_CHANNEL_1, LL_ADC_CHANNEL_2, ... The channel parameter must be a value defined from literal definition of a ADC internal channel (LL_ADC_CHANNEL_VREFINT, LL_ADC_CHANNEL_TEMPSENSOR, ...), ADC external channel (LL_ADC_CHANNEL_1, LL_ADC_CHANNEL_2, ...), must not be a value from functions where a channel number is returned from ADC registers, because internal and external channels share the same channel number in ADC registers. The differentiation is made only with parameters definitions of driver.

__LL_ADC_CHANNEL_INTERNAL_TO_EXTERNAL

Description:

- Helper macro to convert a channel defined from parameter definition of a ADC internal channel (LL_ADC_CHANNEL_VREFINT, LL_ADC_CHANNEL_TEMPSENSOR, ...), to its equivalent parameter definition of a ADC external channel (LL_ADC_CHANNEL_1, LL_ADC_CHANNEL_2, ...).

Parameters:

- __CHANNEL__: This parameter can be one of the following values:
 - LL_ADC_CHANNEL_0
 - LL_ADC_CHANNEL_1
 - LL_ADC_CHANNEL_2
 - LL_ADC_CHANNEL_3
 - LL_ADC_CHANNEL_4
 - LL_ADC_CHANNEL_5
 - LL_ADC_CHANNEL_6
 - LL_ADC_CHANNEL_7
 - LL_ADC_CHANNEL_8
 - LL_ADC_CHANNEL_9
 - LL_ADC_CHANNEL_10
 - LL_ADC_CHANNEL_11
 - LL_ADC_CHANNEL_12
 - LL_ADC_CHANNEL_13
 - LL_ADC_CHANNEL_14
 - LL_ADC_CHANNEL_15
 - LL_ADC_CHANNEL_16
 - LL_ADC_CHANNEL_17
 - LL_ADC_CHANNEL_18
 - LL_ADC_CHANNEL_VREFINT (1)
 - LL_ADC_CHANNEL_TEMPSENSOR (1)(2)
 - LL_ADC_CHANNEL_VBAT (1)

Return value:

- Returned: value can be one of the following values:
 - LL_ADC_CHANNEL_0
 - LL_ADC_CHANNEL_1
 - LL_ADC_CHANNEL_2
 - LL_ADC_CHANNEL_3
 - LL_ADC_CHANNEL_4
 - LL_ADC_CHANNEL_5
 - LL_ADC_CHANNEL_6
 - LL_ADC_CHANNEL_7
 - LL_ADC_CHANNEL_8
 - LL_ADC_CHANNEL_9
 - LL_ADC_CHANNEL_10
 - LL_ADC_CHANNEL_11
 - LL_ADC_CHANNEL_12
 - LL_ADC_CHANNEL_13
 - LL_ADC_CHANNEL_14
 - LL_ADC_CHANNEL_15
 - LL_ADC_CHANNEL_16
 - LL_ADC_CHANNEL_17
 - LL_ADC_CHANNEL_18

Notes:

- The channel parameter can be, additionally to a value defined from parameter definition of a ADC internal channel (LL_ADC_CHANNEL_VREFINT, LL_ADC_CHANNEL_TEMPSENSOR, ...), a value defined from parameter definition of ADC external channel (LL_ADC_CHANNEL_1, LL_ADC_CHANNEL_2, ...) or a value from functions where a channel number is returned from ADC registers.

__LL_ADC_IS_CHANNEL_INTERNAL_AVAILABLE
Description:

- Helper macro to determine whether the internal channel selected is available on the ADC instance selected.

Parameters:

- __ADC_INSTANCE__: ADC instance
- __CHANNEL__: This parameter can be one of the following values:
 - LL_ADC_CHANNEL_VREFINT (1)
 - LL_ADC_CHANNEL_TEMPSENSOR (1)(2)
 - LL_ADC_CHANNEL_VBAT (1)

Return value:

- Value: "0" if the internal channel selected is not available on the ADC instance selected. Value "1" if the internal channel selected is available on the ADC instance selected.

Notes:

- The channel parameter must be a value defined from parameter definition of a ADC internal channel (LL_ADC_CHANNEL_VREFINT, LL_ADC_CHANNEL_TEMPSENSOR, ...), must not be a value defined from parameter definition of ADC external channel (LL_ADC_CHANNEL_1, LL_ADC_CHANNEL_2, ...) or a value from functions where a channel number is returned from ADC registers, because internal and external channels share the same channel number in ADC registers. The differentiation is made only with parameters definitions of driver.

__LL_ADC_ANALOGWD_CHANNEL_GROUP

Description:

- Helper macro to define ADC analog watchdog parameter: define a single channel to monitor with analog watchdog from sequencer channel and groups definition.

Parameters:

- `__CHANNEL__`: This parameter can be one of the following values:
 - `LL_ADC_CHANNEL_0`
 - `LL_ADC_CHANNEL_1`
 - `LL_ADC_CHANNEL_2`
 - `LL_ADC_CHANNEL_3`
 - `LL_ADC_CHANNEL_4`
 - `LL_ADC_CHANNEL_5`
 - `LL_ADC_CHANNEL_6`
 - `LL_ADC_CHANNEL_7`
 - `LL_ADC_CHANNEL_8`
 - `LL_ADC_CHANNEL_9`
 - `LL_ADC_CHANNEL_10`
 - `LL_ADC_CHANNEL_11`
 - `LL_ADC_CHANNEL_12`
 - `LL_ADC_CHANNEL_13`
 - `LL_ADC_CHANNEL_14`
 - `LL_ADC_CHANNEL_15`
 - `LL_ADC_CHANNEL_16`
 - `LL_ADC_CHANNEL_17`
 - `LL_ADC_CHANNEL_18`
 - `LL_ADC_CHANNEL_VREFINT (1)`
 - `LL_ADC_CHANNEL_TEMPSENSOR (1)(2)`
 - `LL_ADC_CHANNEL_VBAT (1)`
- `__GROUP__`: This parameter can be one of the following values:
 - `LL_ADC_GROUP_REGULAR`
 - `LL_ADC_GROUP_INJECTED`
 - `LL_ADC_GROUP_REGULAR_INJECTED`

Return value:

- Returned: value can be one of the following values:
 - LL_ADC_AWD_DISABLE
 - LL_ADC_AWD_ALL_CHANNELS_REG
 - LL_ADC_AWD_ALL_CHANNELS_INJ
 - LL_ADC_AWD_ALL_CHANNELS_REG_INJ
 - LL_ADC_AWD_CHANNEL_0_REG
 - LL_ADC_AWD_CHANNEL_0_INJ
 - LL_ADC_AWD_CHANNEL_0_REG_INJ
 - LL_ADC_AWD_CHANNEL_1_REG
 - LL_ADC_AWD_CHANNEL_1_INJ
 - LL_ADC_AWD_CHANNEL_1_REG_INJ
 - LL_ADC_AWD_CHANNEL_2_REG
 - LL_ADC_AWD_CHANNEL_2_INJ
 - LL_ADC_AWD_CHANNEL_2_REG_INJ
 - LL_ADC_AWD_CHANNEL_3_REG
 - LL_ADC_AWD_CHANNEL_3_INJ
 - LL_ADC_AWD_CHANNEL_3_REG_INJ
 - LL_ADC_AWD_CHANNEL_4_REG
 - LL_ADC_AWD_CHANNEL_4_INJ
 - LL_ADC_AWD_CHANNEL_4_REG_INJ
 - LL_ADC_AWD_CHANNEL_5_REG
 - LL_ADC_AWD_CHANNEL_5_INJ
 - LL_ADC_AWD_CHANNEL_5_REG_INJ
 - LL_ADC_AWD_CHANNEL_6_REG
 - LL_ADC_AWD_CHANNEL_6_INJ
 - LL_ADC_AWD_CHANNEL_6_REG_INJ
 - LL_ADC_AWD_CHANNEL_7_REG
 - LL_ADC_AWD_CHANNEL_7_INJ
 - LL_ADC_AWD_CHANNEL_7_REG_INJ
 - LL_ADC_AWD_CHANNEL_8_REG
 - LL_ADC_AWD_CHANNEL_8_INJ
 - LL_ADC_AWD_CHANNEL_8_REG_INJ
 - LL_ADC_AWD_CHANNEL_9_REG
 - LL_ADC_AWD_CHANNEL_9_INJ
 - LL_ADC_AWD_CHANNEL_9_REG_INJ
 - LL_ADC_AWD_CHANNEL_10_REG
 - LL_ADC_AWD_CHANNEL_10_INJ
 - LL_ADC_AWD_CHANNEL_10_REG_INJ
 - LL_ADC_AWD_CHANNEL_11_REG
 - LL_ADC_AWD_CHANNEL_11_INJ
 - LL_ADC_AWD_CHANNEL_11_REG_INJ
 - LL_ADC_AWD_CHANNEL_12_REG
 - LL_ADC_AWD_CHANNEL_12_INJ
 - LL_ADC_AWD_CHANNEL_12_REG_INJ
 - LL_ADC_AWD_CHANNEL_13_REG
 - LL_ADC_AWD_CHANNEL_13_INJ
 - LL_ADC_AWD_CHANNEL_13_REG_INJ
 - LL_ADC_AWD_CHANNEL_14_REG
 - LL_ADC_AWD_CHANNEL_14_INJ

- LL_ADC_AWD_CHANNEL_14_REG_INJ
- LL_ADC_AWD_CHANNEL_15_REG
- LL_ADC_AWD_CHANNEL_15_INJ
- LL_ADC_AWD_CHANNEL_15_REG_INJ
- LL_ADC_AWD_CHANNEL_16_REG
- LL_ADC_AWD_CHANNEL_16_INJ
- LL_ADC_AWD_CHANNEL_16_REG_INJ
- LL_ADC_AWD_CHANNEL_17_REG
- LL_ADC_AWD_CHANNEL_17_INJ
- LL_ADC_AWD_CHANNEL_17_REG_INJ
- LL_ADC_AWD_CHANNEL_18_REG
- LL_ADC_AWD_CHANNEL_18_INJ
- LL_ADC_AWD_CHANNEL_18_REG_INJ
- LL_ADC_AWD_CH_VREFINT_REG (1)
- LL_ADC_AWD_CH_VREFINT_INJ (1)
- LL_ADC_AWD_CH_VREFINT_REG_INJ (1)
- LL_ADC_AWD_CH_TEMPSENSOR_REG (1)(2)
- LL_ADC_AWD_CH_TEMPSENSOR_INJ (1)(2)
- LL_ADC_AWD_CH_TEMPSENSOR_REG_INJ (1)(2)
- LL_ADC_AWD_CH_VBAT_REG (1)
- LL_ADC_AWD_CH_VBAT_INJ (1)
- LL_ADC_AWD_CH_VBAT_REG_INJ (1)

Notes:

- To be used with function LL_ADC_SetAnalogWDMonitChannels().
Example: LL_ADC_SetAnalogWDMonitChannels(ADC1, LL_ADC_AWD1, __LL_ADC_ANALOGWD_CHANNEL_GROUP(LL_ADC_CHANNEL4, LL_ADC_GROUP_REGULAR))

__LL_ADC_ANALOGWD_SET_THRESHOLD_RESOLUTION

Description:

- Helper macro to set the value of ADC analog watchdog threshold high or low in function of ADC resolution, when ADC resolution is different of 12 bits.

Parameters:

- __ADC_RESOLUTION__: This parameter can be one of the following values:
 - LL_ADC_RESOLUTION_12B
 - LL_ADC_RESOLUTION_10B
 - LL_ADC_RESOLUTION_8B
 - LL_ADC_RESOLUTION_6B
- __AWD_THRESHOLD__: Value between Min_Data=0x000 and Max_Data=0xFFFF

Return value:

- Value: between Min_Data=0x000 and Max_Data=0xFFFF

Notes:

- To be used with function LL_ADC_SetAnalogWDThresholds(). Example, with a ADC resolution of 8 bits, to set the value of analog watchdog threshold high (on 8 bits): LL_ADC_SetAnalogWDThresholds (< ADCx param >, __LL_ADC_ANALOGWD_SET_THRESHOLD_RESOLUTION(LL_ADC_RESOLUTION_8B, <threshold_value_8_bits>));

__LL_ADC_ANALOGWD_GET_THRESHOLD_RESOLUTION

Description:

- Helper macro to get the value of ADC analog watchdog threshold high or low in function of ADC resolution, when ADC resolution is different of 12 bits.

Parameters:

- `__ADC_RESOLUTION__`: This parameter can be one of the following values:
 - `LL_ADC_RESOLUTION_12B`
 - `LL_ADC_RESOLUTION_10B`
 - `LL_ADC_RESOLUTION_8B`
 - `LL_ADC_RESOLUTION_6B`
- `__AWD_THRESHOLD_12_BITS__`: Value between `Min_Data=0x000` and `Max_Data=0xFFF`

Return value:

- Value: between `Min_Data=0x000` and `Max_Data=0xFFF`

Notes:

- To be used with function `LL_ADC_GetAnalogWDThresholds()`. Example, with a ADC resolution of 8 bits, to get the value of analog watchdog threshold high (on 8 bits): `< threshold_value_6_bits > = __LL_ADC_ANALOGWD_GET_THRESHOLD_RESOLUTION(LL_ADC_RESOLUTION_8B, LL_ADC_GetAnalogWDThresholds(<ADCx param>, LL_ADC_AWD_THRESHOLD_HIGH) );`

__LL_ADC_MULTI_CONV_DATA_MASTER_SLAVE

Description:

- Helper macro to get the ADC multimode conversion data of ADC master or ADC slave from raw value with both ADC conversion data concatenated.

Parameters:

- `__ADC_MULTI_MASTER_SLAVE__`: This parameter can be one of the following values:
 - `LL_ADC_MULTI_MASTER`
 - `LL_ADC_MULTI_SLAVE`
- `__ADC_MULTI_CONV_DATA__`: Value between `Min_Data=0x000` and `Max_Data=0xFFF`

Return value:

- Value: between `Min_Data=0x000` and `Max_Data=0xFFF`

Notes:

- This macro is intended to be used when multimode transfer by DMA is enabled: refer to function `LL_ADC_SetMultiDMATransfer()`. In this case the transferred data need to be processed with this macro to separate the conversion data of ADC master and ADC slave.

__LL_ADC_COMMON_INSTANCE

Description:

- Helper macro to select the ADC common instance to which is belonging the selected ADC instance.

Parameters:

- `__ADCx__`: ADC instance

Return value:

- ADC: common register instance

Notes:

- ADC common register instance can be used for: Set parameters common to several ADC instancesMultimode (for devices with several ADC instances) Refer to functions having argument "ADCxy_COMMON" as parameter.

__LL_ADC_IS_ENABLED_ALL_COMMON_INSTANCE

Description:

- Helper macro to check if all ADC instances sharing the same ADC common instance are disabled.

Parameters:

- **__ADCXY_COMMON__**: ADC common instance (can be set directly from CMSIS definition or by using helper macro)

Return value:

- Value: "0" if all ADC instances sharing the same ADC common instance are disabled. Value "1" if at least one ADC instance sharing the same ADC common instance is enabled.

Notes:

- This check is required by functions with setting conditioned to ADC state: All ADC instances of the ADC common group must be disabled. Refer to functions having argument "ADCxy_COMMON" as parameter. On devices with only 1 ADC common instance, parameter of this macro is useless and can be ignored (parameter kept for compatibility with devices featuring several ADC common instances).

__LL_ADC_DIGITAL_SCALE

Description:

- Helper macro to define the ADC conversion data full-scale digital value corresponding to the selected ADC resolution.

Parameters:

- **__ADC_RESOLUTION__**: This parameter can be one of the following values:
 - LL_ADC_RESOLUTION_12B
 - LL_ADC_RESOLUTION_10B
 - LL_ADC_RESOLUTION_8B
 - LL_ADC_RESOLUTION_6B

Return value:

- ADC: conversion data equivalent voltage value (unit: mVolt)

Notes:

- ADC conversion data full-scale corresponds to voltage range determined by analog voltage references Vref+ and Vref- (refer to reference manual).

__LL_ADC_CONVERT_DATA_RESOLUTION

Description:

- Helper macro to convert the ADC conversion data from a resolution to another resolution.

Parameters:

- **__DATA__**: ADC conversion data to be converted
- **__ADC_RESOLUTION_CURRENT__**: Resolution of the data to be converted This parameter can be one of the following values:
 - LL_ADC_RESOLUTION_12B
 - LL_ADC_RESOLUTION_10B
 - LL_ADC_RESOLUTION_8B
 - LL_ADC_RESOLUTION_6B
- **__ADC_RESOLUTION_TARGET__**: Resolution of the data after conversion This parameter can be one of the following values:
 - LL_ADC_RESOLUTION_12B
 - LL_ADC_RESOLUTION_10B
 - LL_ADC_RESOLUTION_8B
 - LL_ADC_RESOLUTION_6B

Return value:

- ADC: conversion data to the requested resolution

__LL_ADC_CALC_DATA_TO_VOLTAGE

Description:

- Helper macro to calculate the voltage (unit: mVolt) corresponding to a ADC conversion data (unit: digital value).

Parameters:

- __VREFANALOG_VOLTAGE__: Analog reference voltage (unit mV)
- __ADC_DATA__: ADC conversion data (resolution 12 bits) (unit: digital value).
- __ADC_RESOLUTION__: This parameter can be one of the following values:
 - LL_ADC_RESOLUTION_12B
 - LL_ADC_RESOLUTION_10B
 - LL_ADC_RESOLUTION_8B
 - LL_ADC_RESOLUTION_6B

Return value:

- ADC: conversion data equivalent voltage value (unit: mVolt)

Notes:

- Analog reference voltage (Vref+) must be either known from user board environment or can be calculated using ADC measurement and ADC helper macro __LL_ADC_CALC_VREFANALOG_VOLTAGE().

__LL_ADC_CALC_VREFANALOG_VOLTAGE

Description:

- Helper macro to calculate analog reference voltage (Vref+) (unit: mVolt) from ADC conversion data of internal voltage reference VrefInt.

Parameters:

- __VREFINT_ADC_DATA__: ADC conversion data (resolution 12 bits) of internal voltage reference VrefInt (unit: digital value).
- __ADC_RESOLUTION__: This parameter can be one of the following values:
 - LL_ADC_RESOLUTION_12B
 - LL_ADC_RESOLUTION_10B
 - LL_ADC_RESOLUTION_8B
 - LL_ADC_RESOLUTION_6B

Return value:

- Analog: reference voltage (unit: mV)

Notes:

- Computation is using VrefInt calibration value stored in system memory for each device during production. This voltage depends on user board environment: voltage level connected to pin Vref+. On devices with small package, the pin Vref+ is not present and internally bonded to pin Vdda. On this STM32 series, calibration data of internal voltage reference VrefInt corresponds to a resolution of 12 bits, this is the recommended ADC resolution to convert voltage of internal voltage reference VrefInt. Otherwise, this macro performs the processing to scale ADC conversion data to 12 bits.

__LL_ADC_CALC_TEMPERATURE_TYP_PARAMS

Description:

- Helper macro to calculate the temperature (unit: degree Celsius) from ADC conversion data of internal temperature sensor.

Parameters:

- **__TEMPSENSOR_TYP_AVGSLOPE__**: Device datasheet data Temperature sensor slope typical value (unit uV/DegCelsius). On STM32F4, refer to device datasheet parameter "Avg_Slope".
- **__TEMPSENSOR_TYP_CALX_V__**: Device datasheet data Temperature sensor voltage typical value (at temperature and Vref+ defined in parameters below) (unit mV). On STM32F4, refer to device datasheet parameter "V25".
- **__TEMPSENSOR_CALX_TEMP__**: Device datasheet data Temperature at which temperature sensor voltage (see parameter above) is corresponding (unit mV)
- **__VREFANALOG_VOLTAGE__**: Analog voltage reference (Vref+) voltage (unit mV)
- **__TEMPSENSOR_ADC_DATA__**: ADC conversion data of internal temperature sensor (unit digital value).
- **__ADC_RESOLUTION__**: ADC resolution at which internal temperature sensor voltage has been measured. This parameter can be one of the following values:
 - **LL_ADC_RESOLUTION_12B**
 - **LL_ADC_RESOLUTION_10B**
 - **LL_ADC_RESOLUTION_8B**
 - **LL_ADC_RESOLUTION_6B**

Return value:

- Temperature: (unit: degree Celsius)

Notes:

- Computation is using temperature sensor typical values (refer to device datasheet). Calculation formula: $Temperature = (TS_TYP_CALX_VOLT(uV) - TS_ADC_DATA * Conversion_uV) / Avg_Slope + CALx_TEMP$ with TS_ADC_DATA = temperature sensor raw data measured by ADC (unit: digital value) Avg_Slope = temperature sensor slope (unit: uV/Degree Celsius) $TS_TYP_CALx_VOLT$ = temperature sensor digital value at temperature $CALx_TEMP$ (unit: mV) Caution: Calculation relevancy under reserve the temperature sensor of the current device has characteristics in line with datasheet typical values. If temperature sensor calibration values are available on on this device (presence of macro **__LL_ADC_CALC_TEMPERATURE()**), temperature calculation will be more accurate using helper macro **__LL_ADC_CALC_TEMPERATURE()**. As calculation input, the analog reference voltage (Vref+) must be defined as it impacts the ADC LSB equivalent voltage. Analog reference voltage (Vref+) must be either known from user board environment or can be calculated using ADC measurement and ADC helper macro **__LL_ADC_CALC_VREFANALOG_VOLTAGE()**. ADC measurement data must correspond to a resolution of 12bits (full scale digital value 4095). If not the case, the data must be preliminarily rescaled to an equivalent resolution of 12 bits.

Common write and read registers Macros

LL_ADC_WriteReg

Description:

- Write a value in ADC register.

Parameters:

- **__INSTANCE__**: ADC Instance
- **__REG__**: Register to be written
- **__VALUE__**: Value to be written in the register

Return value:

- None

LL_ADC_ReadReg

Description:

- Read a value in ADC register.

Parameters:

- `__INSTANCE__`: ADC Instance
- `__REG__`: Register to be read

Return value:

- Register: value